בס"ד

בותרות

מושגים והערות חשובות

שאלות לדוג'

סיכום – מערכות הפעלה

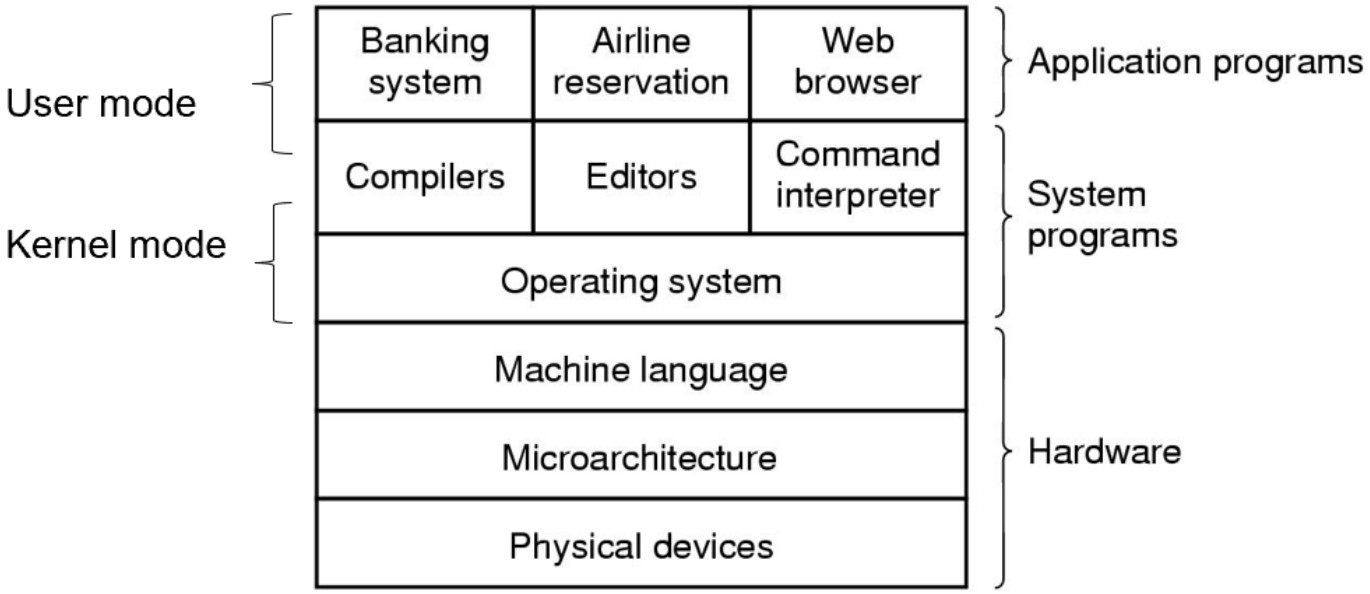
תמלול ההרצאות של אירנה ליפוב, מוגבר, 2025ב

נכתב ע"י ספיר עדינה חביבזדה

שיעור 1 – הקדמה , מבנה מערכת ההפעלה

← **מבנה מערכת מחשב:**

- מחשב כולל שלושה מרכיבים : חומרה ,תוכנות מערכת ,תוכנות יישום (אפליקציות)
- מערכת ההפעלה היא **הממשק** שבין המשתמש והיישומים לבין החומרה



1. **application programs = אפליקציות**

אפליקציה = תוכנות פשוטות שאנו משתמשים בהן ביומיום – משחקים, דפדפנים, אפליקציות של בנקים וכו'.
מצד אחד - הן עושות עבודה חשובה אך אנו לא יודעים מי כתב אותן וקשה לנו לסמוך על יצרן האפליקציות. מי שכתב את האפליקציה יכול להיות שניגש לזיכרון המחשב ולחומרה וידרוס שם מידע חשוב, או יצור גישה למידע האישי שלי, יעשה איזושהי העמסה על סיפי CPU וכו'.
מצד שני - אני עדיין רוצה להשתמש בשירות שהאפליקציה נותנת לי.
לכן אנו ניתן לאפליקציה הרשאות מוגבלות בגישה שלהם לחומרה שעל המחשב שלנו, ויהיה לנו קשה "לסמוך" עליהן.

2. **System programs = תוכנות מערכת**

מהדרים = compilers = כשיש תוכנית כתובה צריך לקמפל אותה לשפת המכונה. מהדר זהו יישום שמעביר משפת עילית לשפת המכונה – עושה **קומפילציה**. הוא גם עושה **אופטימיזציה** , עושה תרגום לשפת מכונה שרצה במחשב ותשתמש במינימום זיכרון ותרוץ בזמן הקצר ביותר. המהדר יודע את החומרה שיש לנו במחשב ויודע מה העומס על החומרה, וכך הוא יודע לתרגם את הקוד בצורה חכמה וטובה וכך אנו מקבלים ביצועים טובים. אז החלק העיקרי של המהדר הוא אופטימיזציה ושיפור ביצועים. אנו סומכים על המהדרים וצריך לתת להם הרבה הרשאות לעשות דברים מעמיקים עם החומרה והעומס שעליה.

← *מסקנה – ההבדל בין האפליקציות לתוכנות סיסטם הוא מבחינת רמת הרשאות שכל אחת מקבלת:*
האפליקציות מקבלות הרשאות נמוכות,
תוכנות הסיסטם מקבלות הרשאות גבוהות.

מפרשים = command interpreter = דומה למהדר, אך ההבדל ביניהם הוא:
מהדר - לוקח הוראות ומתרגם את הקוד בשלמותו ורק אח"כ מקבל את כל הקוד בשפת המכונה ומריץ אותו.
מפרש - לוקח את הקוד שורה שורה בנפרד ומתרגם אותה לשפת מכונה ומיד מריץ, כלומר לא מחכה להידור של כל הקוד בשלמותו.
במהדר, החיסרון הוא שהקוד יכול לחזור על עצמו. כלומר אם כתבנו קוד יש שורות בקוד שנחזור עליהן כמה פעמים (לדוג' בקריאות חוזרות לאותה הפונ') והמהדר יתרגם כל חזרה כזו שוב ושוב – וזה בזבוז של זמן.
היתרון הוא שלא צריך לחכות לקימפול של הכל, כי כל שורה כותבים ומיד מקמפלים.

← *מפרשים ומהדרים , כדי לעשות את התרגומים בצורה חכמה (=שיפור של הפרמורמנס) שניהם צריכים לדעת **מה מצב החומרה ומה העומס עליה** כדי לעשות את ההידור בצורה כזו שאנו מקבלים עדכון מבחינת כל מיני שימושים ברכיבים שונים של חומרה. יש לנו רצון לתת לשניהם הרשאות גבוהות לחומרה (בניגוד לאפליקציות שאותן נגביל).*

Editors = הם נמצאים יחד עם מהדר או מפרש ב-ide, אך לא צריך לתת להם הרשאות גבוהות.

מערכות הפעלה = operating system – מתווכת בין התוכנה (הקוד, ההוראות) לחומרה (המכשיר שמבצע את ההוראות). ניתן לה הרשאות גבוהות.

3. Hardware = חומרה

מכילה את:

א. התקנים פיזיים – CPU (מעבד), דיסק , GPU (כרטיס גרפי), RAM (זיכרון), SSD (אחסון קבוע), מדפסת , עכבר, מקלדת ...

ב. שפת המכונה – שפת החומרה שמריצה את הקוד

ג. מיקרוארכיטקטורה – רמה מעל המכשירים עצמם שמתארת את הקשר והזרימה הכבדה בין החלקים השונים שיש בחומרה.

← בכל פעם שתוכנית רצה מאחת מ3 הקבוצות שדיברנו עליהן:

- אם זו תוכנית שהיא אפליקציה –

לא ניתן לה הרשאות גבוהות, מחשש שמי שכתב את האפליקציה לא עשה עבודה נקייה והיא יכולה לפגוע במחשב שלנו. ההרשאות לאפליקציות הן מוגבלות. כשהן רצות על המעבד למשל, וזה לא משנה מה מספר הליבות במעבד – המעבד לא יכול לעשות דברים רגישים, ואז הוא עובד בuser mode (מצב משתמש), זהו מצב שבו הCPU מריץ קוד בהרשאות מוגבלות. ואז יש הרבה דברים שה CPU לא יכול לבצע.

- אם זו תוכנית שהיא תוכנת מערכת –

ניתן לה הרשאות גבוהות, וכשהתוכנה הזו מתבצעת במעבד אנו רוצים גישה לדברים רגישים בחומרה, ולכן המצב של המעבד נקרא kernal mode (מצב ליבה), שהו מצב שהCPU מריץ קוד בהרשאות גבוהות.

מכאן, בהתאם למה שהמעבד מבצע הוא יכול להיות במצב של user mode (הרשאות מוגבלות) או במצב של kernal mode (הרשאות גבוהות) . בעצם המעבד צריך לדעת לעשות את המעבר הזה בין המצבים בהתאם לתוכנה שהוא מריץ – והוא עושה זאת ע"י קריאות מערכת (system call).

← **מצבי פעולה:**

- **User Mode** - מצב בו תוכנות רגילות רצות עם גישה מוגבלת.
  - גישה מוגבלת: מצב משתמש מיועד להפעלת קוד יישום עם גישה מוגבלת למשאבי מערכת. תוכניות הפועלות במצב משתמש אינן יכולות לגשת ישירות לחומרה או לשנות הגדרות מערכת מרכזיות.
  - ממשק קריאת מערכת: כאשר יישום זקוק לפעולה מועדפת (כגון קריאה מקובץ או הקצאת זיכרון), הוא משתמש בקריאת מערכת, אשר מפעילה מעבר למצב ליבה באמצעות מלכודת מבוקרת.
  - הגנת זיכרון: התוכנית מבודדת בתוך מרחב הזיכרון שלה, מה שמגן עליה מפני השפעה על תוכניות אחרות או על ליבת מערכת ההפעלה עצמה.
  - בטיחות ברמה נמוכה: שגיאות או באגים ביישומי מצב משתמש משפיעים רק על היישום עצמו, לא על המערכת כולה

- **Kernel Mode** - מצב בו פועלת מערכת ההפעלה עם גישה מלאה למשאבים
  - הרשאה מלאה: מצב ליבה מאפשר גישה מלאה לכל משאבי המערכת, כולל זיכרון, התקני חומרה, הוראות CPU ויציאות קלט/פלט.
  - בקרת חומרה ישירה: הליבה יכולה לתקשר ישירות עם חומרה דרך מנהלי התקנים, לנהל את זיכרון המערכת ולשלוט בתהליכים.
  - פונקציות קריטיות של מערכת ההפעלה: כל פונקציות הליבה של מערכת ההפעלה, כגון תזמון תהליכים, ניהול זיכרון ופעולות קלט/פלט של המכשיר, מבוצעות במצב ליבה.
  - מעבר מצב (מלכודת): כאשר יישום מבצע קריאת מערכת, הוא מפעיל מלכודת כדי להיכנס למצב ליבה. לאחר עיבוד קריאת המערכת, הליבה חוזרת למצב משתמש.

- הפרדה בין המצבים נועדה להגן על המערכת מפני קריסות/בעיות אבטחה

תכונה	מצב משתמש	מצב ליבה
רמת הרשאה	הרשאה נמוכה (גישה מוגבלת)	הרשאה גבוהה (גישה מלאה למערכת)
גישה לחומרה	אין גישה ישירה לחומרה או להתקני קלט/פלט	גישה מלאה לחומרה ולהתקני קלט/פלט
גישה לזיכרון	מוגבל לזיכרון במרחב המשתמש	גישה מלאה לכל הזיכרון, כולל מרחב המשתמש
השפעת שגיאה	שגיאות משפיעות רק על האפליקציה הבודדת	שגיאות עלולות לגרום לקריסה של המערכת כולה
הקשר ביצוע	מבצע קוד אפליקציה	מבצע קוד מערכת הפעלה מרכזי ופעולות קריטיות
החלפת מצבים	לא ניתן לעבור ישירות למצב ליבה; יש להשתמש בקריאה למערכת	ניתן לעבור למצב משתמש לאחר השלמת משימות

← **<u>מהי מערכת הפעלה:</u>**

- **מכונה מורחבת (Extended Machine)** - מסתירה את פרטי החומרה, מספקת ממשק פשוט
- **מנהל משאבים (Resource Manager) -** מקצה משאבים כמו מעבד, זיכרון, התקני קלט/פלט
- מאפשרת **ריבוי משימות** על ידי שיתוף המשאבים ב-2 דרכים:
  - **בזמן (Time-sharing)**
  - **במרחב (Space-sharing)**

מה זו מערכת הפעלה?

1. זו מכונה מורחבת

שכבת ביניים שמסתירה את המורכבות של חומרת המחשב, ובכך מקלה על המשתמשים והתוכנות לתקשר עם המחשב.
הקוד שאנו כותבים בתוכנה בשפה עילית ירוץ בסופו של דבר בחומרה שיודעת לבצע אותו.
*מערכת ההפעלה מהווה **שכבת תיווך** בין התוכנה לבין החומרה.*

<u>מה זה חומרה?</u>

CPU זו יחידת העיבוד המרכזית המריצה את הקוד.
GPU אלה כרטיסים גרפיים שמרצים קוד גרפי של משחקים למשל, או AI.
מכשירים נלווים – דיסקים, מדפסות, מקלדות... מרכיבים שמבצעים את מה שאנו מריצים במחשב.
מערכת ההפעלה היא מכונה מורחבת שיודעת להריץ, ללא שום קשר איזו חומרה יש לנו, יודעת לספק API מורחב שהוא בלי שום קשר לטיב החומרה יודע להריץ, לבצע אופטימיזציה ולספק כל מיני שירותים שלא קשורים לסוג החומרה הספציפי ויודע לגרום לכל התוכנות לעבוד. בזכות מערכת ההפעלה, אותו קוד יכול לרוץ גם על מחשב נייד, גם על שרת וגם על טאבלט – בלי שנצטרך לשנות אותו. מערכת ההפעלה מתרגמת את הפקודות לפעולות המתאימות לסוג החומרה.

2. זהו מנהל משאבים

כל תוכנית מקבלת זמן שימוש במשאב
כל תוכנית מקבלת מקום (זיכרון) במשאב
תפקיד נוסף של מערכת ההפעלה הוא הגנה על עבודה משותפת.
יכולות להיות כמה תוכנות שישתמשו באותו אזור זיכרון, ואנו חושדים שתוכנית אחת תכתוב באותו אזור זיכרון בצורה לא מוגנת כך שכמה תוכניות יכתבו בו"ז באותו איזור זיכרון ואז ייווצר מצב של **corruption** (השחתת נתונים), כלומר אזור הזיכרון יהיה לא עקבי, כי כמה תוכנות יכתבו בו יחד ואי אפשר להתחייב מה תהיה התוצאה שם. בנוסף, אם תוכנית אחת כותבת ותוכנית אחרת קוראת, יכול להיות שאין הלימה בין שתי הפעולות ואז התוכנית שקוראת מקבלת מידע חלקי שלא נכתב בשלמותו.
מערכת ההפעלה מספקת גישה לזיכרונות (בדיסק או בRAM) כדי למנוע את המצב של קורפשן.
מערכת ההפעלה מקבלת מאיתנו הרשאות גבוהות.
לסיכום, תפקידי מערכת ההפעלה הם:

1. הרצת הקוד לחומרה , ללא תלות בסוג החומרה.
2. הגנה על עבודה משותפת של כמה תוכנות באותו אזור זיכרון, ומניעת מצב של corruption בזיכרון

← **סקירה של חומרה:**

- רכיבים עיקריים: CPU, RAM , דיסק מסך , Bus ,
- **מעבדים מודרניים** : מרובי ליבות, צינורות עיבוד ,(Pipeline) מעבדים סקלריים
- **היררכיית זיכרון :** Registers → Cache → RAM → Disk
- ניהול Cache כולל החלטות על מה להכניס, להחליף ולשמור

← **התקני קלט/פלט:**

- פעולות I/O כוללות הפעלת התקן והמתנה לפסיקה(Interrupt)
- פסיקה גורמת ל Kernel-לעצור את מה שעשה ולטפל בפעולה הנדרשת

← **תהליכים ותהליכונים:**

- **Process תהליך :**יישות פעילה שמכילה קוד, זיכרון ונתונים. היא יותר מאשר רק "קובץ רץ "
- **Thread תהליכון :**יחידת ביצוע בתוך תהליך. כל הפתילים משתפים את אותו זיכרון
- תהליכים יוצרים תהליכים אחרים – מבנה של **עץ תהליכים**

היבט	תהליך (Process)	תהליכון (Thread)
הגדרה	תוכנה עצמאית שפועלת בפועל	יחידת הביצוע הקטנה ביותר בתוך תהליך
שיתוף משאבים	יש לו זיכרון ומשאבים נפרדים משלו	משתף זיכרון ומשאבים עם תהליכונים אחרים באותו תהליך
תקורה (Overhead)	תקורה גבוהה יותר עקב ניהול זיכרון ומשאבים נפרדים	תקורה נמוכה יותר – משתף את משאבי התהליך האב
תקשורת	נדרש מנגנון תקשורת בין תהליכים (IPC) כמו צינורות או תורים	מתקשר בקלות עם תהליכונים אחרים בזיכרון משותף
בידוד תקלות	תקלה בתהליך לא משפיעה על תהליכים אחרים	תקלה בתהליכון עשויה להפיל את כל התהליך ואת יתר התהליכונים
זמן יצירה	איטי יותר – דורש הקצאת זיכרון ומשאבים נפרדים	מהיר יותר – לא נדרש להקצות זיכרון נפרד
החלפת הקשר	איטית יותר – כל תהליך עם זיכרון משלו	מהירה יותר – תהליכונים משתפים את אותו מרחב זיכרון
שימושים נפוצים	מתאים להרצת תוכנות נפרדות או משימות בלתי תלויות	מתאים למשימות מקביליות בתוך אותה תוכנה, כמו טיפול במספר לקוחות

כל תוכנה רצה לא כתוכנה נפרדת, אלא בתוך מעטפת הנקראת תהליך (process).

מה זה תהליך?

לדוג' יש לנו 4 תוכנות במחשב. כולן פועלות במקביל. אז אם יש 4 ליבות במחשב ו4 תוכנות, אז כל יישום רץ על ליבה אחרת וכל אחד עושה את העבודה שלו. אך מצד שני, אם אני מריצה 100 תוכנות על המחשב הרי שאין לי 100 ליבות על המחשב אבל עדיין הכל רץ במקביל. איך יכול להיות? כי אף תוכנה לא רצה בנפרד, וישנו תהליך שעוטף אותה.

תהליך הוא כלי בסיסי במערכת ההפעלה. מערכת ההפעלה תופסת את הפעלת התוכנית, ומייצרת אובייקט של תהליך שצריך לבצע את הקוד של התוכנה הזו. הקוד המקומפל של התוכנה מועבר לתהליך שמבצע אותו.

יש במערכת ההפעלה כלי שנקרא מתזמן (Scheduler) שמריץ אלגוריתם שאחראי שהמשתמש יקבל אשליה שכל התוכנות עובדות במקביל. הרכיב אחראי על ניהול זמן הריצה של תהליכים במעבד. זהו מנגנון קריטי שמחליט איזה תהליך ירוץ ומתי, במיוחד כאשר יש מספר תהליכים שמבקשים להשתמש במעבד בו-זמנית.

יש כל מיני מערכות הפעלה. למשל windows היא מערכת הפעלה המיועדת למשתמש פשוט, וכל אחת מנהלת מקביליות שונה עם אלגוריתם שונה.

האלגוריתם הכי פשוט שעושה מקביליות הוא round robin (RR) (בחלק C בציור) שמקצה לכל תהליך **קצבת זמן קבועה מראש שנקראת quata \ קוואנטום \ סלוט \ רובין** . ברגע שתהליך מתחיל לרוץ, הוא מקבל את זמן המעבד למשך אותה קוואטה. אם הוא לא סיים בזמן הזה – הוא מושהה ומועבר לסוף התור, והתהליך הבא נכנס. התהליך הבא בתור יקבל גם הוא זמן קבוע – וחוזר חלילה.

בציור – יש 4 תהליכים. את התהליך A הוא מריץ קוואטה של זמן. הוא עוצר אותו, מתחיל את B , שירוץ גם קוואטה של זמן. וכן הלאה. כל תהליך עבד מעט זמן, וכשכל התהליכים עבדו קוואטה שלזמן מתחילים שוב מהתהליך הראשון מאיפה שעצר ושוב לסלוט זמן. כלומר שכל התהליכים לא עובדים ברצף אלא בחלקים של קוואטה של זמן. העין האנושית של כל אחד לא תופסת את התמונה הזו של פרקי

הזמן כי הם מאוד קצרים, ורואה את הרצת התוכנית כהרצה ברצף. אנחנו לא רואים את ההחלפות בין התהליכים ויש לנו תחושה שהכל עובד ברצף ובאופן מקביל, למרות שפועל לא כך הדבר.

מקביליות מדומה = החלפה מהירה ביון תהליכים, אך יש תקרה לכך.

החלפה = לא ניתן לעשות הנחות על זמן, התהליך לא מודע להשהייתו.

ההחלפה בין התהליכים היא לא קלה ומבזבזת זמן כי לכל תהליך יש זיכרון משלו שבכל החלפה נשמר בצד ונטען מחדש.

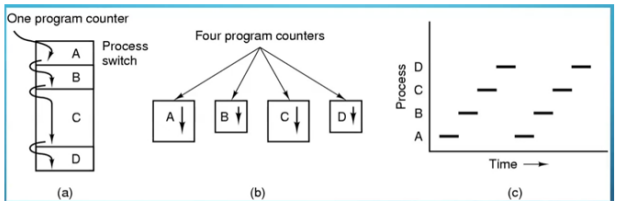
עכשיו מבינים למה צריך תהליכים ? כי אם התוכנות היו רצות כתוכנות, אז מערכת ההפעלה לא הייתה יכולה לגשת אליהן ולבחור את מי להריץ. היא לא הייתה יכולה לעשות את ההחלפות בין ההרצות של התכונות.

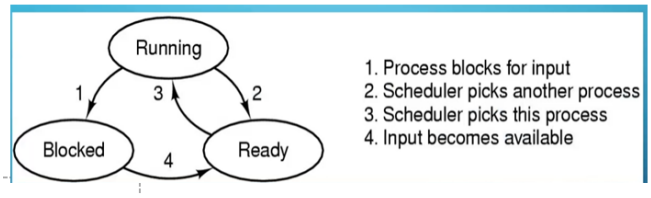
תוכנה אחת לא צריכה לדעת על קיום תוכנה אחרת. ובהתאם, תהליך אחד לא צריך לדעת על תהליך אחר. לכן יש חשש שתוכנה אחת תדרוס תוכנה אחרת, ולכן כל תהליך משתמש באזור זיכרון משלו. ולכן כל תוכנה ותהליך הוא "אנוכי" וחושב שכל הזיכרון הוא רק שלו, ולא אכפת לו מתוכנות ותהליכים אחרים. אז כשמתבצעת ההחלפה בין התהליכים אנו חוששים שתתבצע דריסה של זיכרון של תהליכים שהתרחשו לפניו. לכן כשמתבצעת החלפה בין תהליכים מערכת ההפעלה שומרת את מצב התהליך שעבד עכשיו באוגרים בצד, ובהמשך כשהיא חוזרת להריץ את התהליך הזה היא מטעינה את המידע שנשמר באוגרים בצד באוגרים פיזיים וחוזר חלילה. השמירה של האוגרים והטענת אוגרים חדשים בכל החלפה זה בזבוז של זמן. ניהול התהליכים מבזבז זמן.

ולכן- האם ניקח את הקוואטה ונרצה שיהיה קטן או גדול?

קוואטה גדול – משפר ניצולת מעבד. אך תהיה פגיעה במקביליות המדומה.

קוואטה קטן – משפר מקביליות מדומה. כי תדירות ההחלפות היא גבוהה, אבל אז יהיה בזבוז זמן גדול על ההחלפות ובעקבותיה פגיעה בפרפורמנס.





PCB – process control block. מכיל הרבה הגדרות ותיאורים של התהליך. מכיל:

- מצביע לקוד התהליך - כל תהליך מכיל קוד. המצביע לקוד נמצא בתוך ה-PCB.
- PID – Process Identifier, process ID, מזהה התהליך. המערכת מתזמנת תהליכים על פי הPID שלהם.
- אזור זיכרון לניהול – מערכת ההפעלה יודעת שכשהיא מבצעת החלפות היא צריכה לשמור את מצב התהליך באוגרים, והאזור לשמירה הזה נמצא ב-PCB.

Process state – מצב של תהליך.

- כשתהליך מסתיים הוא לא בהכרח נעלם.
- יכול להיות תהליך אב שממנו נוצרים תהליכים נוספים.

אנחנו לא רוצים לסגור את המסמך הראשי לפני שכל בניו סיימו את העבודה. אז אם תהליך האב הסתיים, הוא עדיין קיים במערכת ההפעלה כי הבנים שלו עוד רצים. יש לזהות מתי התהליך יכול להסתיים ולהיעלם ומתי הוא צריך לחכות למרות שהוא הסתיים כבר, וזאת כדי שהביצועים יהיו טובים ולא יתבזבז זמן יקר של המעבד. לדוג' גם בהמתנה של תהליך להדפסה שלו או לפעולה של המשתמש – חבל לתהליך להמשיך לרוץ עד שזה יקרה, והוא יכול להיות בהשהיה ולהמשיך לרוץ כשיתרחש שינוי.

לכן, לכל תהליך יש מצב (בציור למטה):

Running – התהליך רץ, והמעבד מבצע את הקוד שלו.

Ready – התהליך מוכן, זה המצב שהתהליך נמצא בו בין ההחלפות

Blocked – התהליך בבלוק, בהמתנה. יש כל מיני סוגים של בלוק (לדוג' בהמתנה להדפסה, המתנה לסיום כתיבה של אזור זיכרון....)

התהליכים שעוברים ממצב מוכן לרץ וחוזר חלילה, זה הטווח הקצר.

התהליכים שעוברים ממצב בלוק לרץ בלוק לרץ וחוזר חלילה, זה הטווח הארוך.

חץ 1 – התהליך נעצר בהמתנה לקלט

חץ 2 – המתזמן בחר תהליך אחר בהחלפות

חץ 3 – המתזמן בחר בתהליך הזה להחלפות

חץ 4 – הקלט הפך זמין

← תקשורת בין תהליכים:

- דרכים עיקריות:

Pipes צינורות – חיבור ישיר בין שני תהליכים

Signals אותות – כמו kill -SIGKILL, kill -SIGSTOP

System Calls - קריאות מערכת לניהול תהליכים

← קריאות מערכת:(System Calls)

- מאפשרות לתהליכים לבצע פעולות מערכת כמו:
  - יצירה (fork)
  - הרצה (execve)
  - המתנה (waitpid)
  - סיום(exit)
  - קריאה (read)
  - כתיבה (write)
  - הריגה(kill)

קריאת מערכת (System Call**)** היא מנגנון המאפשר לתוכנית הפועלת ב-user mode לבקש שירותים מליבה של מערכת ההפעלה, אשר פועלת במצב ליבה. באמצעות קריאות מערכת, יישומים יכולים לבצע בבטחה פעולות הדורשות אינטראקציה ישירה עם חומרה, זיכרון, קבצים ומשאבי מערכת קריטיים אחרים, שאליהם אין להם גישה ישירה עקב אילוצי אבטחה ויציבות.

היבטים מרכזיים של קריאות מערכת:

1. מטרה: קריאות מערכת מאפשרות לתוכניות לבצע משימות הדורשות את הרשאות מערכת ההפעלה, כגון גישה לקבצים, הקצאת זיכרון, ניהול תהליכים וטיפול בתקשורת רשת.
2. מעבר ממצב user למצב kernel: כאשר תוכנית מבצעת קריאת מערכת, המעבד עובר ממצב משתמש למצב ליבה. לאחר מכן, מערכת ההפעלה מבצעת את המשימה המבוקשת בשם התוכנית וחוזרת למצב משתמש לאחר השלמתה.
3. סוגי קריאות מערכת:
   - בקרת תהליכים: ניהול תהליכים, כגון יצירה או סיום שלהם (למשל, fork(), exec(), exit()).
   - ניהול קבצים: טיפול בקבצים, כולל פתיחה, קריאה, כתיבה וסגירה (למשל open(), read(), write(), close())
   - ניהול התקנים: אינטראקציה עם התקני חומרה, כגון קריאה מהתקן או כתיבה אליו.
   - תחזוקת מידע: אספקת מידע מערכת או שינוי הגדרות מסוימות (למשל, getpid() כדי לקבל מזהה תהליך).

קריאה לדוגמה:

- read(fd, buffer, nbytes) כוללת 11 שלבים, כולל מעבר לקרנל דרך **Trap**
- **Trap** משמש למעבר ממוד משתמש למוד ליבה לצורך ביצוע פעולות רגישות
- write(STDOUT, msg, strlen(msg)) שולחת פלט ישירות למסך דרך הקרנל

← קריאת המערכת read

```
ssize_t read(int file_descriptor, void *buffer, size_t count);
```

- `file_descriptor`: The file descriptor of the file to be read.
- `buffer`: A pointer to the memory location where the data will be stored.
- `count`: The number of bytes to read.

file_descriptor מתאר הקובץ של הקובץ שיש לקרוא.

buffer מצביע למיקום הזיכרון שבו יאוחסנו הנתונים.

Count מספר הביטים לקריאה.

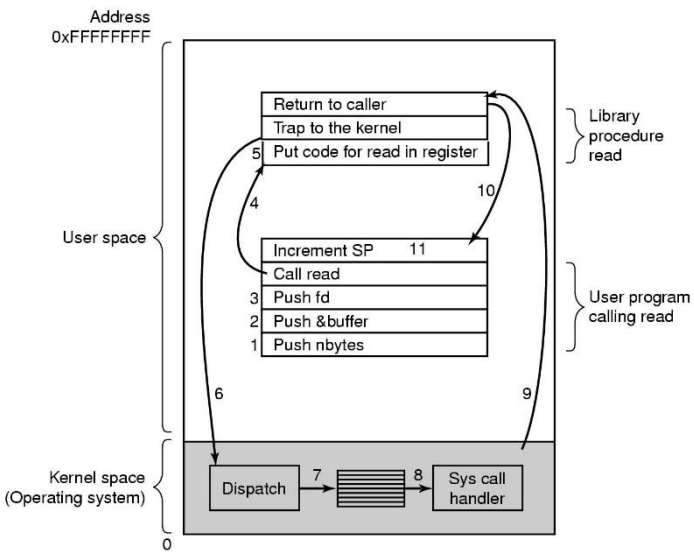
הארגומנטים מועברים במחסנית. הארגומנט האחרון שנשים במחסנית הוא הראשון שנוציא.

אז file_descriptor הוא הראשון שכתוב בחתימה של קריאת המערכת – ולכן הוא האחרון שיכנס למחסנית והראשון שיישלף ממנה. הראשון שנכנס הוא האחרון שיוצא.

בחתימה של קריאת המערכת –

הארגומנטים יכנסו למחסנית בסדר הפוך מהכתוב בחתימה,

וישלפו ממנה על פי הסדר שכתוב בחתימה



11 שלבי יצירת קריאת המערכת read :

1-3. הכנסת הארגומנטים למחסנית (בסדר הפוך מהכתוב בחתימה)

4. קריאה לפונ' עצמה, מתחילים ביצוע. הפונ' עובדת כשהמעבד בuser mode, והמטרה של קריאת המערכת היא להעביר שליטה למערכת ההפעלה כדי שהיא תבצע איזושהי עבודה רגישה , כלומר לעבור לkernel mode.

איך המעבר מתבצע?

5. הכנה של הארגומנטים במערכת ההפעלה בכך שהם מושמים באוגרים, ובנוסף שמירת האוגרים בחישוביים "בצד", והכנת המזהה (טיונינג) של מה שאנו רוצים שמערכת ההפעלה תעשה.

ארגומנטים צריכים לעבור למערכת ההפעלה, וזאת ע"י 2 שיטות:

א. דרך האוגרים (רגיסטרים) – זיכרון המחשב הוא פירמידה והמבנה שלו הוא: (מהגדול האיטי והזול אל הקטן המהיר והיקר) :

הדיסק - הגדול והאיטי ביותר

RAM

זיכרונות מטמון

רגיסטרים, אוגרים – הקטן והמהיר ביותר, וזאת בגלל קרבתם ל-CPU. מספר מצומצם.

כשה-CPU עובד הוא שם את הארגומנטים שהוא קיבל באוגרים כדי שהגישה אליהם תהיה מהירה.

ב. כשהמידע שברשותנו גדול מדי, יכול להיות שלא יהיו מספיק אוגרים כדי להכיל את המידע כי מספר האוגרים מצומצם. ואז נכיל את המידע בצורה שונה – נשים את הארגומנטים בזיכרון מסוים, ורק את הכתובת שבה שמנו את הארגומנטים נעביר למערכת ההפעלה. ישנם אוגרים שמשתמשים בהם לחישובים. מערכת ההפעלה תוכננה בצורה כזו שהיא "לא חושבת אחורה ומתחילה מאפס" ואם היא מקבלת משימה היא תעשה אותה בצורה מושלמת, בלי להתחשב בדברים אחרים, ולכן יכולה להיווצר סיטואציה שהיא דורסת נתונים שהיו לפני כן בזיכרון. לכן את האוגרים החישוביים נשים בצד.

6. **Trap** – פסיקה מיוחדת שבה ניגשים למעבד ומשנים לו את ההרשאות מיוזר מוד לקרנל מוד. טרפ עושה טיונינג של ההרשאות הגבוהות.

עכשיו מערכת ההפעלה מתחילה לעבוד. רכיב ה-dispatch נמצא בקרנל ספייס , מקבל שליטה ובוחר מי יעשה את מה שמערכת ההפעלה נדרשה לעשות.

7. לכל פונקציה יש מספר, ורכיב ה-dispatch מזהה את המספר וניגש לטבלה של כתובות של פונקציות של מערכת ההפעלה שנועדו לביצוע הדברים האלה. אז הרכיב לוקח את המספר הזה, ניגש לטבלה, לוקח את הכתובת של הפונקציה שביקשו להריץ, ומפנה אותה לאוגרים/אזור הזיכרון שבו הארגומנטים שהתקבלו נמצאים .

8. הכל מוכן וניתן להריץ את הפונקציה!

Sys call handle מתחזק את הביצוע שמערכת ההפעלה תבצע. חוזרת הודעת שגיאה, מודפס משהו, מכינים מידע כלשהו שיחזור למשתמש וכו' – פשוט מה שהיה על הפונ' לבצע.

9. הפונקציה מסתיימת. חוזרים מקרנל ספייס ליוזר ספייס. ההרשאות חוזרים להיות מוגבלות.

10-11. מעבר לפונקציה **increment SP** שנמצאת אחרי הסיסטם קול, שהיא מנקה את המחסנית מהארגומנטים ששמנו בה.

SP – זהו המצביע לארגומנטים שבמחסנית. כל פעם שמעבירים ארגומנט הוא מתקדם, וכעת בגלל שזורקים ארגומנטים הוא מתקדם אחורה.

write , read , printf ←

קריאת המערכת **write** קורא מהזיכרון וכותב לקובץ.

קריאת המערכת **read** קורא מהקובץ וכותב לזיכרון.

פונקציית הספרייה **printf** מקבלת מספר לא מוגבל של ארגומנטים, אך הארגומנט הראשון הוא פירמוט - כלומר באיזה צורה אנו רוצים שההדפסה תתבצע. לדוג' אם רוצים מספר ממשי עם נקודה אז נרצה שיופיעו רק 2 ספרות, או שמחרוזת תהיה מיושרת לצד שמאל.

Printf זו פונקציית ספריה, מספריית STDIO.

Printf לא יכולה לכתוב על המסך, כי גם הזיכרון של המסך מוגבל ואם כמות ההדפסה גדולה מדי היא תדרוס דברים אחרים שהודפסו. לכן printf משתמשת בקריאת מערכת , כלומר מערכת ההפעלה היא זו שכותבת למסך בפועל. Printf עושה פירמוט לנתונים ומכין את המחרוזת להדפסה ואז לוקח קריאת מערכת (למשל write) שתעשה את ההדפסה למסך.

גם המסך נחשב כקובץ מיוחד הנקרא STDOUT .

אם נשווה את הביצועים של write ושל printf – של מי יותר טובים (כלומר מהירים יותר)?

של write , כי printf עצמו נעזר ב-write .

← **<u>מערכת קבצים :</u>**

- תמיכה בתיקיות, קבצים, קישורים (Links) , הרכבות(Mount)
- תהליך כולל 3 אזורי זיכרון:
  1. קוד text
  2. משתנים data
  3. מחסנית stack

← **<u>מבני מערכת הפעלה:</u>**

1. **Monolithic מונוליטי**
   - כל הפונקציות באותו קובץ – תלות הדוקה בין הרכיבים
   - אין הסתרת מידע – קשה לתחזוקה ולשדרוג
2. **Layered שכבות**
   - כל שכבה תלויה רק בזו שמתחתיה
   - מתאימה למודלים עם רמות הרשאה
3. **Client-Server שרת-לקוח**
   - קרנל מינימלי, השירותים השונים פועלים כשרתים נפרדים
   - יתרון: קל להרחבה, עדכון ואבטחה
   - חסרון: תקורת תקשורת גבוהה
4. **Microkernel**
   - גרסה מצומצמת של שרת-לקוח
   - הקרנל רק מטפל בהעברת הודעות ושעון – כל השאר בשירותים חיצוניים
5. **Virtual Machine**
   - מדמה מחשב שלם
   - מאפשר הרצת כמה מערכות הפעלה במקביל על חומרה אחת

Exp.	Explicit	Prefix	Exp.	Explicit	Prefix
10^{-3}	0.001	milli	10^{3}	1,000	Kilo
10^{-6}	0.000001	micro	10^{6}	1,000,000	Mega
10^{-9}	0.000000001	nano	10^{9}	1,000,000,000	Giga
10^{-12}	0.000000000001	pico	10^{12}	1,000,000,000,000	Tera
10^{-15}	0.000000000000001	femto	10^{15}	1,000,000,000,000,000	Peta
10^{-18}	0.000000000000000001	atto	10^{18}	1,000,000,000,000,000,000	Exa
10^{-21}	0.000000000000000000001	zepto	10^{21}	1,000,000,000,000,000,000,000	Zetta
10^{-24}	0.000000000000000000000001	yocto	10^{24}	1,000,000,000,000,000,000,000,000	Yotta

← **<u>יחידות מידה:</u>**

Kilo (10^3), Mega (10^6), Giga (10^9)

← **<u>fork</u>**

אנו לא מאפשרים יצירה של תהליך מהקוד או מקריאת מערכת כי זה מסוכן. לכל תהליך זיכרון משלו ואם משתמש\אפליקציה ייתן יותר מדי זיכרון לתהליך או מקום בזיכרון שידרוס תהליך אחר זו בעיה.
Fork היא קריאת מערכת שיוצרת תהליך חדש על ידי שכפול של תהליך קיים.
ביו תהליכים יש היררכיה, משמעותו היא שיש תהליכי אב שמהם נוצרים תהליכים נוספים, תהליכי בן:

- תהליך בן יורש מאביו את **הקוד מהשורה הבאה** (כלומר מהמקום בקוד בו האב היה כאשר נוצר הבן והלאה). כלומר קוד שכבר בוצע הבן לא יורש מאביו.
- תהליכי הבן יורש מאביו את **המשתנים** , אבל כדי לא לנפח יותר מדי את הזיכרון הם לוקחים **רק את אלה שיתבצע בהם שימוש בקוד בו ירשו.**

כל פונ' מורכבת מ3 שלבים:

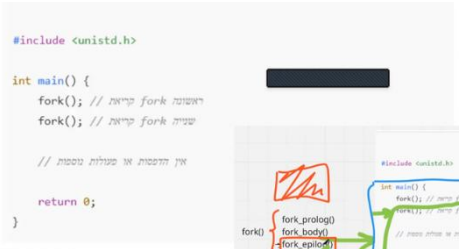
1. פרולוג
2. באדי (גוף)
3. אפילוג

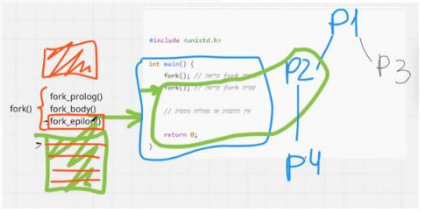
גם השלבים בתוך fork הם כאלה:
Fork_prolog()
Fork_body()
Fork_epilog() – זהו החלק בקוד שנמצא באב, וכשנוצר תהליך בן הוא יירש גם כן את החלק הזה.

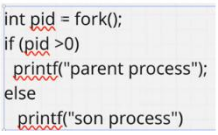
ערכי החזרה של fork()
- בתהליך אב ← יוחזר PID של תהליך הבן .(>0)
- בתהליך הבן ← יוחזר 0
- במקרה של כישלון ← יוחזר -1

באיזה שלב נולד תהליך הבן ב-fork?
בפרולוג יש הכנה, בבאדי יש יצירה של הבן. אז הבן יורש ומבצע את <u>אפילוג</u> ואת <u>הקוד שאחריו</u>.
לכל תהליך יש מזהה משלו, והם ניתנים בצורה סדרתית. נוצרים תמיד המון מזהים, אך כמותם היא סופית ויש לתחזק אותם. ההעדפה היא לתחזק את המספרים (של המזהים) האלו בצורה רציפה. עבור התהליך הראשון שנולד PID=1.

← **שאלה**







יש תוכנית שקוראת פעמיים לfork .
כמה תהליכים ירוצו אחרי ביצוע קוד זה ?
FORK לא מקבל קוד כאן, אז למה נוצר תהליך אם לא סופק קוד?
בהתחלה, יש תהליך שמבצע את הקוד הזה. נקרא לו p1.
בשורה הFORK הראשונה נוצר תהליך בן שנקרא p2.
p2 מקבל לעשות גם כן את המשך הקוד, אז גם p1 וגם p2 יבצעו את המשך הקוד.
מי ירוץ קודם? המתזמן יחליט, לכן במבחן אנחנו נבחר במה שהגיוני יותר. שניהם הגיונית יכולים לרוץ באותה מידה, זה לא משנה במי נבחר. אז נבחר בp1 שימשיך לבצע קודם את שורת הfork השנייה ונוצר תהליך נוסף p3. p3 רוכש את המשך הקוד שהוא ביצוע הreturn 0 .
אז p1 יבצע return 0 ואז p3 יבצע return 0 .
נחזור לp2 שיבצע את שורה ה fork השנייה ונוצר תהליך נוסף p4 . שניהם יבצעו את שורת הreturn 0 .
סה"כ נוצרו 4 תהליכי בן.
אם יהיו 3 שורות fork ייווצרו 8 תהליכי בן.
מה הנוסחה?
בשיש לנו n פעולות fork אז ירוצו סך הכל n^2 תהליכים.
בנוסף , fork מחזיר את מספר המזהה של התהליך שהוא יצר. מספר שלם int. זה מתרחש בחלק האחרון של האפילוג. אז גם האב וגם הבן יבצעו את החלק הסופי של הfork, כלומר **גם הבן אמור להחזיר ערך, אבל הערך שהוא יחזיר זה ברירת מחדל 0** כי הבן לא באמת יצר תהליך , אלא רק ביצע את סוף הFORK שירש מהאב.
Int pid = fork();
איך נבדוק האם תהליך הוא תהליך אב או בן ?
טכניקה מקובלת לזיהוי בצד שמאל של השקופית:
אם הערך המוחזר גדול מ0 זה אומר שזה תהליך אב (כי נוצר תהליך בן שהוחזר הPID שלו)
ואחרת זה אומר שזה תהליך בן (כי לא נוצר שום תהליך חדש וערך ברירת מחדל של int זה 0 וPID מתחיל מהמספר 1)

שיעור 2 – תהליכים ותהליכונים

← **מודל התהליך(Process Model)**

- **Multiprogramming**:ריבוי תהליכים במקביל, כל אחד פועל באופן עצמאי ורציף.
- בכל רגע רק תהליך אחד פעיל בפועל על המעבד.
- **הקשר :(Context Switch)** כל מעבר בין תהליכים מחייב שמירה/שחזור של מצב הרגיסטרים, מחסנית ועוד – פעולה כבדה ("תקורה").
- **שיפור ניצול מעבד :**אם תהליך ממתין ל I/O-ניתן להריץ תהליך אחר בינתיים.

← **יצירה וסיום של תהליכים**

יצירת תהליך:

1. אתחול מערכת(system boot)
2. קריאה לפונקציית מערכת ליצירת תהליך
3. בקשת משתמש
4. הפעלת תהליך באצ(batch) '

סיום תהליך:

1. יציאה רגילה(voluntary)
2. שגיאה בתהליך(voluntary)
3. קריסת תהליך(involuntary)
4. הריגה ע"י תהליך אחר(involuntary)

← היררכיית תהליכים

- תהליך הורה יכול ליצור תהליכי צאצא.
- UNIX : קבוצת תהליכים - ניתן לשלוח אות לכל הקבוצה.
- Windows : אין היררכיה – כל תהליך עצמאי.

← מצבי תהליך(Process States)

- מצב רץ (Running)
- מוכן לריצה (Ready)
- ממתין ל-I/O (Blocked)

מבנה מערכת ההפעלה מבוסס שכבות – השכבה התחתונה מטפלת בפסיקות, תזמון וכו.'

← יישום תהליכים

- **Process Table Entry** טבלה המכילה מידע על כל תהליך (מצב PID , זיכרון, קבצים פתוחים וכו')
- בעת פסיקה OS (Interrupt) שומר מצב תהליך ומבצע פעולות תזמון.

← תהליכונים(Threads)

- תהליכון (Thread) = תהליך קל משקל
- תהליך כולל לפחות תהליכון אחד ראשי (main thread)
- תהליכונים משתפים את אותו מרחב זיכרון וקבצים של התהליך.
- יתרון משמעותי : ההחלפה (context switch) בין תהליכונים קלה וזולה יותר מאשר בין תהליכים
- מבנה תהליכון:
  - רגיסטרים
  - program counter
  - stack
  - אין להם מידע על הקבצים או על כל הזיכרון של התהליך – רק מידע שדרוש להפעלה עצמאית.

← סוגי תהליכונים:

א. תהליכוני משתמש(User-Level Threads)

- מנוהלים במרחב המשתמש – ללא מעורבות של מערכת ההפעלה.
- **יתרונות :**מהירים, חסכוניים במשאבים.
- **חסרונות:**
  - חסימה של תהליכון אחד = חסימת כל התהליך.
  - לא מנצלים ליבות מרובות – ריצה בליבה אחת בלבד.

ב. תהליכוני ליבה(Kernel-Level Threads)

- מנוהלים ע"י ה.Kernel-
- **יתרונות :**ניתן להריץ תהליכונים במקביל על מספר ליבות.
- **חסרונות :**החלפה בין תהליכונים יקרה יותר (מעורבת קריאות מערכת).

← מודלי מיפוי בין תהליכונים

חיסרון	יתרון	תיאור	מודל
חסימת תהליכון = חסימת כל התהליך	מעבר מהיר, פשוט לניהול	כל תהליכוני המשתמש ממופים לתהליכון ליבה אחד	**Many-to-One (M:1)**
תקורה גבוהה יותר(Overhead)	מקביליות מלאה, ניצול ליבות	כל תהליכון משתמש ממופה לתהליכון ליבה	**One-to-One (1:1)**
פחות נפוץ, מורכב לניהול	גמישות, ניצול יעיל	כמה תהליכוני משתמש ממופים לכמה תהליכוני ליבה	**Many-to-Many (M:N)**

← למה צריך גם User Mode וגם Kernel Mode ?

1. **בלי Kernel Mode** לא ניתן לנהל משאבים, תזמון, זיכרון, קריאות מערכת.
2. **בלי User Mode** כל הקוד ירוץ בהרשאות גבוהות – סכנה לאבטחה ויציבות.
3. מודל 1:1 מאפשר הרצה בטוחה של קוד משתמש תוך שמירה על יכולות מערכת ההפעלה.

← שימושים בתהליכונים

- עיבוד מקבילי ביישומים –למשל: מעבד תמלילים עם תהליכונים נפרדים להצגה, שמירה, בדיקת שגיאות.
- Web server - תהליכון נפרד לכל לקוח.
- יתרונות :יצירה/סיום/החלפה מהירים, גישה לזיכרון משותף.
- חסרונות :סינכרון, בעיות תחרות על זיכרון משותף.(race conditions)

← מודל התהליכונים(Thread Model)

1. תהליך עם תהליכון אחד לעומת תהליך עם מספר תהליכונים.
2. תהליכונים משתפים : זיכרון, קבצים, אזור נתונים.
3. לכל תהליכון יש stack ו- registry משלו.
4. POSIX Threads ספריית סטנדרט ליצירת תהליכונים.(pthreads)

← מימוש תהליכונים

- **במרחב המשתמש :**ניהול עצמאי ע"י התוכנה.
- **בקרנל :**ניהול מלא של OS
- **היברידי :**שילוב בין השניים. מאפשר ביצועים טובים + שליטה.
- **Scheduler Activations** מנגנון להקצאת מעבדים לתהליכים – שיפור ביצועים תוך שמירה על שליטה.

← Pop-up Threads

- תהליכון שנוצר אוטומטית כאשר מתקבלת הודעה – למשל בקבלת בקשות בשרת.
- מפחית את זמן ההמתנה לעיבוד.

← תקשורת בין תהליכים (IPC) ובעיות Race Conditions

- **בעיה קלאסית :**שני תהליכים ניגשים בו-זמנית לזיכרון משותף וגורמים לתוצאה לא צפויה.
- **דוגמה :**שני אנשים מושכים כסף מאותו חשבון בו זמנית – תוצאה עלולה להיות שגויה. קריטי מי מבניהם יסיים לבצע את המשיכה ראשון.

← קטע קריטי

קטע קריטי = חלק בקוד בו מתבצע שימוש במשאב משותף. גישה לא מבוקרת אליו עלולה לגרום לבעיות כמו:

- תנאי מרוץ (race conditions) = תוצאות לא צפויות כאשר 2 תהליכים משנים משאב משותף בו זמנית. (פתרון: הקצאת משתנים "גלובליים פרטיים" לכל תהליכון.
- שחיתות נתונים (Data corruption) = ערכים שגויים במשתנים משותפים.

הרעבה = מצב שבו תהליך או תהליכון לא מקבל לעולם את המשאב שהוא צריך כדי להמשיך לעבוד, למרות שהמערכת ממשיכה לפעול ולחלק משאבים לאחרים. כלומר –הוא לא תקוע בגלל תקלה, אלא פשוט כל פעם המשאב הולך לאחרים לפני שהוא מספיק לקבל אותו.

עקרונות המניעה ההדדית (חשוווב למבחן!)

1. גישה בלעדית (nutual exlusion) – רק תהליך אחד יכול להיות בקטע הקריטי בכל רגע נתון.
2. הוגנות (fairness) – כל תהליך שמבקש גישה לקטע הקריטי צריך לקבל אותה בסופו של דבר (אין הרעבה starvation)
3. מניעת קיפאון (deadlock) – יש להבטיח שהתהליכים לא יכנסו למצב של נעילה הדדית.
4. שחרור משאב – התהליך חייב לשחרר את המשאב בסיום השימוש בו (unlock)

Busy waiting = מצב שבו תהליך יבצע **המתנה פעילה** ע"י בדיקה חוזרת ונשנית של תנאי סיום. במקום להיכנס למצב שינה ולהמתין לאירוע שיתעורר ע"י מערכת ההפעלה או תהליך אחר. (לדוג' – תהליך בודק שוב ושוב אם משאב זמין, במקום להמתין לשחרור בצורה יעילה. מה היא כוללת?

- בדיקה חוזרת – התהליך משתמש בלולאה אינסופית לבדיקת מצב.
- (down) Wait – עבור סמפור 1 , ישתנה משתנהל-0 והתהליך ממשיך.
  עבור סמפור 0 , התהליך נכנס למצב המתנה עד שהמשאב ישתחרר.
- (up) Signal – משתנה את ערך הסמפור ל-1 מה שמאפשר לתהליכים ממתינים להמשיך לפעול.

סמפור (Semaphore) = מבנה נתונים המשמש בסנכרון במערכות הפעלה ובתכנות מקבילי, כדי לאפשר שליטה בגישה למשאבים משותפים (כמו זיכרון, קבצים, מדפסות או כל משאב אחר שמספר תהליכים/תהליכונים רוצים להשתמש בו בו־זמנית).
אפשר לחשוב על סמפור כמו **מנעול עם מונה**:**(counter)**

- המונה מייצג כמה "יחידות" של המשאב זמינות.
- כל תהליך שרוצה להשתמש במשאב מבצע פעולה **שמקטינה** את המונה.(**wait**)
- כשסיים, הוא מחזיר את המשאב **ומגדיל** את המונה.(**signal**)
- אם המונה מגיע ל־0, התהליך צריך להמתין עד שמישהו אחר ישחרר משאב.

<u>סוגי סמפורים:</u>

1. סמפור בינארי = ניהול גישה למשאב יחיד במערכת מרובת תהליכים. מוגבל ל-2 ערכים:
   - 1 המשאב פנוי
   - 0 המשאב תפוס

   <u>שלבי הסמפור:</u>
   - **אתחול** – הסמפור יאותחל ל-1 אם המשאב פנוי, או ל-0 אם המשאב תפוס.
   - **Wait** – כאשר התהליך מבקש גישה למשאב:
     - אם ערך הסמפור הוא 1 ← המשאב פנוי, הסמפור משתנה ל-0 (תופס את המשאב), והתהליך ממשיך לפעול.
     - אם ערך הסמפור והא 0 ← המשאב תפוס, התהליך נכנס להמתנה עד שהמשאב ישוחרר.
   - **Signal** – כאשר התהליך מבקש לשחרר משאב:
     - הסמפור משתנה ל-1 (המשאב משוחרר)
     - אם יש תהליכים שממתינים למשאב, אחד מהם יתעורר וימשיך לפעול.

2. סמפור מונה = ניהול גישה למספר משאבים סופי במערכת מרובת תהליכים. יקבל ערכים שלמים גדולים מ-1.
   <u>שלבי הסמפור:</u>
   - **מונה משאבים counter** –
     - **C > 0** יש C משאבים פנויים
     - **C = 0** אין משאבים זמינים
     - **C < 0** |C| תהליכים ממתינים לשחרור משאבים.
   - **Wait** – כאשר התהליך מבקש גישה למשאב:
     - מקטינה את C ב-1
     - אם C > 0 ← כל המשאבים תפוסים, ותהליך נכנס להמתנה עד שישוחרר משאב.
     - אחרת – התהליך ימשיך לפעול.
   - **Signal** – כאשר התהליך מבקש לשחרר משאב:
     - מגדילה את C ב-1
     - אם יש תהליכים ממתינים למשאב (C<=0) , אחד מהם יתעורר וימשיך לפעול.

3. Mutex = ניהול גישה <u>בלעדית</u> למשאב במערכת מרובת תהליכים. המטרה היא למנוע מצבים של תנאי מרוץ.
   - רק תהליך אחד יכול להחזיק את ה-mutex
   - תהליך יכול:
     - לנעול Lock
     - לשחרר Unlock

- גישה בלעדית – מבטיחה שאף תהליך לא יכול לגשת למשאב בזמן שהוא נעול
- אם תהליך מנסה לנעול mutex שכבר נעול, הוא נכנס למצב המתנה עד לשחרור המשאב.

מה ההבדל בין סמפור בינארי ו-mutex ? מי משחרר אותם.
אחד התנאים לניהול קטע קריטי הוא שצריך בסוף לשחרר אותו.
בסמפור בינארי, מי שעשה את ההגנה לקטע קריטי לא חייב לשחרר את הקטע הקריטי, כי מישהו אחר יכול לשחרר את הקטע הקריטי.
אין בלעדיות לשחרור. מישהו אחד תפס, אבל מישהו אחר יכול לשחרר אותו.
ב-mutex , השחרור הוא בלעדי. התהליך שתפס את הקטע הקריטי הוא זה שחייב לשחרר אותו.

סיגנלים = מנגנון תקשורת בין תהליכים.
כשתהליך מקבל סיגנל, הוא יכול:
- להתעלם ממנו (למעט סיגנלים מסוימים)
- לטפל בו ע"י פונקציה מותאמת אישית
- לטפל בו התאם לברירת מחדל

שם סיגנל	מספר סיגנל	תיאור	התנהגות ברירת מחדל
`SIGINT`	2	עצירת תהליך (למשל `Ctrl+C`).	סיום התהליך.
`SIGKILL`	9	הורג את התהליך מידית (לא ניתן להתעלם).	סיום התהליך.
`SIGTERM`	15	בקשה לסיים את התהליך בצורה מסודרת.	סיום התהליך.
`SIGSTOP`	19	עוצר את התהליך זמנית (`Ctrl+Z`).	עצירת התהליך.

סיגנל א-סינכרוני = סיגנל שלאחר שנשלח, אין התחייבות למתי יגיע (יכולות להתרחש השעיות), ובשיגיע אין התחייבות של טיפול מידי בו.
נשתמש בו למשל בתקשורת בין תהליכים - אם תהליך נמצא באמצע משהו חשוב הטיפול בסיגנל יכול לחכות.
סיגנל סינכרוני = נשלח מיידית ומטופל מיידית.
נשתמש בו כשיש פסיקות (למשל באלה שיוצרות בעיות בחומרה, כמו חילוק ב-0 וזה דורש טיפול מידי כי המחשב לא יכול להמשיך).

קריטריון	סיגנל סינכרוני	סיגנל אסינכרוני
מתי הסיגנל נשלח	נשלח בתגובה לאירוע שמתרחש במהלך ביצוע הקוד של התהליך.	נשלח לתהליך בכל זמן, ללא תלות במה שהתהליך עושה באותו רגע.
דוגמה לסיגנל	`SIGSEGV` (שגיאה בגישה לזיכרון), `SIGFPE` (שגיאת חישוב).	`SIGINT` (לחיצה על Ctrl+C), `SIGUSR1` (סיגנל מותאם אישית).
סיבת השליחה	נגרם כתוצאה מפעולה ישירה של הקוד, לדוגמה ניסיון גישה לא חוקי לזיכרון.	נגרם על ידי משתמש, תהליך אחר, או מערכת ההפעלה.
תזמון	מתרחש בדיוק בזמן שבו האירוע קורה בתהליך עצמו.	עשוי להתרחש בכל שלב, תוך כדי שהקוד מבצע פעולות אחרות.
טיפול בסיגנל	לעיתים קרובות דורש טיפול מיידי כדי למנוע סיום התהליך.	דורש טיפול על פי צורך, בהתאם לסוג הסיגנל.
שליטה בסיגנל	התהליך אינו שולט אם או מתי הוא יתרחש (נגרם כתוצאה משגיאה בקוד).	התהליך יכול להגדיר ולשלוח סיגנלים בעצמו (לדוגמה, `kill`).
מקור הסיגנל	פעולת התהליך עצמו.	תהליך חיצוני, מערכת ההפעלה, או משתמש.

<u>Signal היא גם קריאת מערכת שמשנה התנהגות של סיגנל</u>. מקבלת לפי הסדר:
ההתנהגות החדשה שנרצה לשנות אליה, שם הסיגנל.
לדוג':

```c
void handle_signal(int sig) {
    printf("Received signal %d\n", sig);
}

int main() {
    signal(SIGINT, handle_signal);  // הגדרת SIGINT למטפל מותאם אישית
```

<u>איך הורגים סיגנל?</u>

1. מה- command line

```
kill -SIGTERM <PID>  # שולח SIGTERM מסוים לתהליך
kill -9 <PID>        # שולח SIGKILL (הורגת את התהליך מידית)
kill -SIGSTOP <PID>  # עוצר את התהליך
kill -SIGCONT <PID>  # ממשיך את התהליך
```

2. מה-EPI

```c
#include <signal.h>
#include <unistd.h>

int main() {
    pid_t pid = 12345;  // PID של התהליך
    kill(pid, SIGTERM); // שולח SIGTERM
    return 0;
}
```

פעולות אטומיות = פעולה שמתחילה ואי אפשר לעשות במהלכה את ההחלפת הקשר. רק כאשר היא תסתיים ניתן לעשות הקשר.
פעולות לא אטומיות = פעולה שמתחילה ואפשר לעשות במהלכה החלפת הקשר. לדוג' חישוב מתמטי, חישוב לוגי, השמה.

1m left to save your board. Sign u

```
1   int flag[2] = {0};
2   int label[2] = {0};

3   void lock(int i) {
4       int j = 1-i;

5       flag[i] = 1;
6       label[i] = 1 + label[j];

7       while ((flag[j]) && (label[i] > label[j]));
8   }

9   void unlock(int i) {
10      flag[i] = 0;
11  }
```

← **שאלה**

נתון קוד שיש בו קטע קריטי, שיסומן ב-CS.

קיימים 2 תהליכונים שרצים. תהליכון 0 ותהליכון 1.

לפני הקוד צריך:



- לזהות שזה CS ויש שם משאב משותף
- לפניו לקרוא ל-()lock
- בסופו לקרוא ל-()unlock

וזאת על מנת ליצור סנכרון והגנה על פי אחת הגישות שלמדנו.

נתון בקוד מימוש של שתי הפונ' הללו, וצריך לבדוק אם הוא טוב או לא – כלומר האם הדרישה לאמצעי סנכרון והגנה על קטע קריטי מתקיימת. אנו חייבים שתנאים 1 ו-3 יתקיימו – גישה בלעדית ומניעת קיפאון.

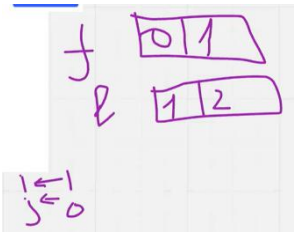
אם הם טובים יש להוכיח, אם הם לא טובים יש להביא דוגמא נגדית.

i הוא מזהה התהליכון שהפעיל את lock.

j הוא מזהה התהליכון האחר.

Flag הוא מערך שמודיע שתהליכון נמצא בקטע קריטי. אם תהליכון 1 פועל הערך בתא 1 הופך ל1. אם תהליכון 0 פועל הערך בתא 0 הופך ל-0.

תהליך בהמתנה יכנס ל"לופ" בלולאה ה-while. לדוג' במצב בו תהליך 1 עובד, תהליך 0 יכנס ללופ:



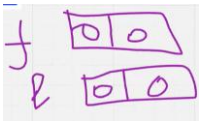
בעצם ברגע שהתהליך השני יסיים התהליך הזה יוכל לצאת מהלופ של הלולאה ולהיכנס ל-CS.

תהליך שסיים את lock מגיע ל-CS . כשהוא מסיים אותו, הוא מגיע ל-unlock .

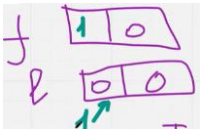
שם, הוא משנה בתא שלו ב-flag את הערך ל-0 .

הקוד לא טוב מכיוון שהוא לא אטומי. יכול להגיע המתזמן ולהחליף במהלך פעולת lock בין שני התהליכים ואז ייווצר בלאגן. לדוג':

שום תהליך לא עובד. כל התאים 0.

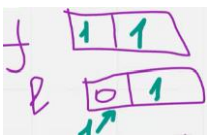


תהליך 0 רוצה לעבוד והתא שלו משתנה ל-1 במערך flag.

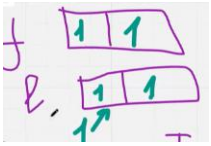


גם במערך label הוא צריך להפוך ל-1 , אבל לפני ביצוע שורה 6 מתבצעת החלפת הקשר לתהליך 1 ע"י המתזמן אז הערך 1 עוד לא הוכנס ונמצא במצב המתנה.

תהליך 1 מתחיל לעבוד עד שהמתזמן עובד לפני ביצוע שורה 7.



חוזרים לתהליך 0 שממשיך את שורה 6 , והערך 1 נכנס לתא.



כשתהליך 0 מגיע לשורה 7 הוא בודק האם הוא חסום או לא – אבל מספרי התור שלהם זהים, התנאי לא מתקיים והוא נכנס ל-CS.
מגיע תור של תהליך 1, וגם הוא בשורה 7 בודק את התור שלו , התנאי בלולאה לא מתקיים והוא נכנס גם כן ל-CS . הגענו למצב ששני התהליכים עובדים ב-CS **וקורה מצב של race condition שיכול לגרום לתקלות**.

```
1   int flag[2] = {0};
2   int label[2] = {0};

3   void lock(int i) {
4       int j = 1-i;

5       flag[i] = 1;
6       label[i] = 1 + label[j];

7       while ((flag[j]) && (label[i] >=label[j]));
8   }

9   void unlock(int i) {
10      flag[i] = 0;
11  }
```

עבור קוד זהה בו בשורה 7 יש סימן = ,
נכנסים למצב של deadlock , שני התהליכים "טוחנים" את הלולאה באותו תרחיש שנתנו בדיוק כי אצל שניהם התנאי בלולאה מתקיים כל הזמן, ונכנסים למצב המתנה. אף אחד אף פעם לא יכנס ל-CS.

```
int flag[2] = {0
int label[2] = {

void lock(int i) {
    int j = 1 - i;
    flag[i] = 1;
    label[i] = label[j] + 1;

    while (flag[j] && (
          label[i] > label[j] ||
         (label[i] == label[j] && i > j)));
}
```

איך נתקן את הקוד כך שיהיה טוב?
נשנה את לולאת ה-while , כך שאם המספרים ב-label שווים ניצור מצב של חוסר הוגנות בj>i , כלומר שנתעדף את התהליכון 0 על תהליכון 1. כלומר שמצב ששניהם במצב 1 , נתעדף פעולה של תהליכון 0 ותהליכון 1 יישאר בהמתנה.

← **שאלה**
נתונים 2 סיגנלים מסוג Custom Signal (שהמתכנת מגדיר ומשתמש בעצמו).
הכי "כיף" להשתמש בהם, כי אם משנים סיגנל מבונה במערכת יכול להיות שנפגע במשהו, אך כאן זהו סיגנל שאנחנו יצרנו מ-0 אז לא קיים חשש בזה.
שמותיהם SIGUSR2 , SIGUSR1

צריך לנסות לממש סמפור על סמך סיגנלים בלבד - כלומר ליצור את האתחול, פונ' wait ופונ' signal . בנוסף, אסור להשתמש באמצעי סנכרון אחרים, חוץ ממצבים שצריך ליצור הגנה למשתמש.
איזה מהמימושים שלפנינו טוב ?

```
//Implement 1
volatile sig_atomic_t semaphore;  // 1 = free, 0 = taken
sigset_t signal_set;

// Initialize semaphore (1 = open, 0 = closed)
int init_sem(int initial_value) {
  semaphore = initial_value;

  // Set up signal set and block SIGUSR1 globally
  sigemptyset(&signal_set);
  sigaddset(&signal_set, SIGUSR1);
  pthread_sigmask(SIG_BLOCK, &signal_set, NULL);

  return 0;
}

// Wait (down) operation — acquire semaphore
void down_sem() {
  int sig;
  while (1) {
    if (semaphore == 1) {
      semaphore = 0;  // acquire semaphore
      return;
    }
    // Wait for signal indicating semaphore release
    sigwait(&signal_set, &sig);
    // After signal, recheck semaphore
  }
}

// Signal (up) operation — release semaphore
void up_sem() {
  semaphore = 1;  // release semaphore

  // Send signal to wake one waiting thread
  kill(getpid(), SIGUSR1);
}
```

המימוש לא טוב.
Volatile – מגדירים כמה תהליכים יכולים לגשת לסמפור בו זמנית.
Sig_atomic_t – האינטג'ר שהכתיבה שלו והקריאה ממנו היא אטומית.
Sigset_t – מבנה נתונים שיכול להכיל בתוכו כמה סיגנלים.
1 פנוי
0 תפוס
בחלק של wait , יכול להיווצר מצב של race condition אם מתרחשת החלפת הקשר. לכן יש צורך בשיפור בתוך if של wait – צריך ליצור החלפה אטומית ולא מופרדת.
פתרונות אפשריים:

1. **נוכל להגדיר mutex ולקבל אטומיות באמצעותו**. לפני בדיקת הסמפור "לתפוס" את ה-mutex וכך לדאוג לבלעדיות של הבדיקה והשמת ערך של הסמפור.

```
// Wait (down): always wait for signal first
void down_sem() {
  int sig;

  while (1) {
    sigwait(&signal_set, &sig);  // block until signal
    // After wakeup, check if semaphore is free
    if (semaphore == 1) {
      semaphore = 0;  // acquire
      return;
    }
    // Else loop and wait again
  }
}
```

2. **פונ' CAS או Test-and-Set**. זו פונ' של מערכת ההפעלה שהיא עצמה אטומית, שבודקת ערך ואם הוא 1 היא שמה במקומו 0, ואחרת היא חוזרת מיידית.

```
// Semaphore state: 1 = free, 0 = taken
volatile int semaphore = 1;

// CAS-based down (wait)
void sem_down() {
  while (1) {
    if (__sync_bool_compare_and_swap(&semaphore, 1, 0)) {
      return;  // acquired
    }
    // Busy wait (spin) — optional: sleep or yield to avoid CPU burn
    sched_yield();
  }
}

// CAS-based up (signal)
void sem_up() {
  __sync_lock_release(&semaphore);  // sets semaphore = 1
}
```

שיעור 3 – תרגול שיעורים 1+2

← **שאלה** – 2022 מועד א שאלה 1

שאלה 1 (22 נקודות)

שני הסעיפים מתייחסים לבעיה הבאה.

במערכת ישנם שלושה תהליכונים (THREADS) A, B ו-C. יש לכתוב אלגוריתם (בפסאודוקוד או דמוי C) המממש קטע קריטי בעל התכונות הבאות:

אם A - נמצא בקטע הקריטי, אזי הוא נמצא בו לבדו.

תהליכונים B ו-C יכולים להימצא בקטע הקריטי בו זמנית. בפרט, אם אחד מהם נמצא בקטע הקריטיאזי השני יכול להיכנס לקטע הקריטי גם כן, אפילו אם A כבר ממתין להיכנס לקטע קריטי.

על האלגוריתם לקיים את כל הדרישות של פתרון בעיית קטע קריטי, חוץ מהוגנות(FAIRNESS).

(13 נק') א. ממשו את הפרוצדורות:

A-enter ,A-exit, B-enter , B-exit, C-enter ו C-exit

המממשות את קטעי הכניסה והיציאה של התהליכונים לקטע קריטי בצורה היעילה ביותר.

בסעיף זה מותר להשתמש אך ורק במשתנים הבאים:

1. סמפורים S1 ו-S2 המאותחלים שניהם לערך 1.
2. משתנה C שמאותחל לערך 0.

אין להשתמש בהמתנה פעילה(busy waiting). כלומר, תהליכונים יכולים להמתין רק ע"י שינה על סמפור.

אין חובה להשתמש בכל המשתנים.

משתנה c – רמז למשתנה count

טריק מחשבתי – נסתכל על הבעיה מזווית כזו, ש-A תהליכון אחד, ו-B ו-C נסתכל עליהם כתהליכון אחד.

איך נגרום ל-B ו-C להיות ישות משותפת בסמפורים?

צריך לחשוב על דרך שאם B או C נכנסים הם נועלים את הכניסה ל-A, ובשהאחר נכנס הוא אמור לדעת שהוא נכנס שני ולא לבצע נעילה ל-A כי היא כבר בוצעה, אלא רק להצטרף לחבר שלו. לשם כך יש לנו סמפור נוסף ואת משתנה C.

אתחול:

sem_t s1 = 1;

sem_t s2 = 1;

int c = 0;

C שווה 0 כי אף תהליכון לא נמצא בתוך הקטע הקריטי בהתחלה.

סמפורים שווים 1 כי שניהם פנויים.

פונקציות:

נעזר במשתנה c על מנת לדעת כמה מתהליכונים B ו-C נמצאים בקטע הקריטי (ערכים 0,1 או 2).

S1 סמפור בין B ל-C , S2 סמפור בין A ל-BC.

נזכור להתייחס למצבים שהמתזמן מחליף בין התהליכונים באמצע !!

כאשר B ו-C נכנסים, הם במצב wait על סמפור 1. מי מהם שהוא הנציג הראשון נכנס ללולאה וחוסם את סמפור 2 , עבור A.

```
Void enter_critical_section_BC()
{
   sem_wait(&s1);        // acquire mutex to update counter
   if (c == 0)
   {        // first update
      sem_wait(&s2);
   }
   c++;                  // increment the count of readers (B/C)
   sem_signal(&s1);      // release mutex
}
```

האחרון מבין BC שיעזוב יוכל להיכנס ללולאה ולשחרר את סמפור 2 עבור A.

```
Void leave_critical_section_BC()
{
   sem_wait(&s1);        // acquire mutex to update counter
   c--;
   if (c == 0)
   {        // last update
      sem_signal(&s2);
   }
   sem_signal(&s1);        // release mutex
}

void enter_critical_section_A()
{
   sem_wait(&s2);        // wait until no B/C thread is inside
}

void leave_critical_section_A()
{
   sem_signal(&s2);        // release the critical section for others
}
```

← **שאלה** – 2025 מועד א שאלה 7

שאלה 7 (25 נקודות)

א. (10 נק')

חלק ממעבדים (לדוגמא ARM) תומכים בשתי פקודות מכונה אטומיות. לתיאום בין תהליכונים (THREADS).

פקודת מכונה אחת LL(p) מחזירה את הערך השמור בכתובת p (שקול לאופרטור * על מצביע).

כלומר v=LL(p) שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v.

הפקודה השניה SC תשמור את הערך בכתובת זיכרון רק אם מאז ביצוע פקודת LL על אותה הכתובת לא היו בה שמירות ע"י תהליכוניים אחרים. במקרה של הצלחה סטטוס סיום הפקודה שווה ל 1 ובמקרה של כישלון ל 0. ההוראה $r = SC(v, p)$ מנסה לכתוב את הערך v לכתובת p בזיכרון.

אם הפעולה האחרונה של התהליכון הנוכחי על הכתובת p הייתה LL ,ואף תהליכון אחר לא כתב ערך לכתובת p מאז שהתהליכון הנוכחי ביצע LL(p) , הכתיבה מצליחה והפקודה מחזירה 1.

בכל מקרה אחר, כלומר אם תהליכון לא ביצע LL , או אם תהליכון אחר כתב לכתובת p מאז שהוא ביצע (בין אם בעזרת LL ובין אם בעזרת השמה בכל דרך), הכתיבה נכשלת (הערך v לא נכתב לכתובת p) והערך המוחזר הוא 0 . במילים אחרות, הכתיבה מצליחה רק אם הערך שכתוב בזיכרון הוא הערך שהתהליכון בעצמו קרא בעזרת ה LL ואף תהליכון אחר לא כתב לתוכו מאז.

ממשו שגרות lock(m) ו unlock(m) למניעה הדדית, כאשר m הוא מצביע למספר שלם שמייצג את המנעול MUTEX.

המימוש צריך להתבסס על 2 מקרואים(MACROS) התואמים את הפקודות בשאלה:

- v=LL(p) (שקול לטעינת הערך ששמור בכתובת p לתוך המשתנה v)
- $r = SC(v, p)$ (המנסה לכתוב את הערך v לכתובת p בזיכרון ומחזיר את סטטוס סיום)

המימוש חייב להבטיח את **מניעה הדדית והתקדמות**, אך **אינו צריך להבטיח המתנה מוגבלת** (העדר הרעבה).

ב. (15 נק')

על בסיס שימוש בפונקציות נעילה ושחרור מסעיף א', כתבו פסוודו-קוד המדמה פתרון לבעיה הבאה:

קיים קטע כביש צר חד נתיבי שהתנועה בו מותרת בכל רגע נתון רק בכיוון אחד. כל עוד שיש תנועה באותו הכיוון, מכוניות נוספות יכולות לנסוע באותו הכיוון. אבל מכוניות שרוצות לנסוע מהצד השני של הקטע הצר בכיוון הפוך, צריכות להמתין עד שהתנועה הקיימת תיפסק. כל מכונית ממומשת כ תהליכון THREAD . מותר להשתמש אך ורק במשתנים הבאים: 3 מנעולים m1, m2, m3 ושני מונים c1, c2 .

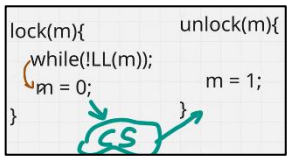
צריך למשש רק את פרוטוקול הסנכרון ואין צורך למשש נסיעה עצמה, מספיק לכתוב את זה בהערה.

א. פקודה LL(p) שקולה ל: v=*p .

נרצה שאם m=1 ה-lock פנוי, ואם m=0 ה- lock תפוס.

Lock נמצא לפני קטע קריטי CS , unlock נמצא אחריו.

פתרון לא טוב:



אין בקרה בין התהליכונים!

לדוג' 1T נכנס לקטע הקריטי, m=0, ושני תהליכונים 2T 3T נכנסו אחריו ונמצאים בהמתנה בלולאה. כש-1T ישחרר , m=1 . במצב בו המתזמן יתזמן את 2T , שיספיק רק לצאת מהלולאה ואז התיזמון יועבר ל3T , מבחינתו עדיין m=1 כי 2T לא הספיק לשנות את הערך של m ל-0 כלומר לתפוס, ואז גם T3 יצא מהלולאה וייווצר מצב ששני תהליכונים שוחררו ונכנסו לCS וזה לא טוב.

פתרון טוב:

Volatile אומר שכמה תהליכונים יכולים לגשת במקביל למשתנה.

success מראה על הצלחה לכניסה לסמפור. 0 כשלון שקר , 1 הצלחה אמת. מאותחל ל-0.

Value או m - האם סמפור פנוי. 1 פנוי, 0 תפוס. מאותחל ל-1

- אם המתזמן לא קטע את פעולת התהליכון בין ההשמה ל-value לבין ההשמה ל-success , הערך של success ישתנה ל-1 כלומר הצלחה בכניסה לסמפור, הסמפור ייתפס m=0 , והתהליכון יכנס ל-CS.

- אם המתזמן קטע את פעולת התהליכון בין ההשמה ל-value לבין ההשמה ל-success , ותהליכון אחר הספיק לבצע השמה ל-value, ברגע שהמתזמן יחזור לתהליכון כשהוא יגיע ל- success הערך יהיה 1 כלומר כשלון בכניסה לסמפור כי תהליכון אחר כתב ערך ל-m מאז שהתהליכון הנוכחי ביצע LL ולכן לא תהיה כניסה ל-CS ונשאר בלופ על הלולאה עד שהוא יתפנה.

```
volatile int m = 1; // 1 = Unlocked, 0 = Locked

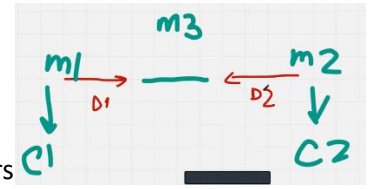
// Lock function using LL/SC with while loop
void lock(volatile int *m) {
   int success = 0;
   int value;

   while (!success)
   {
      value = LL(m);
      if (value == 1)
      {  // Check if unlocked
           success = SC(0, m);  // Try to set it to 0 (locked)
      }
   }
}

// Unlock function using LL/SC
void unlock(volatile int* m) {
   int success = 0;
   int value;

   while (!success)
   {
      value = LL(m);
      if (value == 0)
      {  // Check if locked
         success = SC(1, m);  // Try to set it to 1 (unlocked)
      }
   }
}
```

ב. נאחד את כל המכוניות שנכנסות מצד שמאל, ואת כל המכוניות שנכנסות מימין. בדומה לשאלה הקודמת.
המכונית הראשונה שנכנסת תנעל את המנעול לקבוצה השנייה, ולהיפך.




```
// Shared variables
int m1 = 1;   // protect C1
int m2 = 1;  // protect C2
int m3 = 1;  // protect CS
int c1 = 0;  // Counter for cars
int c2 = 0;  // Counter for cars moving in direction 2

// Function for cars moving in direction 1
void* Car_Direction_1()
{
   lock(&m1);
```

```
    if (c1 == 0)  { //
        lock(&m3);  // First car locks the road
    }
    c1++;
    unlock(&m1);
    // Critical section - move car1
    lock(&m1);
    c1--;
    if (c1 == 0)   {
        unlock(&m3);  // Last car releases the road
    }
    unlock(&m1);
}

// Function for cars moving in direction 2
void* Car_Direction_2()
{
    lock(&m2);
    if (c2 == 0)
    {
        lock(&m3);  // First car locks the road
    }
    c2++;
    unlock(&m2);

    // Critical section - move car2

    lock(&m2);
    c2--;
    if (c2 == 0){
        unlock(&m3);  // Last car releases the road
    }
    unlock(&m2);
}
```

← **שאלה** – בעיות יצרן-צרכן , 2024א מועד 68 שאלה 7

נתון מפעל ממתקים (הממתק הוא ה-item), שיש בו מכונות שמייצרות אותם ושמות בקופסאות. ניתן לייצר עד N ממתקים. ילדים מגיעים למפעל ולוקחים ממתקים ישירות מהמכונות.

P מייצג את המכונות (P מלשון יצרן producer)

C מייצג את הילדים (C מלשון צרכן customer)

התנגשויות שיכולות להיווצר ויש לנהל אותן:

1. מספר מכונות ישימו את הממתקים שלהן באותה קופסא (התנגשות בין יצרן ליצרן)
2. מספר ילדים ניגשו לאותה הקופסא (התנגשות בין צרכן לצרכן)
3. מכונה לא סיימה לשים את כל הממתקים שלה בקופסא, וילד ניגש והתחיל לקחת ממתקים ממנה (התנגשות בין צרכן ליצרן)

איך ננהל זאת? (כאשר הסוכריות במערך\רשימה מקושרת)

נסתכל על שאלה דומה ממבחן:

א. היצרן – אמור להיות חסום כאשר אין מקומות פנויים להוסיף עוד ממתקים, כלומר N.

הצרכן – אמור להיות חסום כאשר אין ממתקים, כלומר 0.

אנא סיימו את התוכנית להלן עבור בעיית יצרן-צרכנים(יכולים להיות כמה צרכנים, אבל רק יצרן אחד). מלאו בכל שורה ריקה פעולה(פקודה רגילה אחת או פעולה אחת על אמצעי סנכרון) **אחת בלבד**. הניחו:

- הגבול העליון של המאגר הוא N והמאגר ריק בתחילה.
- המאגר מנוהל כרשימה מקושרת כאשר פונקציה Enqueue(item) מכניסה לסוף הרשימה ו Dequeue() מוציאה מראש הרשימה. שתי הפונקציות מעדכנות את המצביעים לראש וסוף הרשימה בהתאם.

אין להשתמש באמצעי הסנכרון ללא צורך ואין להוסיף אמצעי סנכרון נוספים.

ג. (7 נק')

בהמשך לסעיף ב', הניחו שפונקציה ( )Dequeue תמיד דואגת להשאיר לפחות פריט אחד במאגר. לא צריך להתייחס לשאלה מה הסיבה לזה. אם יש רק פריט אחרון במאגר, היא מחזירה NULL וזה מטופל בקוד של צרכן עצמו(שקורא ל ()ConsumerFunction) .

הציעו שכלול לפתרון שיאפשר מקביליות גדולה יותר בין יצרנים לצרכנים. לא צריך להתייחס לשינוי רמת מקביליות בתוך יצרנים ובתוך צרכנים.

```
Semaphore BufferState1= N ; //Counting semaphore
Semaphore BufferState2 = 0 ; //Counting semaphore
Semaphore mutex1 = 1; // Binary semaphore
Semaphore mutex2 = 1; // Binary semaphore


ProducerFunction(item) {
wait(BufferState1) ;
wait(mutex1); ;

Enqueue(item); //Non-atomic function
signal(mutex1); ;

signal(BufferState2) ;

}
ConsumerFunction() {

wait(BufferState2) ;

wait(mutex1) ;
item = Dequeue(); //Non-atomic function
signal(m1) ;

signal(BufferState1) ;

return item;
}
```

לכן סמפור אחד יאותחל למספר המקומות הפנויים שיש להכניס מוצרים חדשים N, והסמפור השני יאותחל ל-0.

ב. האם במידה ויש מספר יצרנים נצטרך לשנות את הקוד? לא.
שמנו את שורת wait(mutex1), שאומרת שכאשר מגיע יצרן הוא מקבל בלעדיות לגישה למבנה, ועשינו זאת מחשש להתנגשויות. בגלל שבסעיף א יש רק יצרן בודד לא היו התנגשויות יצרן-יצרן, אבל שמנו את השורה הזו מחשש להתנגשויות יצרן-צרכן. אך גם במידה ויהיו מספר יצרנים המיוטקס הזה פותר לנו את זה.

ג. תזכורת - Dequeue שולפת איבר מהתור.
נשתמש כעת ב-Mutex2 שעדיין לא השתמשנו בו, **ובפונקציית הצרכן נשנה את כל המיוטקס1 למיטוקס2. בעצם ניצור מיוטקס1 ליצרנים, ומיוטקס2 לצרכנים.**
מקודם, החשש לתקלה היה כשיש מוצר אחד במאגר, שייצור בעיית צרכן-יצרן: היצרן מייצר את אותו מוצר יחיד, ואז הצרכן רואה שיש מוצר והוא רוצה לקחת אותו. אם קרתה החלפת הקשר בין היצרן לצרכן והיצרן לא סיים את הייצור, הצרכן בעצם ייקח מהמאגר כאשר הוא ריק (המוצר עוד בתהליך ייצור) ותהיה תקלת Data corruption הן ליצרן והן לצרכן.
כעת, על פי הנתונים הצרכן וותרן – וכשיש רק מוצר אחד המאגר הוא לא יגע בו ולכן אין בעיות צרכן-יצרן כעת. מכאן שמה שייתן מקביליות גדולה יותר בין צרכנים ליצרנים הם מיוטקסים **נפרדים** כשיפתרו בעיות צרכן-צרכן ויצרן-יצרן.

שיעור 4 – הפרדת משאבים

בידוד והפרדת משאבים נעשים בדר"כ לצורכי אבטחה, כשלא רוצים ששני תהליכים ישלחו סיגנלים זה לזה, ובעצם נרצה שלא ישפיעו זה על זה.

מאפיין	Containers	Namespaces	VMs
בידוד משאבים	חלקי (על בסיס Namespaces ו-Cgroups).	על פי משאב (PID, רשת, מערכת קבצים).	מלא (כולל Kernel).
מערכת הפעלה	חולקים את Kernel של ה-Host.	חלק ממערכת ההפעלה המארחת.	מערכת הפעלה נפרדת.
ביצועים	קרוב לביצועים מקומיים.	קרוב לביצועים מקומיים.	ביצועים נמוכים יותר עקב אמולציה.
משאבים	קל משקל.	תלוי בתהליך.	כבד, צורך הרבה משאבים.
שימושים	Microservices, DevOps.	תהליכים מבודדים, בדיקות.	בידוד חזק, מערכות הפעלה שונות.
ניהול רשת	Network Namespace.	Network Namespace.	רשתות וירטואליות ב-Hypervisor.
גמישות	מוגבל למערכת ההפעלה המארחת.	מוגבל למערכת ההפעלה המארחת.	ניתן להריץ מערכות שונות.

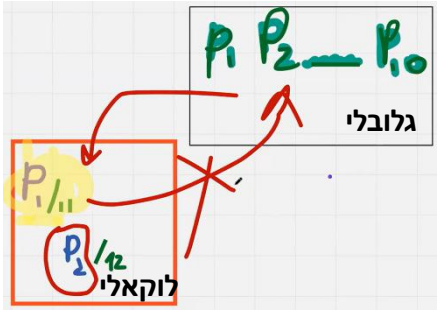
וירטואליזציה (Virtualization) = הרצת מספר סביבות עבודה על **מחשב פיזי אחד VM** תוך כדי הפרדת משאבים בין כולם. זו הפרדה מלאה בין תהליכים, בה כל תהליך עובד בפני עצמו מבלי לדעת על השאר. ווירטואליזציה רואים במכונות וירטואליות.

קונטיינריזציה(Containerization) = הרצת יישומים מבודדים בסביבות קלות משקל שנקראות **קונטיינרים**. ההפרדה מתבצעת במדד אחד או במספר מדדים, וכל שאר הדברים משותפים. כלומר ספקטרום ההפרדה הוא קטן יותר (בניגוד לווירטואליזציה שם ההפרדה היא מלאה).

בווירטואליזציה – כל מכונה וירטואלית (VM) מריצה מערכת הפעלה מלאה משלה + היישום. זה כבד, צורך יותר זיכרון ומעבד.
בקונטיינרים –כולם חולקים את מערכת ההפעלה של ה- Host , אבל כל קונטיינר מבודד ברמה של מערכת קבצים, תהליכים, רשת. זה הרבה יותר קל ומהיר.

Namespace = מרחב שבו מזהים (שמות משתנים, פונקציות, מחלקות וכו') ייחודיים. מנגנון שנועד לבודד שמות או משאבים כדי למנוע התנגשויות וליצור סדר. הוא יכול להיות גלובלי או לוקאלי.

פקודת unshare = מאפשרת ליצור namespace חדשים ולבודד משאבים כמו תהליכים, רשת, קבצים ועוד. כלומר שמרגע הפעלת הפקודה כל התהליכים החדשים שייווצרו מעתה והלאה יכנסו למרחב namespace חדש.
הפקודה משמשת בעיקר להרצת תהליכים בסביבה מבודדת מבלי להשפיע על מערכת ההפעלה המארחת. מקבלת כארגומנט על פי מה לעשות את ההפרדה (לדוג' PID).



לדוג' - נתונה קבוצת תהליכים P1,P2,P3... כל תהליך נוצר ע"י פקודת fork. לכל תהליך יש מספר מזהה PID ומכיל תהליכונים. התהליכון הראשי מייצג כל תהליך. התהליכים כולם נמצאים ב-global space , מרחב גלובלי. התהליכים יכולים לתקשר ביניהם עם share memory . הם מתקשרים ע"י הכרת ה-PID אחד של השני, וזה נקרא Process IDs .

אם נרצה להפריד בין התהליכים, ניצור namespace חדש ע"י פקודת unshare. כל התהליכים שייווצרו מעכשיו יעברו לשם, ויקבלו 2 סוגי מזהים:

1. <u>מזהים לוקאליים</u> – מספרים לתיאור מיקום התהליך בתוך ה-namspace . הראשון יתחיל מ-1.
2. <u>מזהים גלובליים</u> – מספרים לתיאור מיקום התהליך מתוך סך התהליכים שנוצרו ב-global space. הראשון שנוצר יהיה המספר הבא אחרי התהליך שנוצר לפניו.

- תהליכים מ-namespace גלובלי יכולים לשלוח סיגנלים לתהליכים שנוצרו ב-namespace לוקאלי ע"י המזהים הגלובליים.
- תהליכים מ-namespace לוקאלי לא יכולים לשלוח סיגנלים לתהליכים שנוצרו ב-namespace גלובלי.

מכאן – שסיגנלים יכולים להיכנס ל-namespace לוקאלי, אך לא יכולים לצאת ממנו.

- התהליך הראשון שנוצר ב-namespace הוא חסר אבא. ולכן אם ישאלו אותו מי אבא שלו הוא יחזיר 0.
- Getpid() – קריאת מערכת שמחזירה את PID התהליך שכרגע רץ.
- Getppid() – קריאת מערכת שמחזירה את PID של תהליך האב של התהליך שכרגע רץ. אם הוא הראשון יוחזר 0.

עבור 2 הפונקציות הללו:

- אם התהליך רץ ב- namespace גלובלי, היא תחזיר מזהה גלובלי.
- אם התהליך רץ ב- namespace לוקאלי, היא תחזיר מזהה לוקאלי.

שאלה 11

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?
ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת unshare (PID_NS) היה 20 .
ניתן להניח שכל קריאות מערכת מצליחות.

```
pid = fork();
if (pid > 0)  printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
else {
   unshare(PID_NS);
   pid = fork();
   if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n", pid);
   else printf (stdout, " Child's perspective on his parent: PID=%d \n", getppid());
}
```



← **<u>שאלה</u>** – 2024א מועד 68 שאלה 11

בשורה הראשונה נוצר תהליך חדש ע"י fork ב-namespace גלובלי. תהליך האב ממשיך לבצע את הקוד, והבן שנוצר כעת ממשיך לבצע את הקוד של האב גם כן מאותה נקודה ממנה הוא נוצר.

- <u>עבור תהליך **האב**: pid=19 . נמצא ב-namespace גלובלי.</u>

pid=20 , כי מנתוני השאלה זה ה-pid הכי גדול שנוצר לפני פקודת ה-unshare. והרי תהליך הבן הוא האחרון שנוצר ולכן האב קיבל את ה-Pid שלו ששווה 20. הוא נכנס ל-if, **ומדפיס את המשפט בשורה 2 באשר d=20% .**

- <u>עבור תהליך **הבן**: pid=20 . נמצא ב-namespace גלובלי.</u>

pid=0 , ולכן קופץ ל-else. בפקודת ה- Unshare הוא יוצר namespace לוקאלי, ובשורה הבאה נוצר תהליך חדש ב-fork שהולך ל- namespace החדש. לתהליך הבן של הבן החדש, המזהה הגלובלי שלו הוא 21 והמזהה הלוקאלי שלו הוא 1 , אך מכיוון שה-fork הופעל מהגלובלי Pid מתעדכן להיות pid=21. ולכן לאחר מכן הוא נכנס ל- if **ומדפיס את המשפט בשורה 6 באשר d=21% .**

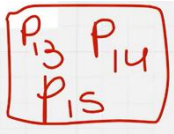
- <u>עבור תהליך **הבן של הבן**: pid=21 בגלובלי, pid=1 בלוקאלי. נמצא ב-namespace לוקאלי.</u>

Pid=0 , ולכן הוא ממשיך לעבר else ושם הוא מדפיס את המזהה הלוקאלי של תהליך האב שלו, אבל מבחינה לוקאלית אין לו אבא, הוא התהליך הראשון ב-namespace שלו - ולכן **מדפיס את המשפט בשורה 7 באשר d=0% .**

שאלה 11

מה יהיה הפלט אחרי הרצת הקוד הבא במערכת XV6 שתומכת בקונטיינרים?
ידוע שה PID הגדול ביותר שהיה במערכת לפני הרצת הקוד היה 13 .
ניתן להניח שכל קריאות מערכת מצליחות.

```
if (fork() == 0)  {
   pid = fork();
   unshare(PID_NS);
   if (pid > 0) printf (stdout, "Parent's perspective on his child: PID=%d \n",
pid);
   else printf (stdout, " Child's perspective on his parent: PID=%d \n",
getppid());
   }
```



← **<u>שאלה</u>** – שאלה 11

בתנאי בשורה הראשונה נוצר תהליך בן. מהנתונים, עבור תהליך האב pid=13 ולכן עבור תהליך הבן pid=14.

- <u>עבור תהליך האב P13 :</u>

Fork()=14, ולכן לא נכנסים לתוך התנאי והקוד מסתיים.

- <u>עבור תהליך הבן P14 :</u>

ממשיכים מהשורה הראשונה בה הוא נוצר, ושם Fork()=0, ולכן נכנסים לתנאי. בשורה הבאה נוצר עוד תהליך חדש P15 , ועבור התהליך שלנו pid=15 . ב-unshare נוצר namespace חדש, והתנאי שבשורה הבאה מתקיים ולכן אנו נכנסים ל-if ומדפיסים את המשפט שבשורה 4 באשר d=15% .

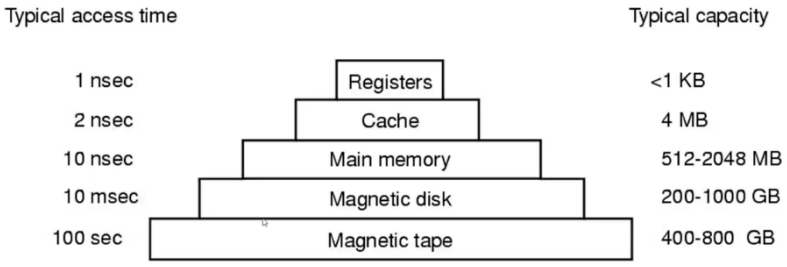
- <u>עבור תהליך הבן של הבן P15 :</u>

ממשיכים מהשורה השנייה בה הוא נוצר, ושם pid=0 . ב-unshare נוצר namespace חדש, והתנאי שבשורה הבאה לא מתקיים ולכן אנו נכנסים ל-else ומדפיסים את המשפט שבשורה 6 באשר d=14% .

נשים לב שבשאלה זו כל התהליכים נוצרו במרחב הגלובלי, וכל ה-unshare שהיו לנו היו לבאורה מיותרים כי לא נוצרו בהם תהליכים חדשים!

שיעור 5 – זיכרון וירטואלי (מדומה) , טבלת הדפים

← **מבנה זיכרון המחשב**



עיקרון ההכלה = זיכרון המחשב מורכב מחלקים שיש ביניהם הכלה (הגדול מכיל את הזיכרון של הקטן). ככל שנתקרב ל-CPU: **נפח הזיכרון קטן, מהירות גדלה (זמן הגישה מתקצר).**

מה גישת המעבד CPU לזה? אם נגיד הוא ניגש לזיכרון הראשי בחיפוש אחר משהו, אם הוא מוצא מצוין. אם הוא לא מוצא, הוא ניגש לחלק הבא בזיכרון שהוא הדיסק ומבזבז יותר זמן בחיפוש אחר הנתון. אם הוא ימצא אותו בדיסק, הוא "יפעפע" את הנתון הזה ל-RAM כי בסבירות גבוהה נרצה להשתמש בנתון הזה שוב בקרוב, ואז הגישה אליו תהיה מהירה יותר. בדר"כ זה כרוך בדריסה של משהו אחר בזיכרון הראשי. בזה מתבטא עיקרון הלוקאליות בזמן ובמקום.

עיקרון לוקאליות בזמן = אם ניגשנו לנתון מסוים (או הוראה מסוימת), סביר שניגש אליו שוב בזמן הקרוב.
עיקרון לוקאליות במקום = אם ניגשנו למיקום מסוים בזיכרון, יש סיכוי גבוה שניגש בקרוב למיקומים שנמצאים לידו.

נפרט את חלקי הזיכרון מהגדול לקטן:

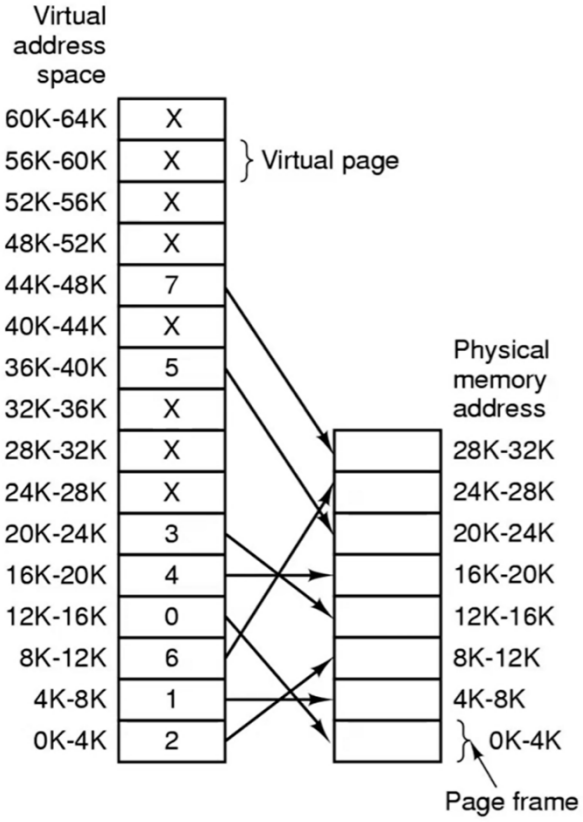
1. סרט מגנטי – magnetic tape. אמצעי אחסון נתונים ישן.
2. דיסק – SSD. פעם נקרא HDD וכבר בקושי משתמשים בו.
3. זיכרון ראשי – RAM . זיכרון פיזי.
4. מטמון – cache . יש כל מיני רמות בתוכו והן ממוספרות.
5. רגיסטרים – אוגרים. הכי מהירים ובעל נפח הזיכרון הקטן ביותר.

← **זיכרון וירטואלי**

כל תהליך חושב ש"כל הזיכרון שלו" ואינו מודע לתהליכים אחרים שפועלים במקביל אליו. לתהליך יש זיכרון וירטואלי (מדומה) - זיכרון גדול ורציף שהוא חושב שיש לו ושבפועל יכול להיות גדול יותר מהזיכרון הקיים בפועל ב-RAM , שהוא קטן ומפוצל.

נזכיר:

- בין תהליכים יש הפרדה בזיכרון
- לתהליכונים של אותו התהליך יש אזור זיכרון משותף



דף page = יחידת חלוקה **לוגית** של **הזיכרון הווירטואלי**. מערכת ההפעלה מחלקת את המרחב הווירטואלי של כל תהליך לבלוקים בגודל קבוע שהם הדפים, כלומר שמבחינת התהליך כל הזיכרון שלו מחולק לדפים. לכל דף יש מספר. הדף הראשון יקבל את המספר 0.

מסגרת frame = יחידת חלוקה **פיזית** של **הזיכרון הראשי RAM.** ה־ RAMמחולק לבלוקים באותו גודל בדיוק כמו הדפים, כלומר שהמסגרות הן המכולות הפיזיות שאליהן נטענים הדפים. לכל מסגרת יש מספר. המסגרת הראשונה תקבל את המספר 0.

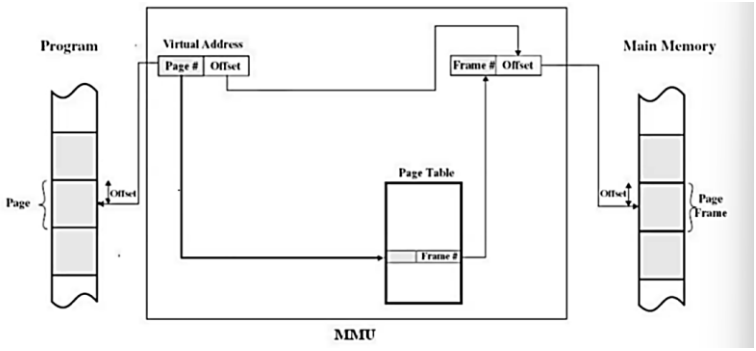
אז איך מתנהל הזיכרון?

בדוג' שלפנינו יש את הזיכרון הווירטואלי, הזיכרון שהתהליכים הקיימים חושבים שיש להם, המורכב מ-16 בלוקים של 4K. לזיכרון הפיזי, כלומר ב-RAM, יש בפועל 8 בלוקים של זיכרון שממוספרים מ-0 ל-7. ישנם תהליכים שממופים לבלוקים של הזיכרון שב-RAM, ואלו שיש בהם X הם תהליכים שמאוחסנים בזיכרון שבדיסק, כי ל-RAM כבר לא היה מקום עבורם. אם נרצה לתהליך שממוקם בזיכרון, המעבד יבצע פסיקה , יעבור לדיסק, ישלוף משם את התהליך ויעביר אותו לזיכרון ב-RAM , ע"פ עיקרון הלוקאליות בזמן ובמקום.

פסיקת דף (Page Fault) = מנגנון שקורה במערכת הפעלה כשמעבד מנסה לגשת לכתובת בזיכרון (RAM) ששייכת לתהליך, אבל העמוד (Page) שמכיל את הכתובת הזו לא נמצא כרגע בזיכרון הפיזי. כלומר, התהליך "מבקש" מידע, אבל הוא לא נמצא ב- RAM , נוצרת פסיקה (Interrupt) , ומערכת ההפעלה צריכה להביא את העמוד מהדיסק אל ה-RAM.

← טבלת דפים

בכותרת program שבתמונה, מוצג הזיכרון הווירטואלי, המדומה, המחולק לבלוקים הנקראים דפים. הדפים מכילים כתובות. איך נגיע לכתובת ספציפית? ע"י offset , וכך אנחנו יודעים מתחילת הדף המסוים לאן לקפוץ על מנת להגיע לכתובת המסוימת. יכולים להיות offset זהים, אך כל אחד מהם יהיו משויכים לדף אחר ולכן יובילו לכתובת אחרת. מכיוון שגודל הדפים שווה לגודל המסגרות בזיכרון



הראשי המתאימות להם, אז ה-offset של כתובת ספציפית זהה גם בדף וגם במסגרת המתאימה לו. משם, ה-offset יוביל אותנו אל כתובת ווירטואלית (המשויכים לה דף מסוים ו-offset) וממנה נרצה להגיע אל הכתובת הפיזית, אל המסגרת המתאימה. איך נעשה זאת? ע"י טבלת הדפים. לכל תהליך יש טבלת דפים משלו. מספר הדף הוא אינדקס בטבלת הדפים ובעזרתו מוצאים את מספר המסגרת המשויכת לאותו דף.

ה-MMU תתרגם את הכתובת הווירטואלית לכתובת הפיזית ע"י כך שהיא עובדת בזמן של החלפת הקשר, שכאשר היא מתרחשת פוינטר בתוך MMU שנקרא Page Table Base Register (PTBR) שמתעדכן ומצביע כל פעם על טבלת הדפים של התהליך החדש שעבר למצב running. בטבלת הדפים יש Page Table Entry) PTE), היא מחולקת לרשומות שלכל אחת קוראים PTE וכל אחת כזו מתארת דף וירטואלי אחד ואיך (או אם) הוא ממופה לזיכרון הפיזי. בתוך כל PTE יש ביטים מיוחדים שמגדירים את מספר הדף, ואחד החשובים בהם הוא ביט V (Valid bit) - ביט הנוכחות בRAM , מסמן אם התרגום בין כתובת וירטואלית לכתובת פיזית קיים ונכון.

Offset היסט = כמה בטים צריך לקפוץ מתחילת הדף (הבסיס) עד לכתובת שאנו רוצים לעבוד איתה (offset). בזיכרון מחשב, כל משתנה או איבר במבנה נתונים נשמר בכתובת. הכתובת מחולקת ל:

- base = בסיס, נקודת התחלה (למשל כתובת העמוד בזיכרון).
- offset = היסט, מיקום יחסי בתוך אותו עמוד/מבנה.

MMU = Memory Management Unit = יחידת ניהול זיכרון = זהו רכיב חומרה (חלק מהמעבד) שאחראי על תרגום כתובות וירטואליות לכתובות פיזיות ,ועל ניהול הגישה לזיכרון. הוא גם מונע מתהליך לגשת לזיכרון שלא שייך לו (למשל, תהליך אחד לא יכול לקרוא נתונים של תהליך אחר).

ה־ MMU נעזר בטבלת עמודים (Page Table) שמנוהלת ע"י מערכת ההפעלה. הטבלה אומרת איזה דף (Page) וירטואלי ממופה לאיזו מסגרת (Frame) פיזית:

- אם V = 1 ← הדף ממופה למסגרת פיזית ב- RAM. כלומר, התהליך יכול לגשת אליו ← MMU מחזיר את כתובת המסגרת
- אם V = 0 ← הדף נמצא בדיסק ולא ב-RAM (או שהתהליך לא מורשה להשתמש בו) ← MMU מכריז על Page Fault

נוסחאות חשובות לשאלות:

עבור גודל דף של 2^n בתים
וגודל כתובת של m ביטים:
ה-offset יהיה n הביטים הימניים,
מספר הדף יהיה m-n הביטים השמאליים

גודל דף = גודל מסגרת

offset = גודל דף % כתובת

מספר הדף = כתובת / גודל דף (וניקח את החלק השלם בלבד)

```
Virtual Address (m bits) = [ Page Number (m-n bits) | Offset (n bits) ]
```

גודל טבלת הדפים = גודל שורה בטבלת הדפים X גודל מספר הדף

כתובת פיזית = offset + (גודל דף X מספר המסגרת)

מספר PTE הקיים בטבלה = גודל דף / גודל PTE

יחידות מידה בביטים:
K = קילו = 2^10
M = מגה = 2^20
G = ג'יגה = 2^30

עבור מרחב זיכרון בגודל 2^n בתים (byte), כל כתובת תהיה מורכבת מ-n ביטים (bit , סיביות)

עבור 2^n כניסות הדרוש לטבלת דפים (PTE), כל מספר הדף יהיה מורכב מ-n ביטים

איך לעבור ממספר בינארי לעשרוני?
הספרה הימנית היא במקום 0, השמאלית לה היא במקום 1 וכן הלאה.
עבור מספר בינארי עם n ספרות נבצע:
מספר עשרוני = $2^0 \cdot digit + \ldots + 2^n \cdot digit$

לדוג':
במספר 1011 → הספרות הן:
- ספרה ימנית (1) = מקום 0 → 2^0
- ספרה הבאה (1) = מקום 1 → 2^1
- ספרה הבאה (0) = מקום 2 → 2^2
- ספרה שמאלית (1) = מקום 3 → 2^3

- $1 = 2^0 \times 1$
- $2 = 2^1 \times 1$
- $0 = 2^2 \times 0$
- $8 = 2^3 \times 1$

$11 = 1 + 2 + 0 + 8$

← **שאלה** – 2024ב שאלה 4

ה"בלוקים" שהזיכרון הווירטואלי והראשי מחולקים הם בגודל 8192 בתים, כלומר שזהו גודל של כל דף/מסגרת. לכן דף 0# יתחיל בכתובת 0 ויסתיים בכתובת 8191 , ודף 1# יתחיל בכתובת 8192 ויסתיים בכתובת 16383 – ולכן הכתובת 9000 תמוקם בדף 1# . ביט V של הדף ע"פ טבלת הדפים הוא 1 , ולכן יש לדף הזה תרגום תקף, כלומר קיימת לו מסגרת. וע"פ הטבלה מדובר במסגרת 2. נציב בנוסחאות ונחשב את הכתובת הפיזית:

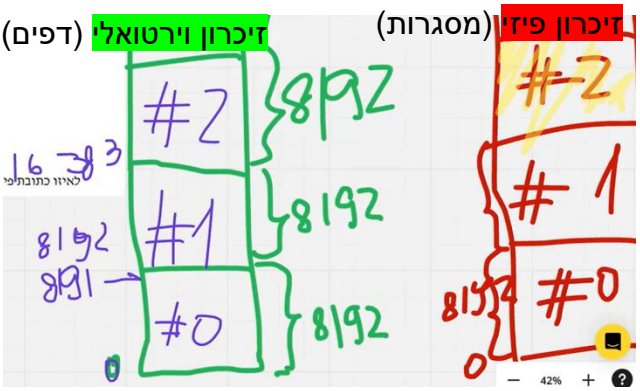
טבלת הדפים של תהליך במערכת עם זיכרון וירטואלי נראית כך. כל המספרים הם דצימליים, מתחילים מאפס, וכל הכתובות הן כתובות של ביית בזיכרון. גודל הדף הוא 8192 בתים(8KByte).

Page Number	Valid bit	Frame Number
0	1	1
1	1	2
2	0	0
3	1	0
4	0	3
5	0	0

לאיזו כתובת פיזית, אם ניתן לחשב, תמופה הכתובת הוירטואלית הבאה: 9000 ?

Offset = 9000 % 8192 = 808

כתובת פיזית = 808 + (2 X 8192) = 17192



← **שאלה** – 2025א שאלה 4

טבלת הדפים של תהליך במערכת עם זיכרון וירטואלי נראית כך. כל המספרים הם דצימליים, מתחילים מאפס, וכל הכתובות הן כתובות של ביית בזיכרון. גודל הדף הוא 4096 בתים(4Kbyte).

Page Number	Valid bit	Frame Number
0	1	1
1	1	2
2	0	0
3	1	0
4	0	3
5	0	0

לאיזו כתובת פיזית, אם ניתן לחשב, תמופה הכתובת הוירטואלית הבאה: 9000 ?

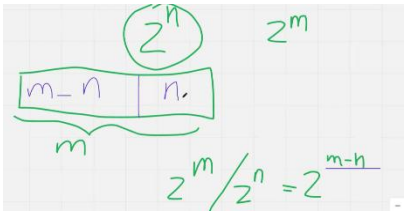
גודל דף = 4096

כתובת = 9000

מספר דף = 9000 / 4096 = 2

עבור דף #2 , הביט V שווה 0 , ולכן התרגום לא תקף ולכן <u>לא ניתן לחשב</u>. מבאן, שבמערכת ההפעלה מתרחשת page fault והמעבד ייגש מה-RAM לדיסק על מנת לחפש שם את המסגרת המבוקשת.

← **שאלה**



נתון:

גודל הדף 2^n בתים.

גודל מרחב הזיכרון הווירטואלי (כלומר מספר הכתובות) הוא 2^m .

מסקנות:

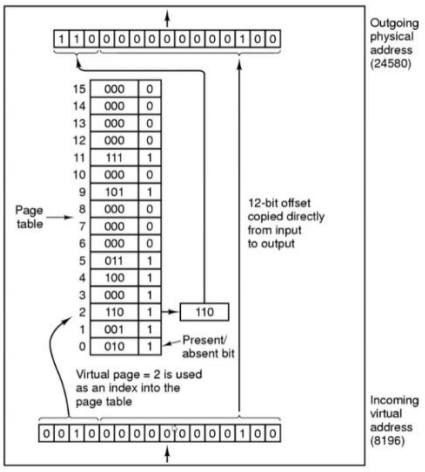
מבאן, שכמות הדפים הקיימים היא 2^m / 2^n = 2^(**m-n**)

ונסיק שכל כתובת תהיה מורכבת מ-m ביטים וכל דף מורכב מ-n ביטים.

מכיוון שיש 2^m כתובות, מספר הכתובת הראשונה הוא 0 ומספר הכתובת האחרונה יהיה 1-(2^m).,

מכיוון שגודל הדף הוא 2^n אז גודל ה-offset המקסימלי הוא n, לכן מספר הביטים שצריך כדי לייצג את ה-offset הוא n . והביטים שנשארו מגודל הכתובת, כלומר **m-n** ייצגו את מספר הדף.

← **שאלה**



נתון MMU עבורו 16 4 KB pages (כלומר שהוא עובד עם 16 דפים שכל אחד בגודל 4KB) , ובאמת ניתן לראות שבטבלת הדפים יש 16 דפים הממוספרים מ-0 עד 15.

תמיד נרצה לעבור לגודל דף ע"פ בתים: 4KB = 2^2 * 2^10 = 2^12

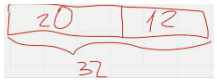
אז אם גודל הדף הוא 2^12 בתים, אז עבור כתובת בלשהי 12 הביטים הימניים יהיו ה-offset .

למטה יש את הכתובת, ולמעלה את המסגרת המתאימה לה. 12 הביטים הימניים זהים בשניהם כי ה-offset של דף ומסגרת תואמים הוא זהה.

4 ביטים הימניים של הדף הוא מספר הדף – שיתורגם מביטים ל-2 , ואז נלך לטבלת הדפים ועבור דף 2 הביט V שווה 1 כלומר קיים תרגום ולכן המסגרת היא 110 .

נזכור – הכפלה של ביטים ב-2 גורמת להזזה אחת שמאלה של הביטים – כלומר הספרה השמאלית נעלמת, ומימין נוסף 0.
לכן לדוג': 0001 X 2 = 0010

← **שאלה**



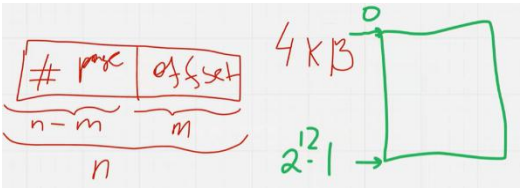
- אזור הזיכרון הווירטואלי שכל תהליך חושב שיש לו הוא: 2^32 = 4 X 2^30 = 4 GB
- נעביר את גודל הדף לביטים: 4 KB = 2^2 X 2^10 B = 2^12bits ומכאן שה-offset בגודל של 12 ביטים.
- אם כל כתובת היא בגדול 32 ביטים, מספר הדף הוא בגודל 32-12=20bits
- אם מספר הדף הוא בגודל 20 ביטים, או 2^20 בתים , ולכן גודל טבלת הדפים יהיה: 2^20 X 4 = 2^22B = 4MB

נתון כי:
- כתובת וירטואלית כוללת 32 bits
- יחידות המען הקטנות ביותר הן בגודל 1 byte
- גודל כל דף הוא 4 kb
- גודל שורה של טבלת דפים הוא 4 bytes

מהו גודל טבלת הדפים?

שיעור 6 – טבלת דפים , טבלת דפים רב שכבתית

← **מילה (יחידת מען מינימלית) בגודל שאינו 1B**



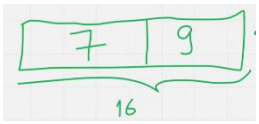
נקודה חשובה: offset זה המרחק מתחילת הדף שבו ממוקמת הכתובת.
מכאן, שעבור כתובת 0 אז offset=0 , ועבור גודל דף של לדוג' 4KB = 2^12B זה אומר שהוא יכול להכיל 2^12 כתובות , ולכן הכתובת האחרונה היא 2^12-1 (כי התחלנו מכתובת 0) וזה יהיה גם ב-offset שלה. אז כמה ביטים מינימום נצטרך על מנת שנוכל לתאר כל offset ? סה"כ 12 ביטים על פי החזקה של הכתובת האחרונה. **השיטה הזו מאוד נוחה כאשר גודל הדף הוא חזקת 2.**

מילה = יחידת המען המינימלית. אנחנו לא יכולים לכתוב פחות ממילה. בדר"כ תהיה 1B.
לדוג' – אם מביאים לנו מילה בגודל 2B :,

- אנחנו לא יכולים לקרוא, לכתוב או לקדם כתובת בפחות מ-2 בתים. כלומר שהכתובת הראשונה תהיה 0, הבאה 2, אחריה 4 וכן הלאה.
- שאר הכתובות, לדוג' כתובת 1 , היא לא תקינה. מבחינה חומרתית, כאשר נוצרה כתובת 0 אז בכתובת 1 בביכול כבר השתמשו. כשאר נוצרה כתובת 2 אז בכתובת 3 כבר השתמשו (הכוונה לכל הכתובות בין הכתובת הנוכחית לבין הכתובת התקינה הבאה) וכן הלאה.
- יכולות להתבצע קפיצות במעבד שהן גדולות יותר - בכפולות של 2.

עבור הדוג' ממקודם גודל מילה היה 1B. מה יהיה ה-offset אם גודל המילה יהיה 2B ?
מספר ה-offset יהיה קטן פי 2 , כי אנו ניגשים רק לזוגיים. אז בעצם מה שנשאר לעשות זה את הכתובת האחרונה ב-2 , נקבל 2^11-1 ולכן מספר הביטים שנצטרך עבור ה-offset יהיה 11.
מסקנה: כאשר נתון **גודל מילה שונה מ-1B**, נחלק את גודל הדף במילה ואז נקבע את ה-offset

במערכת עם ניהול זיכרון באמצעות הדפדוף (paging) נתון כי:
- כתובת מדומה (virtual address) הינה בת 16 סיביות
- גודל הדף במערכת הוא 512 bytes



← **שאלה**

א. עבור גודל מילת זיכרון של 1B :
- כתובת מדומה = 16bits , ולכן ← מרחב הזיכרון המדומה = 2^26B
- גודל הדף = 2^9B = 512B ולכן ← offset = 9bits
- ומכאן שגודל מספר הדף = 16-9 = 7bits
- ולכן כמות הדפים במרחב הזיכרון המדומה = 2^16B / 2^9B = 2^7

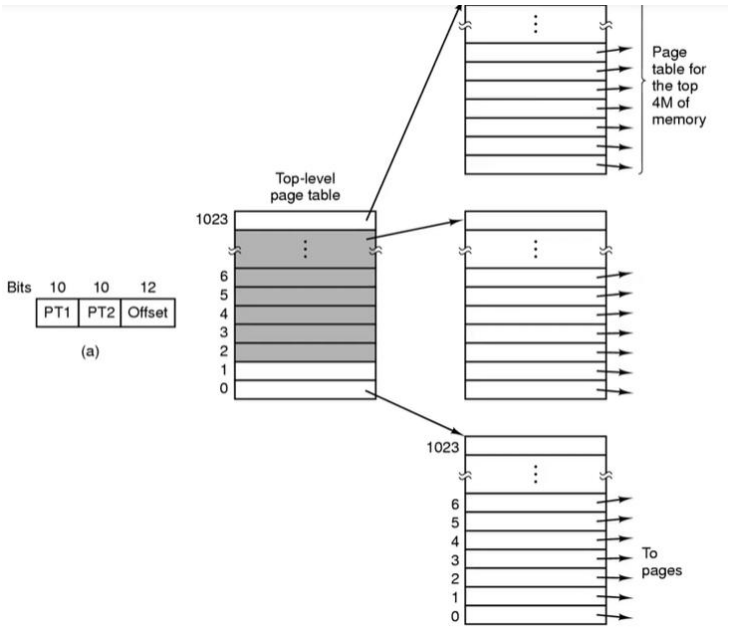
ב. עבור גודל מילת זיכרון של 2B :
- כתובת מדומה = 16bits , ולכן ← מרחב הזיכרון המדומה = 2^26B
- גודל הדף = 2^8B = **2B ** 2^9B ולכן ← offset = 8bits
- ומכאן שגודל מספר הדף = 16-8 = 8bits
- ולכן כמות הדפים במרחב הזיכרון המדומה = 2^16B / 2^8B = 2^8

← **שאלה**

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון:
- גודל הדף הוא 2K
- כתובת וירטואלית בת 16 סיביות כאשר 10 סיביות מיועדת להיסט (offset) בתוך הדף

- גודל מספר הדף = 16-10 = 6 ביטים
- עבור גודל מילה n , ניצור משוואה: n = 2^offset / גודל הדף
  ונציב 2K / n = 2^10 ← n = 2^11 / 2^10 = 2
- וקיבלנו שגודל מילת הזיכרון הוא 2B

← **<u>טבלת דפים רב שכבתית</u>**



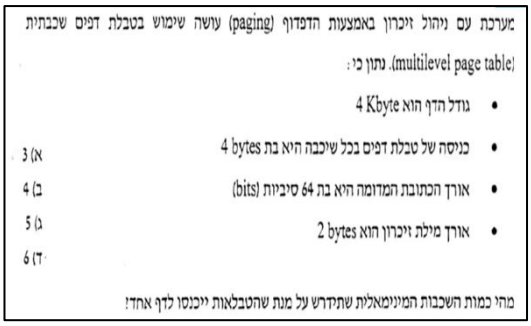
טבלאות הדפים הן חלק מהזיכרון, והוא כלי עזר גדול שתופס הרבה מקום בזיכרון במצב בו מרחב הכתובות גדול. לכן זה יגרום להמון פסיקות, כי למסגרות הפיזיות כמעט ולא נשאר מקום. לכן הומצאה שיטת דפדוף רב שכבתית, שבעזרתה עובדים ברמות שונות: במקום טבלת דפים אחת ענקית, מפרקים אותה לטבלאות קטנות - תהיה לנו טבלת דפים קטנה בדף עצמו (שאין לה קשר למספר הדפים שאנו רוצים לתרגם) עם כמה כניסות (בדוג' בתמונה יש 1024 כניסות, וסה"כ ל-2 רמות של טבלאות) וכל כניסה כזו מהווה מצביע לכתובת לטבלה קטנה אחרת, שתצביע לטבלה נוספת וכן הלאה עד שנגיע לרמת המסגרת הפיזית – כלומר לטבלה האחרונה שהיא זו שתבצע את התרגום מכתובת וירטואלית לפיזית. כלומר, כל חלק מהכתובת הווירטואלית מתפקד כאינדקס לרמה אחרת בטבלת הדפים.

- כל הטבלאות בכל הרמות הן באותו גודל (מספר הכניסות, ה-PTE, זהה) אך תוכן כל טבלה הוא שונה (מצביעים שונים)
- הרמה הראשונה היא הרמה הראשית
- ע"פ עיקרון הלוקאליות בזמן ומקום, אנו לא משתמשים כל הזמן בכל טבלת הדפים הענקית, אלא משתמשים במקטע ממנה. כלומר סביר להניח שנשתמש רק באחת מתת-הטבלאות אליהן חילקנו את הטבלה הענקית. אז את הטבלאות שאנו משתמשים בהן (כלומר הטבלאות שמשתמשים בהן בתרגום בפועל) ישמרו ב-RAM, והשאר ישמרו בדיסק.

<u>מה היתרון והחיסרון של טבלת דפים רב שכבתית ?</u>

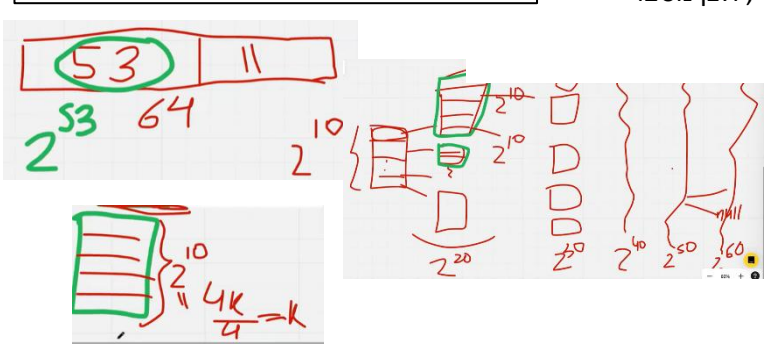
יתרון ← תיחזוק הטבלה הרגילה הענקית לוקח הרבה מקום ואז לא נשאר מקום למסגרות הפיזיות שמתאימות לדפים, ואז רוב המסגרות הפיזיות יושבות בדיסק, מתקבלות הרבה פסיקות שדורשות המון זמן ובעצם אין אופטימיזציה – הזמן לא מנוצל כמו שצריך.

חיסרון ← התיחזוק של כל הטבלאות הוא קשה יותר, וגם עם הטבלאות הללו יש לפעמים פסיקות דף.



← **שאלה**

- גודל דף = 4KB = 2^12B ← נחלק את גדול הדף במילה
  2^12B / 2B = 2^11 ← ולכן offset=11bits
- גודל מספר הדף = 64-11 = 53 bits
- ומכאן שמספר הכניסות לטבלה אחת ענקית הוא 2^53
- על פי השאלה, כל טבלה בשיטת הדפדוף הרב שכבתי אמורה להצליח להיכנס לדף אחד, כלומר היא יכולה להיות בגודל של מקסימום 4KB . כל כניסה שווה 4B , ולכן מספר הכניסות לטבלה אמור להיות 4KB / 4B = K = 2^10
- לכן כמה פעמים 2^10 יכנס ל-2^53 ? (הרי כל הטבלאות שמרכיבות את הטבלה הענקית צריכות להיות בגודל זהה) 5 פעמים עם שארית, ולכן ידרשו 6 <u>רמות</u> של טבלאות סה"כ.
- שימו לב – ברמה 6 לא נעשה שימוש בכל הטבלאות. לכן הערך שהן יכילו הוא Null .



← **שאלה**

נניח שיש לנו טבלת דפים בשתי רמות. אורך כל כניסת טבלת הדפים(PTE) הוא 4 בתים.
נניח שאנו רוצים שכל טבלת דפים בודדת(ראשית וכל אחת ממשניות) תתפוס בדיוק דף זיכרון אחד.

מה צריך להיות הפורמט החלוקה לסיביות של כתובת וירטואלית באורך 32 סיביות שמתאים לזה?
הערה: למרות שלא ניתן גודל הדף, ניתן לחשב אותו ממשוואה שתאפשר את זה או בניסוי וטעיה.
תשובה, חישוב והסבר

Directory (10b)	Table (10b)	Offset (12b)

- נסמן: page number = 32-x , offset=x
- לבן גודל הדף יהיה 2^x
- מכאן שמספר הכניסות (PTE) הקיים בטבלה בודדת יהיה 2^x / 4 = 2^(x-2) ולבן בכל הטבלאות יהיה מספר כניסות כולל של: **2^(2X-4)** = 2^(x-2)*2^(x-2)
- אפשר גם לחשב את מספר הכניסות (PTE) הדרוש לטבלה ענקית מגודל מספר הדף, כך שנקבל **2^(32-X)**
- ואז קיבלנו 2^(2X-4) ≤ 2^(32-x) (מספר ה- entries שיכולים להיכנס בדף יחיד ≤ מספר ה- entries שנדרשים בטבלה) ← 32-x ≤ 2X-4 ← 36 ≤ 3X ← X ≥ 12 ונסתפק במינימום הנדרש ולכן **x=12**
- ומכאן שקיבלנו: offset=12bits , page number=20bits שיחולקו כך ש-10 השמאליות לטבלה הראשית, ו-10 הימניות לטבלה המשנית.

נשים לב: אם למשל היינו מקבלים X>=12.8 היינו בוחרים ב x=13

← **קובץ החלפה**

איך מוצאים דף שפינינו אל הדיסק מבלי לבזבז זמן ולחפש בכל הדיסק?
את הדפים שהתפנו שמים במקום בדיסק הנקרא קובץ החלפה (Swap File). זהו אזור בדיסק שבו מערכת ההפעלה משתמשת כאילו היה חלק מה-RAM. כשניגשים לנתונים בקובץ ההחלפה יש לנו 2 סוגי נתונים:

- נתונים (דוגמת כתובות קיימות) ← בדר"כ מתעדכנת ועוברת שינויים
- קוד (שעובר קומפילציה ואז מתבצע) ← בדר"כ קבוע ולא עובר שינויים

איך קבצי ההחלפה מאורגנים? לא רוצים לערבב בין נתונים (דאטה) לבין הקוד, גם מבחינת מקביליות הגישה וגם מחשש לדריסה וגישות משותפות – לכן הם מאוחסנים בנפרד בקבצי ההחלפה. בנוסף, יכולים להתרחש לנו 2 סוגי page fualte , כשלא מוצאים ב-RAM נתון מסוים או קוד מסוים - ואז יש סיגמנט של נתונים ויש סיגמנט של קוד, שבהתאם אליו ניגשים לאזור המתאים בקובץ ההחלפה שבדיסק ומוציאים אותו משם. בנוסף, יש קוד שמעורבב עם נתונים, וגם אותו נמקם בקובץ ההחלפה.
Bytecode (BC) = ייצוג ביניים של קוד תוכנה, שנמצא באמצע הדרך בין קוד מקור (Source Code) לבין קוד מכונה(Machine Code) . לאחר שכותבים קוד, עושים לו קומפילציה ואז הקובץ הזה נוצר והוא זה שירוץ בפועל על החומרה.

← **שאלה** – 2025א מועד א1 שאלה 8

שאלה 8 (15 נקודות)

א. (5 נק')

קבוצת מעצבי מערכות הפעלה הציע רעיון הבא כדי להפחית את גודל קובץ החלפה SWAP FILE:
לא לשמור את קוד התוכנית בקובץ החלפה בכלל, אלא להביא דפי קוד ישירות מקובץ ההרצה הבינארי בכל פעם שיש צורך בכך. בהנחה שבמערכת לא קיימת הפרדה בין איזור הקוד לאזור הנתונים, איזה מידע נוסף יהיה צורך לשמור בטבלת הדפים?

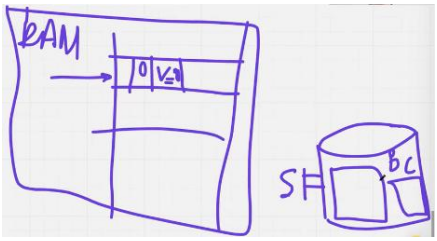
ב.(5 נק')

הסבירו את היתרון הנוסף הקשור למהירות ביצוע של השיטה מעבר לחיסכון במקום.

ג.(5 נק')

האם אותו הרעיון יכול להתאים לנתונים סטטיים שגם הם נמצאים בקובץ הרצה בינארי?

א. השאלה מציעה רעיון: **במקום לשמור את קבצי הקוד של התוכנית בקובץ ההחלפה (swap file), נטען אותו תמיד מהקובץ המקורי שנמצא בדיסק (קובץ ההרצה של התוכנית).**
השאלה: איזה מידע נוסף יהיה צורך לשמור בטבלת הדפים בעקבות שינוי זה?
התשובה: צריך לשמור את המיפוי של כל דף קוד לכתובת שלו בקובץ המקורי. כלומר, עבור דפים שהם קוד – בטבלת הדפים נוסיף אינדיקציה שהם לא נמצאים ב-swap file, אלא שייכים לקובץ הקוד בדיסק, ובאיזה offset (מיקום) אפשר להביא אותם.



ניתן דוגמא בתמונה: נתונה טבלת דפים, ואנו רוצים להבין איפה קובץ קוד מסוים נמצא. נלך ל-PTE שלו, שם יש לנו את ביט V שמראה לנו האם יש תרגום של הקובץ לכתובת פיזית. **במידה ונעשה את השינוי הנתון בשאלה, ביט V יהיה שווה 0 (כלומר שהקובץ לא שמור ב-RAM אלא בדיסק) ואז אוטומטית ניגש לחפש את הקובץ בדיסק אבל בקובץ ההחלפה "כמו תמיד"** – אבל הרי אמרנו בשאלה שקבצי הקוד יהיו שמורים במקומם המקורי באופן ישיר, לכן יש צורך להוסיף לטבלת הדפים ביט שנקרא לו למשל ביט O (מלשון original) שיראה לנו מאיפה להוציא את הדף:

- אם הוא שווה 1 ← מדובר בדף המכיל נתונים, לכן ממוקם בקובץ ההחלפה (כמו תמיד)
- אם הוא שווה 0 ← מדובר בדף המכיל קוד, לכן ממוקם בקובץ Bytecode (השינוי שעשינו, כי לא רצינו כמו תמיד להכפיל את הקוד ואת העותק למקם בקובץ ההחלפה כי רצינו ליצור קבצי החלפה קטנים יותר)

בקיצור: בטבלת הדפים **צריך שדה נוסף (ביט O שהמצאנו)** שמציין מיקום הדף במידה וביט V שלו שווה 0 , כלומר במידה והוא נמצא בדיסק. המיקום יכול להיות בקובץ ההחלפה (אם זה קובץ נתונים) או בקובץ הקוד המקורי (Bytecode)

ב. מעבר לחיסכון במקום, היתרון של הרעיון שהוצע הוא גם מבחינת מהירות - לא מבצעים כתיבה של דפי קוד ל־ swap (כתיבה לדיסק היא איטית), וטעינת קוד נעשית ישירות מקובץ המקור, שחוסך זמן.

ג. נתונים סטטיים = נתונים קבועים שהוגדרו מראש (קבועים, דיפיינדים, טבלאות ניהול, קבצי ג'ייסון וכו').
צריך להבדיל בין שני סוגי נתונים סטטיים, כאלה שלא ישתנו לכל אורך הקוד ולא יעברו עדכון (דוגמת constant) להם השיטה הזו תהיה טובה. אבל לנתון סטטי שיכול להשתנות במהלך הקוד (דוגמת int x=4) השיטה הזו לא תעבוד.

שיעור 7 - מטמון

← **שאלה** – 2025א מועד ב שאלה 8

במערכת עם זיכרון וירטואלי ודפים בגודל 4 KB נתונה טבלה הבאה עבור תהליך P (המספרים הם בינאריים):

מספר מסגרת הזיכרון Page frame number	סיבית הנוכחות Present bit	#השורה(רק סימון מספר שורה, אינו חלק מן הטבלה)
0010	0	0
1000	1	1
1011	1	2
1010	0	3
0101	1	4
1001	1	5
1100	0	6
0010	1	7

את התשובות יש לכתוב כמספר בינארי.

א. אילו ערכים או תוצאות יחזיר ה-MMU עבור שתי הכתובות הוירטואליות הבאות של גישה לזיכרון ראשי(אורך הכתובת הוירטואלית במערכת אינה סטנדרטית):

0011000010010010

0111001100110001

ב. האם הזמנים שיידרשו למערכת ההפעלה לבצע את שתי הדרישות בסעיף א' זהים? נמקו.

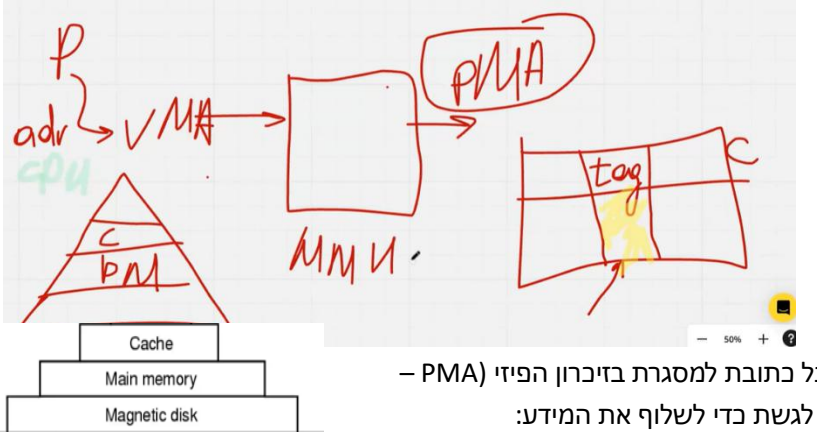
ג. ומה גודלו של מרחב הזיכרון הווירטואלי המרבי האפשרי של התהליך במערכת אם אורך כתובת וירטואלית כמו בטבלה?

ד. מה הוא הגודל המרבי של הזיכרון הפיזי במערכת הנ"ל אם אורך שדה מספר מסגרת כמו בטבלה?

- גודל כתובת = 15bits (ספרנו את מספר הספרות של הכתובות הנתונות)
- גודל דף = 2^{12}B = 4KB
- ולכן 12bits = offset
- ולכן גודל מספר הדף = 15-12 = 3bits
- דרך נוספת : יש לטבלת הדפים 8 כניסות, כלומר 2^3 , ומכאן שגודל מספר הדף יהיה 3bits.
- הכתובת הראשונה היא עבור דף מספר 001 ביט .
  נעביר לעשרוני: 2^0 = 1
  עבור דף 1, ביט V שווה 1 (מטבלת הדפים) ולכן הוא ממופה למסגרת פיזית ב-RAM , למסגרת 1000 ביט.
  לכן ה-MMU יחזיר את כתובת המסגרת הפיזית, כלומר נצמיד למספר המסגרת את ה-offset של הדף (הרי הם זהים) ונקבל שיוחזר 1000100010010010
- הכתובת השנייה היא עבור דף מספר 011 ביט.
  נעביר לעשרוני: 2^1+2^0 = 3
  עבור דף 3, ביט V שווה 0 (מטבלת הדפים) ולכן הוא לא ממופה למסגרת פיזית ב-RAM , כלומר שה-MMU יכריז על page fault.

נשים לב: הזיכרון הווירטואלי הוא מדומה, הוא חושב שהוא י כול להיות באיזה גודל שהוא רוצה, גם אם בזיכרון הפיזי אין לו באמת מקום.
במקרה הזה שבשאלה, הזיכרון הווירטואלי דווקא חושב שהוא יותר קטן מהזיכרון הפיזי (וזה גם בסדר! שיחשוב מה שבא לו 😊). ניתן לראות זאת על פי מספר הביטים שבהם מיוצגים הדפים והמסגרות, 3 ביטים לדף וירטואלי לעומת 4 ביטים למסגרת פיזית.

- בכתובת הראשונה הוחזרה כתובת של מסגרת פיזית מה-RAM (שאנחנו כבר נמצאים בו), ובכתובת השנייה התרחשה פסיקת דף שדורשת מעבר מה-RAM לדיסק, ולכן היא תיקח זמן רב יותר.
- כל כתובת של דף וירטואלי מורכבת מ-15 ביטים, ולכן גודל מרחב הווירטואלי הוא 2^{15}B = 2^5KB = 32KB. ולכן מרחב הדפים הוא מ-0KB עד 32KB
- כל כתובת של מסגרת פיזית מורכבת מ-16 ביטים, ולכן גודל הזיכרון הפיזי הוא 2^{16}B = 2^6KB = 64KB. ולכן מרחב המסגרות הוא מ-0KB עד 64KB



← **מטמון**

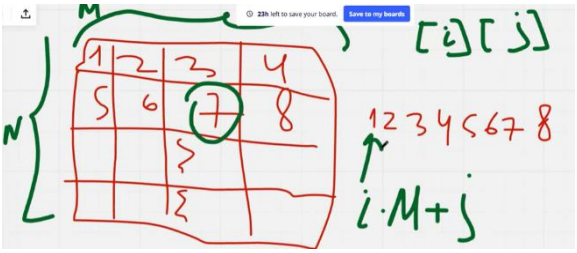
זיכרון המטמון הוא קטן ומהיר. איך הוא עובד?
נניח יש תהליך שאחד מהתהליכונים שלו מריץ קוד שבו יש גישה לזיכרון לכתובת ווירטואלית מסוימת (virtual memory address – VMA), כלומר הגישה היא לזיכרון הווירטואלי (VM) . משם מגיעים ל-MMU שמתרגמת את הכתובת הווירטואלית לפיזית ע"י טבלת התרגום. נניח ואין פסיקת דף והתרגום הצליח כי הכתובת נמצאת בזיכרון הראשי – אז אנו נקבל כתובת למסגרת בזיכרון הפיזי (PMA – physical memory address). משם צריך להבין לאן אנו צריכים לגשת כדי לשלוף את המידע:

- אם הכתובת לא נמצאת בזיכרון הראשי ← תתרחש פסיקת דף ונלך לדיסק
- אם הכתובת נמצאת בזיכרון הראשי ← נבדוק קודם האם היא נמצאת במטמון, שהגישה אליו מהירה יותר מהזיכרון הראשי. בזיכרון המטמון יש מספר צורות ארגון , ואחת מהן היא **חלוקה לבלוקים** שבהם המידע נשמר. איך יודעים אם יש מיפוי של הכתובת למטמון (Cache hit) או שאין (Cache miss)? על ידי Tag Directory , שמאמת שהבלוק שנמצא כרגע במטמון הוא באמת הבלוק המתאים מהזיכרון הראשי. מוציאים ממנו אינדקס ובודקים אם הוא תואם לטאג מהכתובת: אם כן, אז זה אומר שהמידע נמצא במטמון וניקח אותו מהבלוק המתאים (Hit). אם לא, המידע לא נמצא במטמון וניקח אותו מהזיכרון הראשי (miss).

← **<u>איך "משטחים" מטריצה (מערך דו ממדי) בזיכרון?</u>**

כמו בדוגמא בתמונה. בשיטת שורה אחר שורה.

עבור מטריצה עם N שורות ו-M עמודות, איך נחשב את האינדקס במטריצה המשוטחת עבור איבר במקום [j][i] ? ע"י הנוסחה: **i·M+j**



תמיד נחפש איברים בזיכרון על פי <u>שורות</u> (הגישה האופקית) :

בדיסק, אנו ניגשים לכתובת ששייכת לאיזשהו דף שמתמפה למסגרת פיזית. בגלל שניגשנו אליו, הדיסק "מפעפע" אותו אל הזיכרון הראשי (כי בסבירות גבוהה ניגש אליו שוב בקרוב). אבל לא רק הכתובת עצמה עוברת לשם, אלא הדף (הבלוק) כולו (נניח ובדוגמא ניגשנו לכתובת 1 שנמצאת בדף שהוא השורה הראשונה, אז כל השורה הראשונה תעבור לזיכרון הראשי). גישה ע"פ עמודות היא פחות עדיפה, כי זה יכול ליצור פסיקות דף או Eviction.

במטמון, הבלוקים מסודרים בצורה שטוחה, לכן הגישה האופקית עדיפה גם שם.

אם עוברים על המטריצה לפי **שורות** – הנתונים ברצף, ולכן נשארים יותר זמן במטמון ← פחות Evictions ופסיקות

אם עוברים לפי **עמודות** – כל גישה "קופצת" למקום אחר בזיכרון, מה שגורם לטעון כל פעם בלוק חדש ← הרבה Evictions ופסיקות

Cache Eviction = תהליך שבו בלוק קיים במטמון מוצא החוצה כדי לפנות מקום לבלוק חדש.

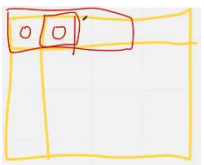
במערכת מחשב יש מטמון שמאחסן באופן זמני נתונים מהזיכרון הראשי כדי לזרז גישה. המטמון מוגבל בגודל ,ולכן לא תמיד יש מקום לכל הנתונים. לכן כשצריך להכניס בלוק חדש ואין מקום חייבים להוציא בלוק ישן. תהליך זה מתרחש כשיש Cache Miss: הנתון לא נמצא במטמון, אז כדי להביא את הבלוק מהזיכרון הראשי, צריך לפנות מקום ומתבצע Eviction

← **<u>שאלה</u>** – 2025א מועד ב שאלה 4

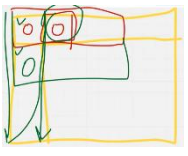
התשובה היא ד'.

תוכנית 1 עוברת על האיברים לפי שורות, תוכנית 2 לפי עמודות.

בתוכנית 1, מרגע שהיא הביאה את האיבר הראשון במטריצה היא הביאה אתו את כל הבלוק בו הוא נמצא והיא קירבה אותו למעבד, ואז יש סיכוי גדול שהמעבר לאיבר השני שמימין (כי הולכים לפי שורות) לא ייצור פסיקות או cache miss כי הוא כבר נמצא במטמון.



בתוכנית 2, האיבר השני שהיא תעבור אליו הוא האיבר שמתחת לאיבר הראשון, וזה יגרום להבאת דף (בלוק) נוסף מהזיכרון הראשי שבו איבר זה נמצא (פסיקת דף). כך זה ימשיך, יהיו המון פסיקות דף, וברגע שנגיע בחזרה לאיבר השני מימין לאיבר הראשון אז כבר הבאנו את כל הבלוקים למטמון (המון פסיקות דף, המון החלפות דפים!) ועבר הרבה זמן ולכן באיבר השני יכול להיות שכבר המטמון "זרק" את הבלוק הזה בחזרה לזיכרון הראשי כי לא היה מקום (Cache Eviction , כלומר יש תחלופה גדולה של תוכן המטמון) ונצטרך להביא את הבלוק הזה שוב וזה מאוד לא יעיל ולוקח המון זמן.



במערכת UNIX נתונה הורצו 2 התכניות הבאות:

```
//Program 1
#define AR_SIZE 2048
main() {
        int  i , j ,a[ AR_SIZE ][ AR_SIZE ];
        for ( i=0; I < AR_SIZE; i++ )
                for ( j=0; j < AR_SIZE;  j++ )
                        a[ i ][ j ]=0;
}
```

```
//Program 2
#define AR_SIZE 2048
main() {
        int  i , j , a[ AR_SIZE ][ AR_SIZE ];
        for ( i=0; I < AR_SIZE; i++ )
                for ( j=0; j < AR_SIZE;  j++)
                        a[ j ][ i ]=0;
}
```

שימו לב, ההבדל היחיד בין התכניות הוא בשורה אחת בלבד(השורה האחרונה). שתי התכניות הורצו במקביל(בו-זמנית) מספר רב של פעמים ובכל פעם נמדד הזמן מתחילת ההרצה עד לסיומה. התברר כי בכל פעם התכנית הראשונה מסתיימת בזמן קצר יותר מאשר התכנית השנייה. ידוע שגודל המערך הדו-ממדי a נכנס לקבוצת העבודה. מה הסיבה **<u>הסבירה ביותר</u>** לכך?

הערה: שימו לב שעקב מספרן הרב של ההרצות ניתן לומר שהתכניות נמצאות בתנאים זהים מבחינת הקצאת משאבים על-ידי מערכת הפעלה.

א. הרצת התכנית השנייה גורמת ליותר פעולות החלפת דפים בזיכרון.

ב. הרצת התכנית השנייה גורמת לתחלופה תדירה יותר של תוכן ה-cache במעבד.

ג. הרצת התכניות במקביל עלולה לגרום לקיפאון ולכן מערכת ההפעלה מעדיפה לסיים את הרצת התכנית הראשונה קודם.

ד. התשובות א ו-ב הן הנכונות.

TLB = מטמון קטן ומהיר במעבד שמיועד לזרז את תהליך תרגום הכתובות מכתובת וירטואלית לכתובת פיזית ב־RAM.
לגשת כל פעם לטבלת העמודים שב-RAM, זה איטי מאוד. ולכן יש TLB : הוא שומר מיפויים אחרונים מכתובות וירטואליות לכתובות פיזיות (בעצם הוא מכיל חלק מטבלת הדפים). מוציאים את ה-tag מהכתובת הווירטואלית, וניגשים ל-tag ב-TLB:

- אם הם שווים – (TLB Hit) התרגום מתבצע ישירות מה־TLB, מהר מאוד (ומבלי לגשת לטבלת הדפים שב-RAM).
- אם הם לא שווים – (TLB Miss) צריך לגשת לטבלת הדפים ב־RAM ואז לעדכן את ה־TLB

מטמון L1 = Level 1 Cache = זהו המטמון הכי קטן והכי מהיר שנמצא בתוך המעבד.

← **<u>סיכום – הוצאת נתונים מהזיכרון (מהמהיר לאיטי):</u>**
מהזיכרון הווירטואלי (VM) יש לנו כתובת ווירטואלית (VA) שעלינו לתרגם לכתובת פיזית:

1. **<u>התרגום נמצא במטמון</u>** : נלך ל-TLB ← ניקח כתובת פיזית (PA) ← נוציא נתונים:
   ← נלך למטמון L1 ואם הנתונים שם מצוין.
   ← אם הנתונים לא שם, ניגש לזיכרון הראשי (RAM) ונוציא נתונים משם.
2. **<u>התרגום נמצא ב-RAM</u>** : נלך לטבלת הדפים ← ניקח כתובת פיזית (PA) ← נוציא נתונים:
   ← נלך למטמון L1 ואם הנתונים שם מצוין.
   ← אם הנתונים לא שם, ניגש לזיכרון הראשי (RAM) ונוציא נתונים משם.
3. **<u>התרגום נמצא בדיסק</u>** : מתרחשת פסיקת דף ונתונים נשלפים מהדיסק.

<u>שיעור 7+8</u> – פינוי (החלפת) דפים
← **<u>פינוי דפים</u>**
נתון דף מסוים שקיים בדיסק ושהתחלנו לעבוד אתו. על פי עיקרון הלוקאליות בזמן ובמקום, יתרחש "פעפוע" של הדף מהדיסק ל-RAM. אך מה נעשה אם ה-RAM מלא לגמרי ואין מקום לדף? נצטרך לפנות דפים משם, כלומר להסתמך על כך שע"פ עיקרון ההכלה כל דף שנמצא ב-RAM נמצא גם בדיסק, ולכן נוכל למחוק אחד מהדפים ב-RAM. איך נמחק דף מבלי שנאבד שינויים שנעשו בו? יש 2 שיטות:

1. write back – כתיבה נדחית
   שינוי נעשה רק בזיכרון הראשי. הדיסק יתעדכן רק כשמוציאים את הדף מהזיכרון הראשי (בפינוי דף). כלומר ששני העותקים של הדף יכולים להיות שונים. איך נדע אם דף שונה או לא? ע"י ביט D (Dirty) (נקרא גם ביט M (Modified)), שמראה אם הדף בזיכרון השתנה מאז שהועלה מהדיסק (או מאז הפעם האחרונה שהוא נכתב לדיסק).
   D = 0 ← הדף לא שונה ב-RAM (למשל עבר קריאה בלבד) ← הדף נמחק מהזיכרון הראשי.
   D = 1 ← הדף שונה ב-RAM (עבר כתיבה) ← קודם מעדכנים אותו בדיסק ואז מוחקים מהזיכרון הראשי.
   לאחר שהדף פונה מהזיכרון הראשי, נוכל לשים במקומו את הדף החדש שרצינו לשמור שם.
2. Write through – כתיבה ישירה
   כל שינוי שנעשה בזיכרון הראשי נכתב מיד גם לדיסק, לכן תמיד יש עותק עדכני של הדף גם בדיסק וגם בזיכרון, ולכן אם נרצה לפנות דף מהזיכרון הראשי פשוט נמחק אותו.

Write back היא הגישה המועדפת, מכיוון שבשיטת write through נעשות המון כתיבות לדיסק שדורשות המון משאבים, וזה פוגע בביצועים.

Demand Paging = העמסת דפים לפי דרישה = טכניקה בניהול זיכרון: במקום לטעון את כל התוכנית מהדיסק ל-RAM מראש (מה שגוזל הרבה זיכרון), **המערכת טוענת ל-RAM רק את הדפים שהתוכנית באמת ניגשת אליהם**. כשמתבצעת גישה לדף שאינו ב־RAM , מתרחשת Page Fault, מערכת ההפעלה עוצרת את התהליך, מעלה את הדף החסר מהדיסק ל-RAM, מעדכנת את הטבלאות וממשיכה את הביצוע.

← **<u>אלגוריתמים לפינוי דפים</u>** – נזכור שהדפים מנוהלים ב-RAM ברשימה מקושרת, עם ראש וזנב. אם אין מקום ב-RAM לדף שנרצה להעביר אליו מהדיסק, נצטרך במהלך פסיקת הדף להחליט איזה דף מפנים מה-RAM על מנת שיתפנה מקום לדף החדש.

1. **<u>תור – FIFO</u>** – head הדף הכי ישן, tail הדף הכי חדש (הרשימה המקושרת מסודרת על פי **זמני הכנסה**).
   הדף הראשון שנכנס הוא הראשון שיוצא. **הדף הכי ישן (head) יפונה ראשון**. מבחינת הרשימה המקושרת, כל מסגרת חדשה נוספת הופכת ל-tail , והמסגרת הראשונה שנכנסה היא ה-head. לכן כשנרצה לפנות דף נפנה את head.
   <u>יתרון</u> – פשוט ליישום ולניהול הרשימה
   <u>חיסרון</u> – יש התייחסות רק לזמני כניסת הדפים, ולא לתדירות השימוש בהם. יכול להימחק דף ישן אך מאוד שימושי, ואז בסיכוי גבוה נרצה לגשת אליו שוב אך הוא כבר לא יהיה קיים ויתרחש page fault.

2. least recently used – **LRU**

head הדף הכי פחות בשימוש לאחרונה, tail הדף האחרון שהשתמשו בו (הרשימה המקושרת מסודרת על פי **זמני שימוש**).
הדף שהיה הכי פחות השימוש לאחרונה יפונה ראשון. לכן, כשניגשים למסגרת מסוימת היא הפכה להיות לשימושית ולכן הוא מוצאת מהרשימה המקושרת ועוברת ל-tail, שם נמצאות המסגרות המעודכנות ביותר.

יתרון - מתחשב בהרגלי שימוש, עובד טוב בהרבה מקרים.

החיסרון –

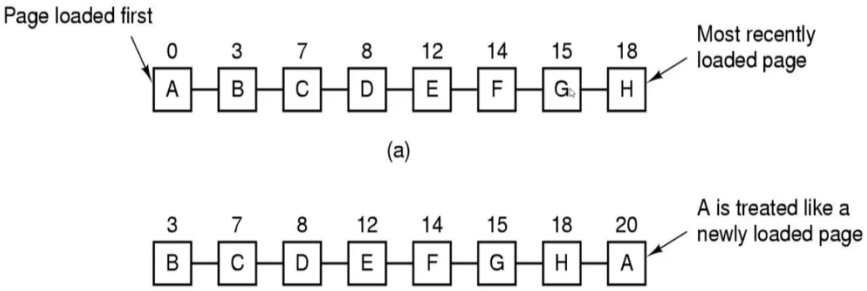
א. אין אופטימיזציה ושימוש טוב בזמנים. בכל פעם יש לחפש ברשימה המקושרת מאיפה להוציא את הדף ולהעביר אותו לזנב. הבדיקה הזו היא בתוכנה ולא בחומרה. בדיקות חומרתיות הן מהירות, אך בדיקות בתוכנה (בקוד) יוצרות קוד כבד כי הוא דורש לוגיקה והן איטיות.

ב. ניהול הרשימה המקושרת נמצא במטמון. מכיוון שהיא כל הזמן משתנה בגלל ההעברות רבות של דפים לזנב, אז גם המטמון משתנה המון – וזה לא טוב ויגרום Cache miss (כי כל פעם שמעדכנים את הרשימה, בלוקים חדשים נדחפים פנימה, ומפנים החוצה בלוקים אחרים שהיו אולי חשובים לקוד או לנתונים. ואז כשנחזור לאותם נתונים "אמיתיים" של התהליך, הם כבר לא יהיו במטמון).

3. **Second chance** – הזדמנות שנייה, אלגוריתם השעון

יש 2 סוגי מימוש:

- רשימה מקושרת **לינארית** - לכל מסגרת נצמיד ביט R (Reference bit) , שהוא ביט שימוש - שיגיד לנו האם המסגרת הייתה בשימוש לאחרונה או לא.
  R=1 ← המסגרת הייתה לאחרונה בשימוש
  R=0 ← המסגרת לא הייתה לאחרונה בשימוש

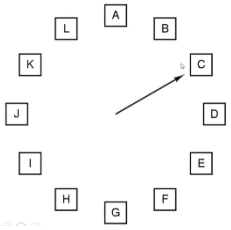


head = הדף הכי ישן (המועמד הראשון לפינוי).
tail = הדף הכי חדש (דפים שזכו ל-"Second Chance" עוברים לשם).
כשמביאים מסגרת חדשה או כשניגשים למסגרת קיימת, ביט R מתעדכן ל-1.
כשהזיכרון מלא וצריך לפנות דף כדי להכניס דף אחר חדש, נעבור על הרשימה ונחפש דף עם R=0 , שיפונה מהזיכרון. כל דף שנבדק והיה R=1 , לא מפונה אבל מתעדכן לR=0 ועובר לזנב– והדף מקבל "הזדמנות שניה" להישאר בזיכרון, אבל פעם הבאה אם הדף ייבדק הוא יפונה.
במקרה הגרוע, כל הדפים הם בעלי R=1 וכולם התעדכנו ל-0. לא מצאנו דף לפינוי, ונעבור על כל המסגרות פעם נוספת וכנראה ניתקל בR=0 במעבר השני והם יפונו. למה כנראה? כי יכול להיות שבין המעבר הראשון לשני יתרחש context switch וכל הדפים יחזרו להיות R=1 וחוזר חלילה וניכנס ללופ אינסופי. הסיכוי שדבר כזה יקרה הוא קלוש, כי:

א. גם אם יש context switch רוב הסיכוי שלא כל הדפים יקבלו שוב שימוש.

ב. במרחב המשתמש לא יכולה להתרחש החלפת הקשר. לכן, במהלך פסיקת הדף (שמביאה את המסגרת החדשה ומחפשת איזו מסגרת לדרוס בזיכרון) לא יכול להיות ש-R ישתנה.

- רשימה מקושרת **מעגלית** – הדפים מוחזקים ברשימה מקושרת לפי הוותק שלהם. כאשר צריך למצוא דף לפינוי, האלגוריתם בוחן את ביט R של הדף שעליו מצביע המחוג (הדף בוותיק ביותר).
  R=0 ← מפנים את הדף
  R=1 ← שמים 0 והמחוג מתקדם לעבר הדף הפחות וותיק במעגל.
  לא יתכן שהמחוג יסתובב לנצח – מקסימום סיבוב אחד שלם אם כל הדפים עם R=1.

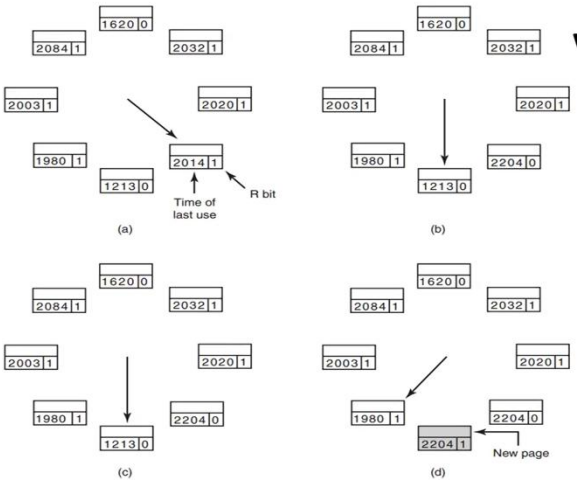


הרשימה המקושרת מעגלית עדיפה.

4. **אלגוריתם WSClock** –

שדרוג לאלגוריתם השעון עם ההזדמנות השנייה, עם תוספת של קבוצת עבודה (WS = working set . קבוצת הדפים מהזיכרון הווירטואלי שהתהליך ניגש אליהם במהלך פרק זמן נתון שיסומן בטאו τ , זה בעצם סף הזמן שמגדיר מה נחשב "שימוש לאחרונה" וכל הדפים שזמן השימוש שלהם קטן ממנו יכללו בקבוצת העבודה). המערכת מנהלת את הדפים בזיכרון ברשימה מעגלית. לכל דף יהיה:

- ביט R
- Time of last use – מתי לאחרונה ניגשנו לאותה מסגרת.



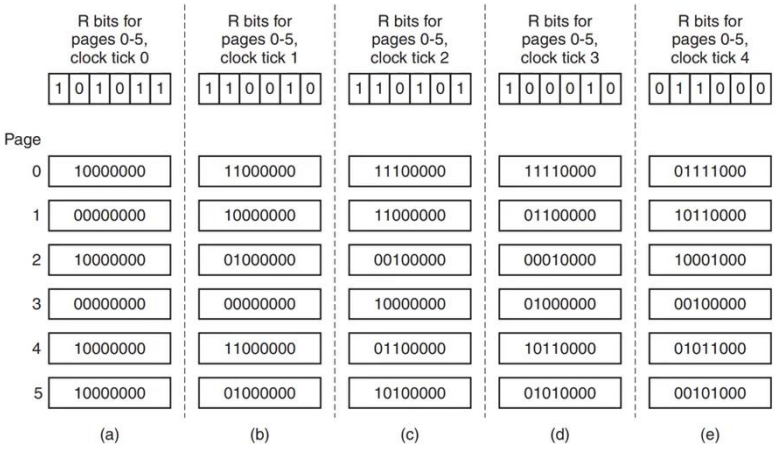
בהתחלה, יש "מחוג" שמצביע על הדף שבו התבצעה הפעולה האחרונה. כאשר יש צורך לפנות דף ולהכניס דף חדש :

- אם ביט **R = 1** ← הדף שומש לאחרונה ← עדכן את זמן השימוש האחרון שלו לזמן הווירטואלי הנוכחי ← אפס את R ל-0 ← קדם את המחוג לדף הבא.
- אם ביט **R = 0** ← זהו מועמד לפינוי (נשווה כעת את הזמנים של קבוצת העבודה, כי אולי יהיה מועמד שלא היה בשימוש יותר זמן מהמועמד הנוכחי):
  - נחשב את גיל הדף:

Age = current virtual time – time of last use

  - אם **age > τ** ← הדף כבר אינו בקבוצת עבודה ← נבדוק את ביט M:
    - M=0 ← הדף לא שונה ← נפנה את הדף
    - M=1 ← הדף שונה ← נתזמן כתיבה של הדף לדיסק (ניצור שם בעצם עותק עדכני) ונקדם את המחוג לדף הבא (אולי יימצא דף שלא צריך לכתוב לדיסק וניתן יהיה לפנותו מיד). תזמן את הכתיבה לכל היותר n פעמים (כדי לא להציף בכתיבות).
  - אחרת ← הדף בקבוצת עבודה ← מבצעים מעקב אחר הדף הוותיק ביותר (ועדיף שיהיה עם R=0)
- לאחר מעבר מעגלי אחד, כאשר עדיין לא פונה דף יתכנו 2 מצבים:
  - אם תוזמנה לפחות כתיבה אחת קודם ← המשך לקדם את המחוג עד אשר יימצא דף עם סיבית שינוי כבויה. הדף שיימצא יפונה.
  - אם לא תוזמנה כתיבה ← כל הדפים הם בקבוצת עבודה ולכן מסמנים את הדף הוותיק ביותר בעל R=0 שעקבנו אחריו. אם אין כזה, מוצאים דף עם R=1 , כותבים אותו לדיסק ומפנים את מסגרתו.

5. **Aging Algorithm** – אלגוריתם ההזדקנות



האלגוריתם הזה הוא קירוב יעיל ל- LRU שנועד להימנע מהעלות הכבדה של שמירת זמן שימוש מדויק. לכל דף יהיה ווקטור ביטים עם מספר ביטים כלשהו שייצג את גיל הדף, שיעודכן לפי איזשהו תדר עדכוני (tick) של המעבד, כלומר כל זמן מסוים, כל טיק. בין כל טיק וטיק יכולות להיות גישות לדף (שיסומנו R=1) או שלא יהיו גישות (נשאר במצב של R=0). איך זה עובד:

- באופן תקופתי (בכל tick) עושים Shift-Right לווקטור הביטים. זה גורם לביט הימני להימחק, ואמור להתווסף ביט משמאל שימלא את המקום שנוצר.
- את הביט השמאלי ביותר (MSB) מכניסים לפי ביט R של הדף:

R=1 ← הדף היה בשימוש לאחרונה (היו גישות בין טיק לטיק) ← מכניסים 1 לביט שמאל

R=0 ← הדף לא היה בשימוש לאחרונה (לא היו גישות בין טיק לטיק) ← מכניסים 0 לביט שמאל

- לאחר מכן מאפסים את ביט R.

בצורה זו, ווקטור הביטים מתעד היסטוריית שימוש אחרונה של כל דף, כאשר ביטים חדשים נכנסים מצד שמאל ודוחפים ימינה את הוותיקים. **כשנרצה לפנות דף, נפנה את הדף עם הגיל הנמוך ביותר.**

← **שאלה** – 2019ב מועד 82 שאלה 3

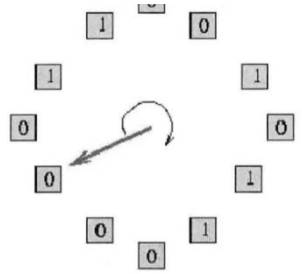
א. בין כל 2 פסיקות עוברים את כל המעגל, וכך הוא מתחיל להזכיר את אלגוריתם FIFO – כי אנחנו בכל פינוי דף נצטרך לעבור על כל המעגל (אין עצירות באמצע, כלומר לא נפנה דף מאמצע המעבר) ובעצם לחזור ל-head והוא יפונה בכל פעם, שזה הדף הישן ביותר.

ב. כשננסה לגשת לדף, יכול להיות שלא תהיה פסיקה (כי הדף יהיה קיים ב-RAM) ויכול להיות שתהיה פסיקה (כי הדף לא יהיה ב-RAM ונצטרך להביא אותו מהדיסק ל-RAM). המדד הוא בעצם מספר הצעדים הממוצע שיידרש בגישה לדף כלשהו. איך נעריך בקירוב מה הוא ה-Q המינימלי והמקסימלי (זו המשמעות בסדרי גודל, גבול עליון O וגבול תחתון Ω)?

- המינימום הוא 1, במקרה שצעדנו צעד אחד קדימה ומיד מצאנו R=0. ולכן Ω(1)
- המקסימום הוא N+1, כי הנחנו שזה user mode, אף אחד לא יכול ליצור החלפת הקשר במהלך החיפוש, ולכן במצב שכל הדפים R=1 נשלים מעגל שבמהלכו ניתן לכל הדפים מהזדמנות שניה ונהפוך אותם ל-R=0 , נעשה צעד נוסף ונפגוש את הדף הראשון שהפכנו ל-R=0 . ולכן O(N)

נגדיר אלגוריתם דפדוף בשם "אלגוריתם השעון". האלגוריתם פועל באופן הבא: כל המסגרות בזיכרון הראשי "מסודרות" במעגל, והאלגוריתם מחזיק "מחוג שעון" שמצביע על מסגרת כלשהי. לכל מסגרת יש ביט גישה (reference bit) המודלק בכל פעם שהחומרה ניגשת לדף שבמסגרת (כמובן, אם המסגרת ריקה, ביט הגישה כבוי). בכל פסיקת דף מתבצע האלגוריתם הבא:

1. הזז את המחוג מסגרת אחת קדימה בכיוון השעון
2. אם ביט הגישה דלוק, כבה אותו וחזור לשלב 1
3. פנה את הדף שבמסגרת (אם יש), טען את הדף הדרוש במקומו והדלק את ביט הגישה.



א. נניח שאלכוריתם השעון מופעל בגישה של demand paging. מהו אלגוריתם הדפדוף אליו "מתנגן" אלגוריתם השעון היה ובין כל שתי פסיקות דף מתבצעת גישה לכל הדפים הממופים לזיכרון? הסבירו.

ב. נניח שמספר המסגרות בזיכרון הוא N, ושמספר פסיקות הדף במהלך סדרת גישות R הוא P(R). יהא L(R) מספר הצעדים שביצע מחוג השעון במהלך הסדרה R. נגדיר את המדד הבא:

$$Q(R) = L(R) / P(R).$$

בתשובותיכם לשאלות הבאות הקפידו להתייחס **גם** למקרים בהם מספר הדפים השונים אליהם ניגשים בסדרה R גדול מ-N. תנו תשובות שהן סדרי גודל (דהיינו, (O(...).

← **שאלה** – 2019א מועד 84

השלם את אלגוריתם WSClock :

כאשר צריכים לפנות דף, עוברים על הדפים שנמצאים בזיכרון ובודקים לכל דף:

אם R=1, אזי עדכן את הזמן הווירטואלי של הדף לזמן הווירטואלי הנוכחי ועדכן את R=0 . לדף הייתה התייחסות (כלומר פנו אליו) לאחרונה, ולכן הוא איננו מועמד לפינוי.

אם R=0 _____, הדף מועמד להיות מפונה.

חשב את age = current virtual time – time of last use

אם age > τ _____, אזי הדף כבר אינו בקבוצת עבודה ולכן הוא מפונה.

אם age <= τ _____, אזי הדף עדיין נמצא בקבוצת עבודה, אבל ייתכן שיפונה אם בסוף המעבר על כל הדפים יימצא שהוא היה ה'יותיק' ביותר מבין כל הדפים בקבוצת העבודה עם סיבית התייחסות "כבויה". (מובן שפינוי דף כזה שהיה בקבוצת עבודה יתבצע רק אם לא התגלה דף מחוץ לקבוצה).

אם עברנו על כל הדפים ולא מצאנו דף עם סיבית התייחסות "כבויה", נבחר באקראי דף עם R=0 _____ 1 ונפנה אותו. במידת האפשר, הדף הנבחר יהיה עם סיבית שינוי "כבויה", כדי לחסוך כתיבה לדיסק.

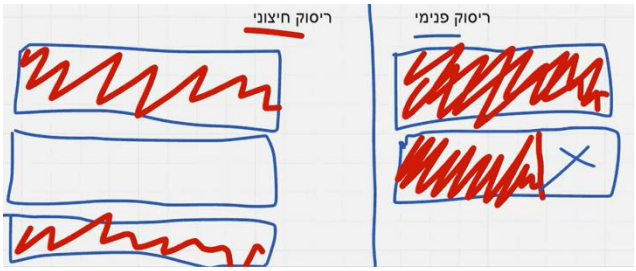
← **גודל דף**

איך מחליטים מהו גודל הדף האידיאלי?

דף קטן מדי –מאפשר לנצל טוב יותר את עקרון הלוקאליות (ניגשים בדיוק למה שצריך), אבל יוצר צורך בהרבה דפים שונים (כי דף הוא קטן ולא מכיל הרבה), ואז נוצרות טבלאות עמודים גדולות ויותר גישות לזיכרון משני, ומכאן הרבה פסיקות דף.

דף גדול מדי –מפחית את מספר הדפים והעמודות בטבלה, אבל מבזבז הרבה מקום בזיכרון (Internal Fragmentation) ומגביל את מספר הדפים שאפשר להחזיק במקביל.

לכן, בוחרים **גודל ביניים**: לא קטן מדי ולא גדול מדי, כך שיש איזון בין ניצול זיכרון, יעילות טבלאות העמודים, ועקרון הלוקאליות.



פרגמנטציה (ריסוק) = מצב שבו הזיכרון אינו מנוצל באופן רציף ויעיל, כלומר יש חלקים פנויים קטנים מפוזרים בזיכרון שלא ניתן להשתמש בהם בקלות. יש שני סוגים עיקריים:

- פרגמנטציה פנימית = זיכרון מבוזבז בתוך הדף ← נוצרת כשיש דף גדול, אבל התהליך לא ממלא אותו עד הסוף כך שיש חלק שלא מנוצל בתוך הדף, וזה בעצם זיכרון מבוזבז. ככל שהדף גדול יותר כך הבזבוז הזה משמעותי יותר.
- פרגמנטציה חיצונית = זיכרון מבוזבז בין הדפים ← נוצרת כאשר יש חללים קטנים ומפוזרים בזיכרון הפיזי, שלא מספיקים להקצאה של יחידה גדולה חדשה, למרות שסך הזיכרון הפנוי מספיק (בשרוצים לשמור משהו תמיד נשמור אותו בזיכרון באופן רציף, ולא "נפזר" אותו ברחבי הזיכרון. כלומר הקצאת זיכרון היא תמיד רציפה). זה תלוי בשימוש החיצוני של המשתמש ובהקצאות הזיכרון שהתרחשו.

לכן, נשתדל שריסוק הפנימי יהיה קטן ככל הניתן (למשל ע"פ סטטיסטיקה להתאים את גודל הדפים). למשל, דפים קטנים יצרו הרבה פסיקות דף אבל יקטינו את הריסוק הפנימי. בריסוק חיצוני הרבה יותר קשה לטפל.

← **שאלה** – 2023א מועד 91 שאלה 2

נתון זיכרון פיזי (RAM) בגודל של 16MB (קטן בשביל פשטות החישובים).

המהנדסים החליטו לבנות את מערכת הזיכרון עם מסגרות (frames) בגדלים שונים.

הדפים/המסגרות יהיו בגדלים הבאים: 1k , 2k , 4k ו- 8k . הרבע הראשון של הזיכרון יחולק לגודל של 1k, הרבע השני יחולק לגודל של 2k, הרבע השלישי יחולק לגודל של 4k והרבע הרביעי יחולק לגודל של 8k.

א. (6 נק')

בהינתן הכתובת הבאה(בייצוג בינארי):

0	1	1	0	0	0	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	0	0	1

מה גודל, מספר המסגרת וה OFFSET שבכתובת הנ"ל?

ב.(5 נק')

נתון כי לכל גודל יש לפחות 10 מסגרות פנויות בזיכרון. המשתמש הפעיל תכנית בגודל של 6.5K, כיצד היית מקצה את הזיכרון לתכנית זו?

ג. (5 נק')

מה הוא היתרון של השיטה הזאת(חלוקת הזיכרון למסגרות בגודל שונה)?

א. נקודה כללית חשובה – גודל המסגרות יהיה תמיד בחזקות של 2, כך המעבד עובד.

- גודל זיכרון פיזי = $2^{24}B = 2^4 \cdot 2^{20}B = 16MB$
- ולכן גודל כתובת פיזית (מסגרת) הוא 24 ביטים.
- את הזיכרון חילקו ל-4 חלקים, ולכן גודל רבע מהזיכרון יהיה: $2^{24} / 4 = 2^{22}B$
- לכן בכל רבע יש 2^{22} כתובות, והזיכרון יראה כך:



בעצם, לכל כתובת בזיכרון יש "אינדקס" לאיזה רבע היא שייכת – ע"פ שני הביטים הגבוהים (השמאליים). לכן הכתובת שלנו נמצאת ברבע השני, כלומר שכל מסגרת שם מהנתונים היא בגודל $2K = 2^{11}$, ולכן גודל ה-offset הוא 11 הביטים הנמוכים (הימניים), כבר אמרנו ש-2 הביטים הגבוהים הם האינדקס לאיזה רבע בזיכרון נמצאים, וזה משאיר אותנו עם 11 ביטים אמצעיים שהם מספר המסגרת.

ב. צריך להיזהר מליצור פרגמנטציה גדולה:

- אם ניקח מסגרת אחת של 8 ← יישאר לנו 1.5 וזו פרגמנטציה פנימית
- אם ניקח 7 מסגרות קטנות של 1 ← יישאר לנו גם 0.5 פרגמנטציה פנימית שהיא יחסית קטנה, וגם 7 פסיקות דף בגלל השימוש במסגרות רבות.
- אם ניקח 2 מסגרות של 4 ← יישאר לנו 1.5 וזו פרגמנטציה פנימית וגם יהיו 2 פסיקות דף, לכן אוטומטית נפסול את האופציה הזו ונעדיף את החתיכה האחת של ה8
- אם ניקח 4 מסגרות של 2 ← יישאר לנו 1.5 וזו פרגמנטציה פנימית וגם יהיו 4 פסיקות דף, לכן אוטומטית נפסול את האופציה הזו ונעדיף את החתיכה האחת של ה8
- אם נשלב גדלים שונים של מסגרות וניקח מסגרות של 4,2,1 ← יישאר לנו 0.5 וזו פרגמנטציה פנימית וגם יהיו 3 פסיקות דף

מכאן, שהאופציות הכי טובות הן של שילוב מסגרות או של מסגרת אחת של 8 – וזה כבר תלוי במה יותר חשוב לנו, ריסוק פנימי או חיצוני מינימלי.

ג. היתרון בשימוש במסגרות שונות הוא היכולת להתאים טוב יותר את הדפים לזיכרון:
מצד אחד, בחירה במסגרת מתאימה בגודל בחזקת 2 מצמצמת פרגמנטציה פנימית,
ומצד שני, שימוש במסגרת גדולה כשאפשר מפחית את מספר הדפים הדרושים ואת כמות פסיקות העמוד.

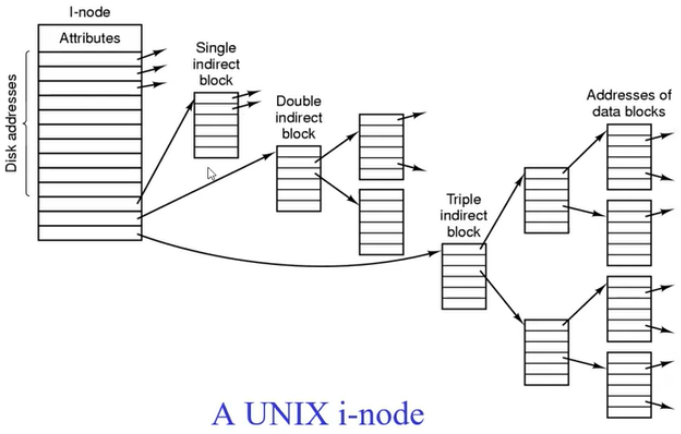
i-node ←

המסגרות נשמרות בדיסק. ישנם 2 סוגי של דיסקים:

א. SSD – הדיסקים המודרניים

ב. HDD - הדיסקים הישנים, הם כבר לא בשימוש.

i-node = במערכת UNIX, זהו מבנה נתונים (Data Structure) ששומר את כל המידע על הקובץ, חוץ מהשם שלו. הוא מתחזק את הבלוקים של הקובץ. הבלוקים (של נתונים או ניהול) בגודל זהה. בדר"כ יש לו **13 כניסות (או מצביעים או כתובות):**



- 10 הכניסות הראשונות = **Direct** Pointers = מצביעים ישירים. מצביעים ישירות לכתובות של בלוקים של מידע (נתונים). מותאמים לקבצים קטנים.
- כניסה 11 = **Single** Indirect Pointer = מצביע עקיף יחיד. בלוק ניהול שיש בו כתובות לבלוקים של מידע. מגדיל את הטווח של גודל הקובץ שניתן לאחסן. (רמה 1 של מצביעים)
- כניסה 12 = **Double** Indirect Pointer = מצביע עקיף כפול. בלוק ניהול שיש בו כתובות לבלוק שהוא Single Indirect. מאפשר קבצים גדולים הרבה יותר. (2 רמות של מצביעים, כלומר המצביע של המצביע לבלוק המידע)
- כניסה 13 = **Triple** Indirect Pointer = מצביע עקיף משולש. בלוק ניהול שיש בו כתובות לבלוק שהוא Double Indirect. מאפשר קבצים ענקיים. (3 רמות של מצביעים, כלומר המצביע של המצביע של המצביע לבלוק המידע)

המבנה הזה מאפשר לנו עבודה באופן מודולרי ב-UNIX.

אם נחשב כמה בלוקים צריך על מנת לנהל קובץ מסוים, לא ניקח בחשבון את מספר ה-i-node כי זה משהו שתמיד נצטרך לקובץ.

כל i-node הוא ממוספר, וזהה למספר הקובץ. גם בהעתקה של קובץ אחד לאחר, הוא גם קובץ עם i-node משלו ויהיה לו מספר אחר.

מספר בלוקים של מידע = מספר רמות המצביעים (גודל בלוק / גודל מצביע)

גודל קובץ בבלוקים = מספר בלוקים של מידע

שאלה ←

נתון דיסק שגודל בלוק הוא 2KB. גודל המצביע הוא 8B.

מה יהיה גודל הקובץ המקסימלי (=מספר בלוקים של מידע, של נתונים):

א. בבלוקים?

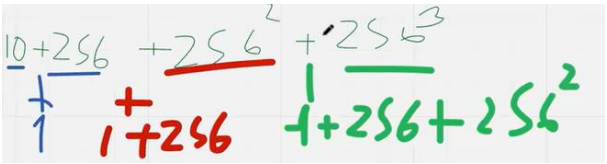
- $2KB = 2^{11}B$, $8B = 2^3B$
- **10** הבלוקים הראשונים הם של מידע, ומתקבלים באופן מידי.
- הבלוק ה-11 מצביע לבלוק ניהול single Indirect שמצביע לבלוקים של מידע. צריך לבדוק כמה כניסות (מצביעים) יהיו לו? נבדוק כמה מצביעים נכנסים בגודל הבלוק: $2^{11}B / 2^3B = 2^8 =$ **256**
- הבלוק ה-12 : אמרנו שכל הבלוקים זהים בגודלם, לכן גם הגודל שלו יהיה 256 מצביעים. כל מצביע כזה מצביע על בלוק ניהול single Indirect שגם הוא מכיל 256 מצביעים שהם יצביעו על בלוקים של מידע. כלומר זה המצביע של המצביע לבלוקים של מידע. לכן מספר הבלוקים של המידע יהיה 256^2
- הבלוק ה-13 : באותו אופן, זה המצביע של המצביע של המצביע לבלוקים של מידע. לכן מספר הבלוקים של המידע יהיה 256^3

וסך הכל מספר הבלוקים של המידע הוא: $10 + 256 + 256^2 + 256^3$

ב. בבתים?

גדלי הבלוקים, הן של נתונים והן של ניהול, הם בגודל זהה – $2^{11}B$, לכן התשובה תהיה מספר הבלוקים של המידע כפול גודל של בלוק יחיד: $2^{11}B \cdot (10 + 256 + 256^2 + 256^3)$

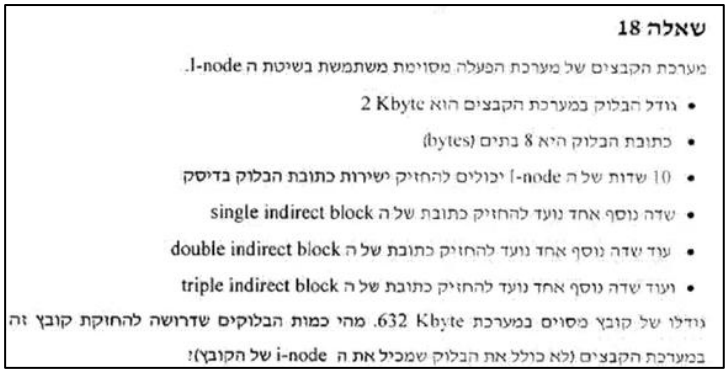
ג. כמה בלוקים סך הכל יש? (מידע+ניהול)



נוסיף לכמות הבלוקים של המידע מסעיף ב גם את מספר הבלוקים של ניהול (בתמונה התוספת הצבעונית בשורה התחתונה):

- 10 הבלוקים הראשונים של המידע הם ישירים, לכן אין כאן בלוקים של ניהול.
- בלוק ניהול single Indirect מצביע על בלוקים של מידע, לכן הוא מבודד אז סה"כ בלוק ניהול **1**
- בלוק ניהול Double Indirect הוא יחיד ומצביע על בלוקים single , אמרנו שלכל בלוק 256 מצביעים לכן הוא יצביע על 256 בלוקים single. לכן **1+256**
- בלוק ניהול Triple Indirect הוא יחיד ומצביע על בלוקים Double שמצביעים על בלוקים single. לכן $1+256+256^2$

← **שאלה**

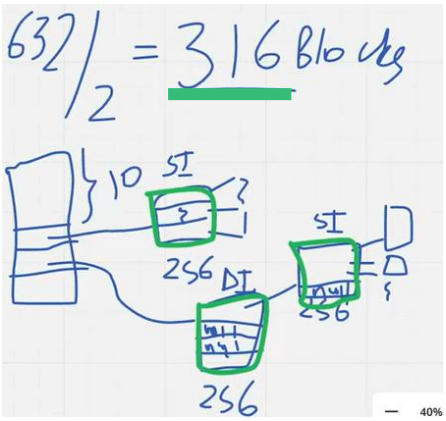


נחשב מספר בלוקים של מידע:

- גודל הקובץ = מספר בלוקים של מידע = 632/2 = **316**

נחשב את מספר הבלוקים של ניהול הדרושים כדי לנהל 316 בלוקים של מידע, וניגש למבנה i-node:

- 10 הבלוקים הראשונים הם של מידע. נחסר ונשאר עם 306 בלוקים של מידע שצריך לנהל.
- בלוק ניהול single Indirect מצביע על בלוקים של מידע, ונצטרך לחשב את כמות המצביעים שיש לו:



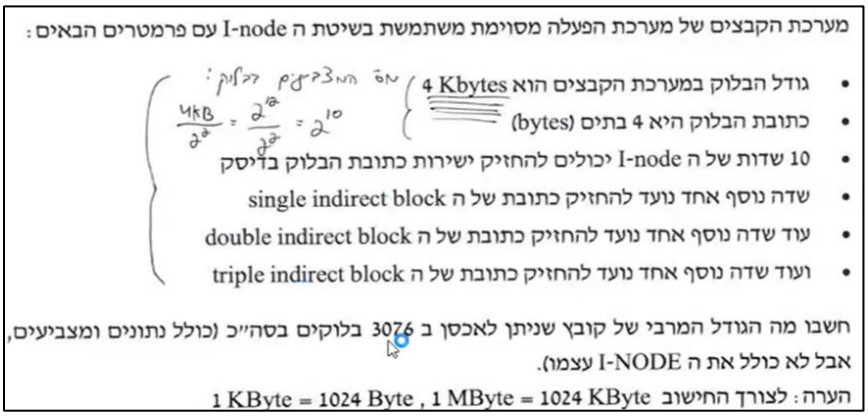
$2KB / 8B = 2^{11}B / 2^{3}B = 2^{8} = 256$. אז יש לבל בלוק 256 מצביעים. נחסר מכמות הבלוקים של מידע ונשאר עם 50 בלוקים של מידע שצריך לנהל. השתמשנו כאן בעוד **1** בלוק ניהול single.

- בלוק ניהול Double Indirect הוא יחיד ומצביע על בלוקים single . נוסיף **1** בלוק ניהול double , ונצטרך עוד **1** בלוק ניהול single שיכול לנהל 256 בלוקים של מידע, אבל ממה שנשאר יצטרך לנהל רק 50.

לכן סך הבלוקים הדרושים להחזקת הקובץ הוא 316+1+1+1=319

שיעור 9 – לינקים, אלגוריתמי תזמון

← **שאלה** – 2025א מועד 92 שאלה 6



- מספר מצביעים בבלוק = 2^{10} = 1K = 4KB / 4B = 1024
- גודל הקובץ הוא 3076 בלוקים – כלומר בלוקים של מידע. נבדוק מתי נגיע אליהם ע"פ מבנה i-node :
- 10 בלוקים ראשונים של מידע
- בלוק ניהול single Indirect יכול להצביע על 1024 בלוקים של מידע , לכן סה"כ בלוקים של מידע עד כה 1034=10+1024
- בלוק ניהול Double Indirect הוא יחיד ומצביע על בלוקים single. אם ניקח בחשבון בלוק ניהול אחד של single שמצביע על 1024 בלוקים של מידע אז סה"כ בלוקים של מידע עד כה 2058=1034+1024 . אם ניקח בחשבון בלוק ניהול single נוסף שמצביע על 1024 בלוקים של מידע אז סה"כ בלוקים של מידע עד כה 3082=2058+1024 וחצינו את הכמות הדרושה.
- לצורך כך השתמשנו ב-4 בלוקים של ניהול סה"כ.
- לכן אם סך הבלוקים שבהם אכסנו את הקובץ הוא 3076 מהשאלה, נוריד את מספר הבלוקים של הניהול ונקבל שיש בסך הכל בלוקים של מידע עבור קובץ זה: 3076-4 = 3072 .
- ולכן גודל הקובץ הוא: **12288KB** = 3072 · 4KB

← **ניהול הi-node**

איך מערכת ההפעלה מנהלת את ה-i-node? לכל אחד יש מזהה עם מספר. כשנוצר קובץ, נוצר אתו i-node חדש עם מספר מזהה זהה למספר הקובץ. ל-i-node של קובץ אפשר גם ליצור לינקים. קיימים 2 סוגים:

1. Hard link = קישור ישיר ל־inode של הקובץ. כמו עוד שם לאותו הקובץ בדיוק. בעצם, גם הקובץ המקורי וגם ה־hard link מצביעים בדיוק לאותו inode. מחיקה של אחד מהם לא מוחקת את המידע כל עוד נשאר Hard Link אחד לפחות. כל שינוי שנבצע יהיה ניתן לראות מכל ה- hard links הקיימים.
   - חייב להיות באותה מערכת קבצים
   - יצירתו אינה דורשת יצירת inode נוסף
   - קיים counter לניהול כל ה- hard links הקיימים לקובץ. מאפשר לדעת כמה גישות (כמה hard link) יש לקובץ האמיתי.

- Soft link = קובץ מיוחד שמכיל נתיב (ב-inode "מפוברק") לקובץ אחר. כלומר הוא מצביע לשם הקובץ, לא ל־inode שלו. זהו קיצור דרך שמצביע למקור.
- לא חייב להיות באותה מערכת קבצים.
- דורש יצירת inode נוסף.
- אם מוחקים את הקובץ המקורי ה־Soft Link נשבר ("Broken link"). לכן, אם משתמש ינסה לגשת לקובץ דרכו הוא יגיע לinode ה"מפוברקים" שנוצרו יחד עם ה-soft link, ולכן למשתמש אין שום אינדוקציה שהקובץ לא קיים כבר, וזו בעיה.

בקריאת קובץ, **גישה דרך hard link תהיה מהירה יותר מ-soft link** מכיוון שהיא ישירה. לעומת זאת, ב-soft link הגישה תיקח קצת יותר זמן מכיוון שהיא עוברת דרך ה-inode החדש שנוצר אתו.

<u>חשוב להבחין בין המקרים השונים:</u>

- בשעושים (cp file1 file2) ← copy, יצירת העתק לקובץ

מערכת ההפעלה יוצרת קובץ חדש לגמרי. נוצר i-node חדש עבור הקובץ המועתק (file2) . התוכן של file1 מועתק פיזית לבלוקים חדשים על הדיסק → הקובץ החדש שייך ל־ i-node אחר. כלומר: אחרי copy שני הקבצים בלתי תלויים – שינוי באחד לא ישפיע על השני .

- בשעושים (ln file1 file2) ← יצירת hard link

לא נוצר תוכן חדש ולא i-node חדש.שני השמות (file1, file2) מצביעים לאותו i-node. המשמעות: זה אותו קובץ פיזי, רק עם שמות שונים. שינוי באחד יופיע גם בשני, כי שניהם מצביעים לאותם בלוקים. ב־ i-node עצמו נספר link count שמייצג כמה שמות מצביעים אליו.

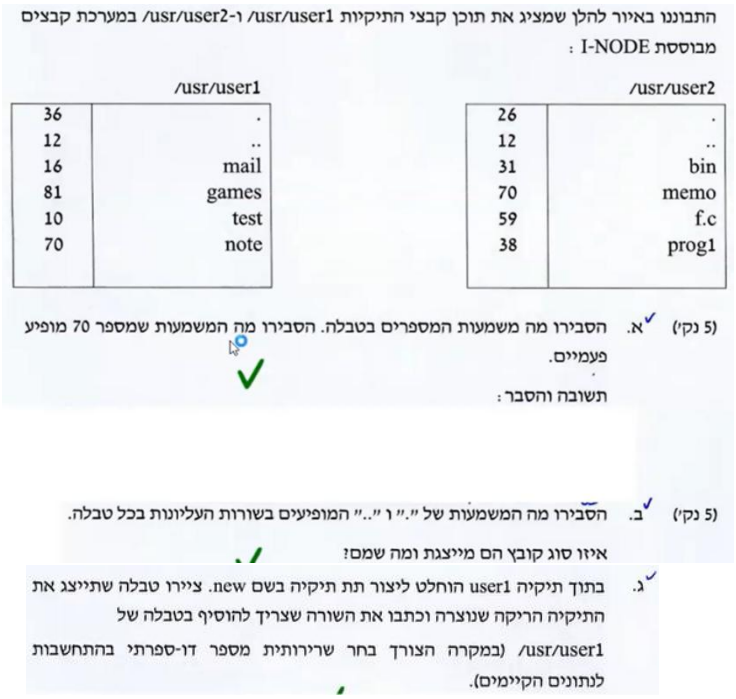
- בשעושים (ln -s file1 file2) ← יצירת soft link

נוצר קובץ חדש עם i-node חדש ,אבל התוכן של הקובץ החדש הוא בעצם נתיב לקובץ הישן. כלומר file2 הוא רק קיצור דרך ל־file1. אם מוחקים את file1 , הקובץ file2 נשבר (כי הוא מצביע לנתיב שכבר לא קיים).

פעולה	פקודה לדוגמה	i-node חדש?	בלוקים חדשים?	קשר בין הקבצים
copy	`cp file1 file2`	כן	כן	עצמאיים לחלוטין
hard link	`ln file1 file2`	לא	לא	אותו קובץ עם שמות שונים (אותו i-node)
soft link	`ln -s file1 file2`	כן	לא (רק טקסט של הנתיב)	קיצור דרך ל־path המקורי, נשבר אם הקובץ נמחק

Directory = סוג מיוחד של קובץ במערכת קבצים שמטרתו היא למפות שמות קבצים ל־ i-nodes שלהם. זה בעצם טבלה שמכילה רשומות בצורה: **שם קובץ → מספר i-node**

← **<u>שאלה</u>** – 2022א מועד 88 שאלה 3


התבוננו באיור להלן שמציג את תוכן קבצי התיקיות /usr/user1 ו-/usr/user2 במערכת קבצים מבוססת I-NODE:

/usr/user2

26	.
12	..
31	bin
70	memo
59	f.c
38	prog1

/usr/user1

36	.
12	..
16	mail
81	games
10	test
70	note

(5 נק') א. הסבירו מה משמעות המספרים בטבלה. הסבירו מה המשמעות שמספר 70 מופיע פעמיים.

תשובה והסבר:

(5 נק') ב. הסבירו מה המשמעות של "." ו "..." המופיעים בשורות העליונות בכל טבלה. איזו סוג קובץ הם מייצגת ומה שמם?

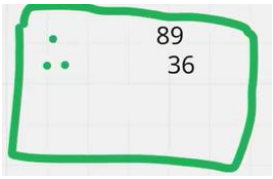
ג. בתוך תיקיה user1 הוחלט ליצור תת תיקיה בשם new. ציירו טבלה שתייצג את התיקיה הריקה שנוצרה וכתבו את השורה שצריך להוסיף בטבלה של /usr/user1 (במקרה הצורך בחר שרירותית מספר דו-ספרתי בהתחשבות לנתונים הקיימים).

א. משמעות המספרים הם inodes המזוהים לכל קובץ או תיקיה. memo ו-note הם hard link של אותו הקובץ מכיוון שמספר ה-inode שלהם זהה, וזו המשמעות של ה-70 שמופיע פעמיים. כל לינק מופיע בתיקיה אחרת, ובעצם מכל אחת מהן יש גישה לאותו הקובץ. שתיהן מצביעות לאותה תיקיה.

ב. משמעות של "." ← מייצג את ה-Directory הנוכחי
משמעות של ".." ← מייצג את ה-Directory שמעל, את תיקיית האב של התיקייה הנוכחית.

לדוג': אם אנו נמצאים ב-home/sapir/doc/ :
"." = home/sapir/doc/
".." = home/sapir/

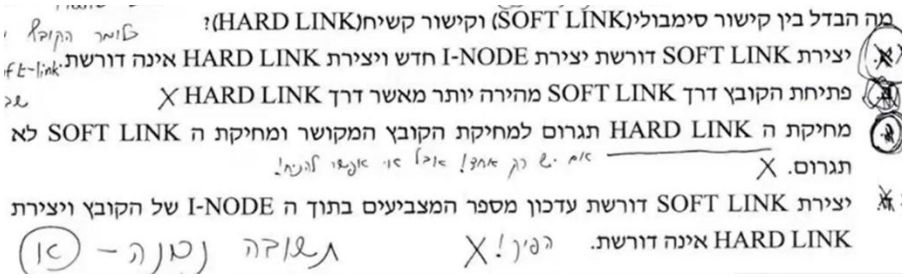
בעצם, שתי התיקיות (user1 , user2) נמצאות תחת אותה תיקיית אב (user) מכיוון של".." יש 12 inodes וכשזה אותו מספר מדובר באותה תיקייה.



ג. דבר ראשון שמים את מספר ה-inode עבור "." ועבור ".."
"." ← התיקייה הנוכחית. נמציא ונכתוב מספר שעוד לא כתבו, למשל 89.
".." ← תיקיית האב שהיא user1 שניתן לראות שיש לה 36 inodes .
מכיוון שהתיקייה ריקה, אין צורך לכתוב עוד משהו.

← **שאלה** – 2025א מועד 92 שאלה 1

התשובה הנכונה היא א'.



← <u>אלגוריתמים לתזמון תהליכים</u>

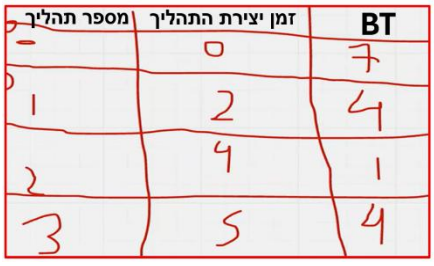
<u>מדדים חשובים בהרצת תהליכים:</u>

$$WT = TAT - BT$$

TAT = Turnaround Time = הזמן <u>הכולל</u> (המתנה+ריצה) שעובר מרגע שהמשתמש הגיש תהליך למערכת ועד שהתהליך סיים לרוץ. נשאף למזער את ה-TAT הממוצע של כלל התהליכים הקיימים.
WT = waiting time = זמן <u>ההמתנה</u> של תהליך לקבלת CPU. כלומר, הזמן שבו התהליך נמצא בתור ההמתנה במתזמן ולא מתבצע בפועל על המעבד. נשאף למזער את ה-WT הממוצע (AWT) של כלל התהליכים הקיימים.
BT = Burst Time = זמן <u>הריצה</u> של התהליך. זמן המעבד שהוקצה לתהליך.

לדוג': נניח תהליך שהגיע בזמן 0, זמן ריצה (BT) שלו 5 שניות, והוא סיים אחרי 8 שניות מהגעתו (TAT = 8) .
אז WT = 8 – 5 = 3 שניות. כלומר, התהליך חיכה 3 שניות בתור לפני שרץ על ה-CPU.

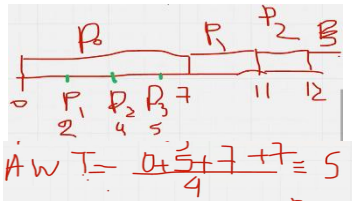
אלגוריתם תזמון Non-Preemptive = תכונה כללית לאלגוריתם תזמון שאינו עוצר תהליך באמצע ריצתו. הוא מתזמן איזה מהתהליכים ירוץ קודם בלבד, מבלי לקטוע אותם באמצע.
אלגוריתם תזמון Preemptive = תכונה כללית לאלגוריתם תזמון שיכול לעצור תהליך באמצע ריצתו. יכול לקטוע תהליך באמצע.



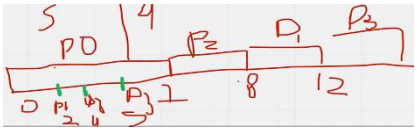
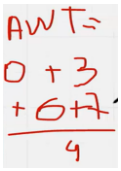
<u>יש 2 סוגי מערכות הפעלה:</u>

1. **<u>Batch System Algorithms</u>** = מערכות בהן <u>אין אינטראקציה</u> עם המשתמש. אין קבלת קלט.
אלגוריתמי התזמון העיקריים הם: (האלגוריתמים מסודרים **מהגרוע לטוב ביותר** סטטיסטית, אבל כמובן שיכולים להיות מקרים יוצאי דופן)

- (First Come, First Served) FCFS - **התהליך שנכנס ראשון מתבצע ראשון**. פשוט וקל למימוש. זהו אלגוריתם Non-Preemptive. חיסרון: שתהליך ארוך "תוקע" מאחוריו את כל התהליכים הקטנים.
לדוג' בציור משמאל, קיימים 4 תהליכים מ0 עד 3. ניתן לראות שתהליך 0 לוקח המון זמן, ותוקע למשל את תהליך 2 שהוא קצרצר אבל חיכה המון זמן כדי לרוץ. וכך נוצר AWT = 5.

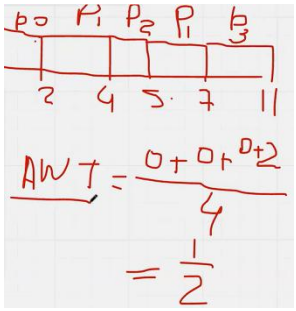


- (Shortest Job First) SJF - **התהליך עם זמן הריצה הקצר ביותר רץ ראשון**. מנצל את המערכת בצורה יעילה ולכן ממוצע זמן המתנה מינימלי. זהו אלגוריתם Non-Preemptive. חסרון: קשה לדעת מראש את זמן הריצה של התהליך.מתבצעת הערכה סטטיסטית מה יהיה זמן הביצוע של התהליך.

- אם בשלב מסוים יש תהליך אחד בלבד, אנחנו פשוט מתזמנים אותו בלי "לעשות חושבים" כמה זמן הוא ירוץ.
- אם שני תהליכים הם בעלי זמן ריצה זהה, האלגוריתם יתעדף את התהליך שהופיע קודם (שחיכה יותר זמן)

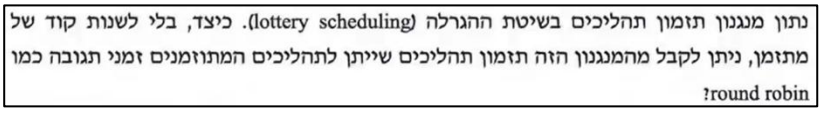
עבור הדוג' הקודמת, p0 קיים בהתחלה לבדו לכן הוא רץ, ועד שהוא מסתיים כל שאר התהליכים מצטרפים ואז הם רצים מהתהליך הקצר ביותר לארוך ביותר. AWT = 4 ושיפרנו מאלגוריתם קודם.



- (Shortest remaining time next) SRTF - הגרסה הקוטעת של האלגוריתם SJF - **התהליך עם זמן הריצה הקצר ביותר שנותר רץ ראשון**. כלומר שזהו אלגוריתם Preemptive . בכל רגע נתון בודקים איזה תהליך ממתין עם זמן הריצה שנותר (Remaining Time) הקצר ביותר. אם מגיע תהליך חדש עם זמן ריצה קצר יותר מהתהליך שרץ כרגע – האלגוריתם קוטע את התהליך הנוכחי ומעביר את ה־ CPU לתהליך החדש.
עבור הדוג' הקודמת, p0 קיים בהתחלה לבדו לכן הוא רץ, ואז p1 צץ וקוטע אותו כי זמן ריצתו קצר יותר, וכן הלאה. בזמן 5 למשל, מופיע p3 עם זמן ריצה 4 אבל לפני כן קטענו את p1 שכבר רץ 2 יחידות זמן ולכן נשארו לו עוד 2 יחידות זמן לרוץ לכן המתזמן יתעדף אותו. AWT = 0.5 ושיפרנו מאלגוריתם קודם.

2. **Interactive / Time-Sharing Scheduling Algorithms** - מערכות בהן יש אינטראקציה עם המשתמש. יש קבלת קלט. אלגוריתמי התזמון העיקריים הם:

- (round robin) RR - אלגוריתם Preemptive . התהליכים מסודרים במעגל, **וכל אחד מהתהליכים מקבל יחידת זמן הנקראת quata, וברגע שהוא רץ זמן quata המתזמן מחליף אותו בתהליך הבא במעגל** שירוץ גם זמן quata ויוחלף וכן הלאה. בעצם בכל סבב תהליך יחכה זמן השווה למספר התהליכים הקיימים כפול quata. **נעדיף quata קטן** כי:
  - יתרון: התהליכים מתחלפים לעיתים קרובות וזה תורם **למקביליות** בעיני המשמש. בנוסף, **זמן תגובה (רספונסיביות)** מהיר יותר.
  - חיסרון: יש המון השקעת זמן בהחלפות הקשר, שזו עבודה כבדה. ולכן מבחינת **TAT סך הזמן העבודה** הוא גדול כי תדירות ההחלפות היא גבוהה. ככל שהקוואנטה מתקצרת ← TAT גדל ← זמן תגובה (רספונסיביות) קטנה
- אלגוריתם Lottery – שיטת ההגרלה לתזמון תהליכים. אלגוריתם סטטיסטי ואקראי. לכל תהליך במערכת נותנים כמות של "כרטיסי לוטו". כל כרטיס מייצג סיכוי לזכות בקבלת ה־CPU. כאשר המעבד פנוי, המתזמן "מגריל" כרטיס, והתהליך שזכה הוא זה שירוץ. ככל שלתהליך יש יותר כרטיסים וכך הסיכוי שלו לרוץ גדול יותר.


נתון מנגנון תזמון תהליכים בשיטת ההגרלה (lottery scheduling). כיצד, בלי לשנות קוד של מתזמן, ניתן לקבל מהמנגנון הזה תזמון תהליכים שייתן לתהליכים המתוזמנים זמני תגובה כמו ?round robin

← **שאלה** – 2017א מועד 83 שאלה 7

תשובה: ניתן להגדיר בין התהליכים סדר מסוים, ואז רק לתהליך אחד לתת את כל הכרטיסים בכל פעם.

שיעור 10 – דיסק, ניהול קבצים בבלוקים

Disk Partitioning = חלוקה של דיסק. פעולה שבה אנחנו מחלקים את הכונן הקשיח (Hard Disk / SSD) ליחידות לוגיות קטנות יותר שנקראות מחיצות (Partitions). כל חלק זה דיסק אחר מבחינת מערכת ההפעלה. כל חלק עובד בדיסק נפרד ולא רואה את השני. יש מערכות קבצים מבוססי i-node אז ניתן לשים על כל חלק של הדיסק מערכת הפעלה אחרת.

במערכות הפעלה, סקטור ובלוק מתייחסות ליחידות אחסון אבל ברמות שונות:

בלוקים = **היחידה הלוגית** שמערכת ההפעלה משתמשת בה על מנת לנהל קבצים.
סקטורים = **היחידה הפיזית** הכי קטנה בדיסק, ברמת החומרה.

- הסקטורים ממוספרים ברצף (מתחילים מ-0).
- הסקטורים מנוהלים בבלוקים שורה-שורה. (לדוג' אם נקצה 2 סקטורים בבלוק עבור קובץ מסוים, 2 הסקטורים הללו יהיו צמודים ויילקחו בצורה אופקית).
- בלוק יכול להיות גדול\שווה מסקטור, למניעת פרגמנטציה (לכן משתדלים לעשות סקטורים יחסית קטנים).
- בלוק יכול להתפרס על סקטור אחד או על כמה סקטורים - לכן בעצם **בלוק הוא "אוסף של סקטורים"**.
- i-node יכול לנהל בלוקים בגדלים שונים כי הסקטורים שבבלוקים מאוחסנים בi-node על פי הכתובות שלהם, במספרים (הכתובות, המספרים ב i-node הם הכתובות של הסקטורים).

← **שאלה** – 2025א מועד א1 שאלה 9

א. (4 נק')

הוצע רעיון הבא לייעל מערכת קבצים: שטח מחיצה PARTITION יחולק ל-2 אזורים בגודל שווה.
באזור הראשון גודל הבלוק יהיה 4Kbyte ובאזור השני 1Kbyte.
מה היתרון של השיטה החדשה מבחינת ניצול שטח האכסון?

ב. (4 נק')

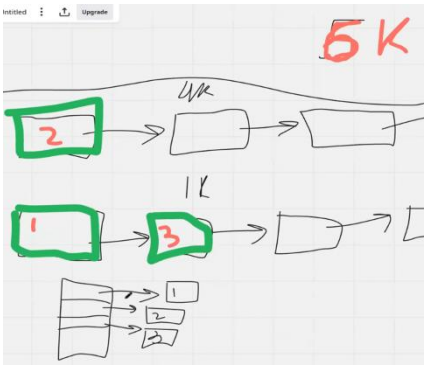
האם החלק הראשון של קובץ מסוים יכול להיות באזור הראשון, החלק השני באזור השני והחלק השלישי שוב באזור הראשון?

ג. (7 נק')

בהנחה שגודל המחיצה 64MByte(1Kbyte=2^10Byte, 1MByte=2^20Byte),
תארו את המהלך היעיל ביותר לתרגום מספר הבלוק למיקום בדיסק (מספר הסקטור).
בשביל פשטות החישוב הניחו שגודל הסקטור 1Kbyte .

א. גמישות וצמצום פרגמנטציה על ידי התאמה מקסימלית בין גודל הקובץ לקומבינציית הבלוקים בה נשתמש לאחסונו.

ב. כן – מצד אחד, **מבחינת תחזוקת הקובץ בi-node** אין בעיה כי כל בלוק וסקטור מיוצג ע"י המספר שלו, הכתובת. הכתובות מאוחסנות בi-node לפי הסדר בו מיקמנו את חלקי הקובץ בסקטורים השונים ומערכת ההפעלה קופצת מכתובת לכתובת להרכבת הקובץ השלם. מצד שני, **מבחינת הפונקציונליות** הקפיצות בסדר הזה (מבלוק אחד לשני ובחזרה לראשון) זה גם בסדר. הבלוקים הם לא משהו קבוע, כי ההקצאות בבלוקים שבדיסק כל הזמן משתנות בגלל שינויים שמתבצעים כל הזמן בקבצים. מערכת ההפעלה מנהלת את ההתייחסות לחלקי הקבצים ע"י רשימה מקושרת או Git . יש בלוקים מוכנים, שממופים לסקטורים, וחלקם תפוסים וחלקם פנויים וזה כל הזמן משתנה. כלומר, בכל שינוי בקבצים נעשים גם שינויים בבלוקים.



נשווה את המקרה שלנו למקרה (בתמונה משמאל) בו אנו רוצים לאחסן קובץ בגודל 6K כאשר במחיצה אחת יש בלוקים בגודל 4K ובאחרת בלוקים בגודל 1K. מערכת ההפעלה מנהלת את הקבצים ב-2 רשימות מקושרות, שכל אחד מייצג מחיצה אחרת. האופן הכי נכון לאחסן את הקובץ מבחינת פרגמנטציה היא בבלוקים של 1,4,1, . המקרה שבשאלה יכול להתרחש למשל בסיטואציה בה מערכת ההפעלה ניגשת לאחסן את הקובץ, ובאותו רגע יש רק בלוק פנוי יחיד ברשימה המקושרת של 1K. לאחר מכן נעשה שינוי בקובץ כלשהו שמפנה בלוק ברשימה של 4K, ובאופן גרידי מערכת ההפעלה "מסתערת" עליו בשביל הקובץ שלנו, ושוב היא מחכה לבלוק שיתפנה ומוצאת 1K נוסף. כך נוצר מצב שמשיקולים של מזעור פרגמנטציה ובלוקים שהיו בנמצא באותו הרגע הקובץ התאחסן בצורה המתוארת בשאלה, ולכן זה אפשרי.

ג. לא נתון גודל הבלוק, לכן במצבים כאלה אנחנו צריכים לסמן אותו בX ולהמשיך לפתור.

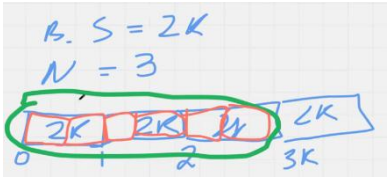
נתון: גודל המחיצה = 2^{26} B = 64MB , גודל הסקטור = 2^{10}B = 1KB

נסמן: BS גודל הבלוק בבתים, N מספר הבלוק

מיקום הבלוק בדיסק הוא בעצם מספר הסקטור ממנו מתחיל הבלוק. מכיוון שהסקטורים ממוספרים לפי סדר ומתחילים מ-0, אז מספר של סקטור הוא גם כמות הסקטורים שעברנו עד כה.

עבור בלוק מספר N , כמות הבתים שעברנו מתחילת הדיסק היא (N·BS) בתים.

נחלק את זה בגודל של סקטור ונקבל (N·BS)B/1KB = **(N·BS)/1024**



<u>דוג' נוספת להמחשה:</u>

המספרים על הציר הם מספרי הבלוקים.

גודל סקטור הוא 1KB , ולמשל כדי למצוא את מיקום בלוק מספר 3 (N=3) לפי הנוסחה שהגענו אליה:

(3·2)KB/1KB = 6 , ואכן בלוק מספר 3 מתחיל לאחר סקטור 6.

mount = פעולה שבה מערכת ההפעלה "מחברת" (**ממפה**) מערכת קבצים שנמצאת על התקן אחסון מסוים (כמו דיסק, USB, CD, מחיצה וכו') אל עץ הספריות הראשי.

בלי mount , מערכת ההפעלה יודעת שהדיסק קיים ,אבל לא יכולה להשתמש בתוכן שלו. mount הוא השלב שמכניס את מערכת הקבצים של הדיסק לעץ הספריות, כאילו הייתה חלק אינטגרלי מהמערכת.



<u>דוג'</u>: נניח שיש לי USB המחובר למחשב ועליו קבצים. הוא נקרא במערכת dev/sdb1/ . זהו שם ההתקן, הוא מייצג את ה-USB אבל עדיין אין דרך להשתמש בו (כמו קלסר סגור שמונח על השולחן. יודעים שהוא שם אבל לא יודעים מה יש בתוכו). אם ננסה להיכנס ל-dev/sdb1/ לא נראה קבצים, אלא רק ייצוג פיזי של ההתקן. שימוש ב-mount במתואר משמאל יאפשר לנו לגשת לתוכן שב-USB (mount בעצם "יפתח את הקלסר כך שנוכל לראות מה הוא מכיל").

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
unshare (PID_NS);
unshare (MOUNT_NS);
int pid = fork();
if (pid == 0) {
  unshare (MOUNT_NS);
  mkdir ("dirS");
  int fd = open ("dirS/file.txt",O_CREATE|O_RDWR);
  write (fd ,"123456789\n",10);
  close (fd);
 }
if (pid > 0) sleep (10000); // wait for child to finish
if( pid == 0) {
  if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "son failed to open\n");
  else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
  if (open ("dirS/file.txt",O_RDONLY) < 0) printf (stderr, "father failed to open\n");
  else printf (stderr, "father succeeded\n");
```

← **שאלה** – 2025א מועד א2 שאלה 12

בהסתכלות על הקוד "מלמעלה" – יש רק fork אחד, לכן סך התהליכים שירוצו הוא $2^1=2$. אז **יש כאן תהליך אב ותהליך בן שירוצו**.

בנוסף, יש 2 unshare לכן ייווצרו שני namespace חדשים. Unshare לפי mount זה אומר **שההפרדה היא לפי איזה מערכת קבצים כל אחד רואה**. **כלומר ברגע שיהיה fork לאחר ה-unshare(MOUNT_NS) וייווצר תהליך בן, הוא יקבל את מערכת הקבצים במצב שהייתה ברגע שהוא נולד, ומעתה והלאה השינויים במערכת הקבצים הם נפרדים לאב ולבן** - שינויים שהבן יבצע בקבצים רק הבן יראה (ולא האב), ושינויים שהאב יבצע בקבצים רק האב יראה (ולא הבן).

אז תהליך האב רץ, נוצרים namespace 2 ואז נוצר תהליך בן בשורה 3. האב ממשיך, ה-pid שלו גדול מ-0 כי הוא אבא ולכן הוא לא נכנס ל-if הראשון ואז נכנס ל-if השני ושם יש לו פקודת sleep(100000) (כלומר התהליך צריך לחכות 100000 שניות שזה המון זמן. שמים sleep ארוך כדי שהתהליך לא יסתיים מהר, וכך אפשר לבדוק מה הבן עושה בינתיים), לכן הוא נעצר וימשיך רק כאשר תהליך הבן יסיים.

נעבור לתהליך הבן שנוצר בשורה 3. Pid=0 ולכן הוא נכנס ל-if הראשון. גם הוא עושה unshare(MOUNT_NS) ואז יוצר תיקייה הנקראת dirS לקריאה וכתיבה, כותב לתוכה מחרוזת וסוגר אותה. **התיקייה הזו נוצרת רק במערכת הקבצים של הבן בגלל unshare(MOUNT_NS) הראשון שהיה.** לאחר מכן הבן מדלג על הif השני ונכנס לשלישי. בתוכו יש if נוסף שמנסה לפתוח את תיקיית .dirs לקריאה בלבד. התיקייה קיימת במערכת הקבצים של הבן, לכן היא נפתחת ומוחזר ה-fd שלה שאינו קטן מ-0 ולכן התנאי לא מתקיים, ותהליך הבן עובר ל-else שם הוא מדפיס "son succeeded" ועובר שורה. ל-if האחרון הבן לא יכנס ובעצם בכך יסתיים.

נחזור לתהליך האב שסיים "לישון". הוא מדלג על הif השלישי ונכנס לרביעי. בתוכו יש if נוסף שמנסה לפתוח את תיקיית .dirs לקריאה בלבד. התיקייה קיימת במערכת הקבצים של הבן בלבד ולא אצל האב, לכן הפעולה לא חוקית ויוחזר 1- והתנאי מתקיים, ותהליך האב מדפיס "father failed to open" ועובר שורה. ואז מסתיים.

לסיכום, יודפס:
son succeeded
father failed to open

Sleep(X) = פונקציה שאומרת לתהליך שהגיע אליה לחכות X שניות ואז להמשיך את פעולתו.
Mkdir("X") = make director. יצירת ספרייה חדשה ששמה X במערכת הקבצים.

Open(const char *pathname, int flags, mode_t mode) – פתיחת קובץ קיים או יצירת קובץ חדש, כדי שהתהליך יוכל לגשת אליו.

- **pathname** - הנתיב לקובץ (בתוך "...")
- **flags** - מציין איך לפתוח את הקובץ (קריאה, כתיבה, יצירה וכו').
- **mode** (רלוונטי רק אם יוצרים קובץ חדש) – הרשאות (כמו 0644, 0777).

הפונקציה מחזירה: **fd** - מספר שלם >0 שחוזר מהקריאה ומייצג את מספר הקובץ, או 1- אם נכשל.

ערכים נפוצים של flags

- `O_RDONLY` → קריאה בלבד.
- `O_WRONLY` → כתיבה בלבד.
- `O_RDWR` → קריאה וכתיבה.
- `O_CREAT` → אם הקובץ לא קיים – צור אותו.
- `O_TRUNC` → אם הקובץ קיים – מחק את התוכן.
- `O_APPEND` → כל כתיבה תתווסף לסוף הקובץ.

write(int fd, const void *buf, size_t count) – כתיבה לקובץ פתוח.

- **fd** - מספר קובץ פתוח (file descriptor) כמו שקיבלת מ־()open
- **buf** - buffer. מה לכתוב בקובץ. נשים מחרוזת או מצביע לאזור בזיכרון שבו נמצא המידע שצריך לכתוב.
- **count** - כמה בתים לכתוב מה־buffer (כל תו זה בית אחד). אם שמנו מחרוזת אז כמה בתים לכתוב מתחילת המחרוזת.

הפונקציה מחזירה:

- את מספר הבתים שנכתבו בפועל (יכול להיות פחות מ־ count אם יש בעיה)
- 1- אם קרתה שגיאה.

Close(X) – סגירת של קובץ הנקרא X.

← **שאלה** – 2025א מועד א1 שאלה 1

מהי שיטת התזמון SCHEDULING המתאימה ביותר בשביל להשיג את תגובתיות RESPONSIVENESS (תגובה ראשונית מהירה) באפליקציות אינטראקטיביות?

א. shortest job first
ב. תזמון ע"פ עדיפויות קבועות
ג. Round robin עם quantum קצר
ד. Round robin עם quantum ארוך

התשובה היא ג.
א – לא נכון. הוא לא אינטראקטיבי
ד – עדיף קוואנטה קצרה כי אז זה אומר שזמן ההמתנה קצר יותר ואז הרספונסיביות מהירה יותר.
ב – לא נכון. עדיפות קבועה לא מצמצמת רספונסיביות. כי אם תהליך ארוך קיבל עדיפות גבוהה זה מונע מהתהליכים האחרים להתחיל לרוץ והם מאוד מתעכבים.

נזכיר – קבצים מנוהלים ב:

- **רשימות מקושרות של בלוקים** - דורש הרבה מקום אחסון, תיחזוק רב, צריך לעבור על כל הרשימה בחיפוש אחר בלוק פנוי.
- **ביטים** - לכל בלוק בדיסק יש ביט אחד: 1 פנוי, 0 תפוס. קל לתיחזוק, לא תופס הרבה מקום אחסון, הרבה זמן חיפוש אחר בלוק פנוי
- **בדיקת עקביות** - לפעמים מתרחשות שגיאות (נפילת חשמל, קריסת מערכת) וזה יוצר מצב שבו טבלאות ניהול הקבצים לא תואמות למה שיש בפועל על הדיסק.לכן מערכת הקבצים מריצה תוכנות בדיקה שבודקות את העקביות:
  - האם כל בלוק שמסומן כתפוס באמת שייך לקובץ?
  - האם יש בלוקים שהוקצו לשני קבצים בו־זמנית?
  - האם גודל הקובץ תואם למספר הבלוקים שהוקצו לו?

  אם יש בעיה – המערכת מנסה לתקן (לשחרר בלוקים, לתקן אינדקסים וכו')

← **שאלה** – 2025א מועד א2 שאלה 9

סעיף א. (6 נק')
טבלה להלן מייצגת את תוצאות של בדיקת עקביות Consistency של מערכת קבצים(קצרה בשביל קלות מעקב).

מספר בלוק	10	9	8	7	6	5	4	3	2	1	0
בלוק בשימוש	0	0	0	0	1	1	0	2	1	0	1
בלוק פנוי	1	1	1	0	0	0	2	0	1	1	0

ציינו את כל הבעיות שהתגלו, לגבי כל בעיה הביאו תסריט ליצירתה ודרך לטפל בבעיה(לתקן במידת האפשר).

א. בשאלה זו, בדקנו עבור כל בלוק 2 דברים: האם בשימוש והאם פנוי. הבלוקים מנוהלים בביטים. הבעיות שמצאנו הן:
- בלוק 2: לא יכול להיות שהוא בשימוש אבל פנוי. לכן יש ללכת ל-i-node ולבדוק האם הבלוק קיים שם. צריך לבטל אחד מהמצבים האלו.
  תיקון מהיר: שימוש יותר חזק לוגית, כי ניתן לומר שאם קרתה תקלה והוא גם בשימוש וגם פנוי אז נשאיר אותו להיות בשימוש (1) ונהפוך אותו לתפוס (0).
  תיקון מעמיק: ללכת לinode של הבלוק. אם קיים, אז נשאיר אותו להיות בשימוש ותפוס. ואם לא קיים, אז נעשה שהוא לא בשימוש ופנוי.
- בלוק 3,4: לא יכול להיות ביט ששווה 2. לכן נחזיר אותו להיות 1
- בלוק 7: לא יכול להיות שהוא לא בשימוש אבל תפוס. אבל כנראה שאם הוא לא בשימוש אז הבלוק פנוי לכן נשים 1 בפנוי (כנראה שהייתה תקלה שי ירידת מתח והתרחשה התקלה הזו).

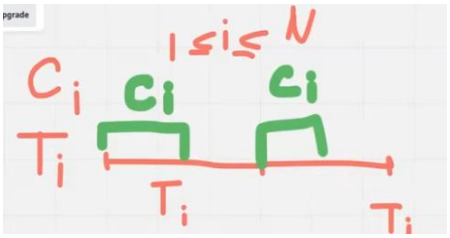
שיעור 11 – מערכות זמן אמת Hypervisor , real time, תקשורת בין תהליכים

← **שאלה** – 2024א מועד 72 שאלה 3

זמן הריצה של התוכנית החדשה הוא 2 ms. מהי שיטת התזמון שתוביל לזמן התגובה הנמוך ביותר של התכנית?
הניחו שבמערכת רצות כבר 3 תכניות אחרות והן תרוצנה עוד זמן רב. התכניות חדשות מוכנסות לראש התור ורצות מיד.

א. round robin עם quantum של 1 ms
ב. round robin עם quantum של 10 ms
ג. shortest remaining time first
ד. זמן התגובה יהיה זהה בכל המקרים כיוון שהתכנית החדשה תהיה בראש התור ותרוץ ראשונה.

תשובה – ג.
ב-round robin יש רשימה מקושרת מעגלית, ולכן ראש הרשימה הוא המקום הבא אחרי התהליך שכרגע רץ. אז אם תהליך נכנס לראש זה אומר שהוא נכנס מיד אחרי התהליך שכרגע מתבצע, ולכן הוא יהיה התהליך הבא שיתוזמן לרוץ. בשאלה זו אם נשתמש באלגוריתם זה (סעיף א או ב) אז נחכה במקרה הגרוע קוואנטה של זמן, ויכול להיות שגם נחכה פחות – תלוי כמה זמן כבר רץ התהליך הנוכחי, וברגע שהוא יסתיים התהליך שהכנסנו יתחיל לרוץ. אז מבין שני הסעיפים נבחר כרגע ב-א' כי הקוואנטה בו קטנה יותר אז במקרה הגרוע נחכה הכי פחות.
בסעיף ג', האלגוריתם SRTF נותן לתהליך שנותר לו את זמן הריצה המינימלי לרוץ קודם. ידוע לנו של-3 התהליכים הקיימים נותר עוד זמן רב לרוץ, ולתהליך שאנחנו מכניסים זמן הריצה הוא 2ms בלבד. ולכן ברגע שהתהליך שלנו יכנס האלגוריתם יתזמן אותו מיידית כי הוא הקצר ביותר ולכן נחכה 0ms – ומכאן שהתשובה היא ג'. (כמובן שיכול להיות ב-א' או ב' שגם נחכה 0ms כי "בפוקס" התהליך הנוכחי שרץ סיים את קוואנטת הזמן שלו, אבל במדעי המחשב תמיד חושבים על המקרה הגרוע ביותר ומכריעים לפיו).



מערכות זמן אמת (Real Time)

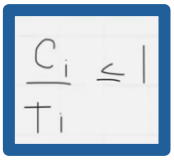
דיברנו על מערכות הפעלה בהן אין/יש אינטראקציה עם המשתמש. כעת נדבר על סוג נוסף של מערכות הפעלה, מערכות זמן אמת. במערכות אלו, התהליכים מתוזמנים באופן מחזורי (דומה ל-RR). עבור תהליך i מבין N תהליכים קיימים (כלומר $1 \le i \le N$) :

Ci = זמן ה-CPU של תהליך i. כמה זמן בפועל התהליך רץ במקרה הגרוע. כמה זמן התהליך יכול לרוץ ברצף על ה-CPU במקרה הגרוע ביותר.

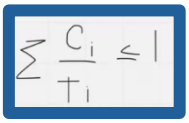
Ti = זמן מחזור תהליך i. כל כמה זמן התהליך מופעל מחדש. הוא יכול להיות הומוגני או לא. המחזוריות עבור מכל תהליך יכולה להיות שונה, כלומר לכל תהליך יש Ti משלו (בניגוד לאלגוריתם RR בו לכל התהליכים יש קוואנטה זהה של זמן).

תנאים לתזמון תהליכים מחזוריים במערכות זמן אמת

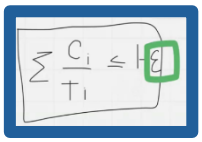
Ci>Ti ← לא אפשרי.
Ci<Ti ← אפשרי.
Ci=Ti ← אפשרי.



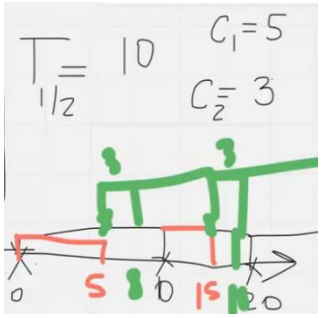
Ci \ Ti = כמה זמן מתוך מחזור של התהליך הוא הזמן בפועל שהתהליך שלנו רץ. הזמן הלא מבוזבז בכל מחזור ריצה.
היחס הזה אמור להיות קטן/שווה ל-1 (כי הרי תהליך לא יכול לרוץ יותר מ-100% מזמן המחזור שלו...).
אם היחס קטן מ-1 זה אומר שישנן יחידות זמן לא מנוצלות,
ואם היחס שווה 1 זה אומר שכל הזמן מנוצל במלואו.



אם מסתכלים על מספר תהליכים, נעשה סכימה של כל ה- Ci \ Ti של התהליכים כי כולם ביחד אמורים לעבוד יחד באינטראקציה שלא תעבור את 100% הזמן שיש להם יחד לביצוע הריצה שלהם.

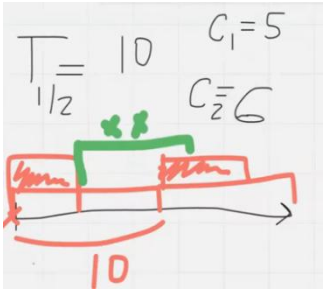


מעבר ליחידות הזמן הלא מנוצלות שראינו, יש במערכות זמן אמת גם זמן מבוזבז מעצם ההחלפה בין תהליכים, מהחלפת הקשר (בדומה לאלגוריתם RR). **בנוסחאות C\T לא לוקחים בחשבון את הזמן המבוזבז של החלפת ההקשר, ולכן תיאורטית התוצאה שקיבלנו מהנוסחאות צריך להיות מעט גדולה יותר** (התוספת הזו מסומנת באפסילון). בעצם הנוסחאות הקודמות מתבססות על כך **שזמן החלפת התהליכים context switch הוא זניח**.



דוגמא א' :
עבור 2 תהליכים אם T זהה שווה ל-10, ו-C שונה להלן.
תהליך 1 (באדום) יתוזמן במחזורים 0,10,20... ומכיוון שה-C שלו שווה 5 זמני הריצה שלו יהיו 0-5 , 10-15, 20-25 וכו'. כדי שהתהליכים יצליחו להיות ב"דו קיום", נגדיר תדירות זמן של תהליך 2 (בירוק) ב-5,15,25 ... כלומר מיד בסיום ריצת תהליך 1 בכל מחזור. ה-C שלו שווה 3 ולכן זמני הריצה שלו יהיו 5-8, 15-18, 25-28 וכו'. כך שני התהליכים מצליחים לרוץ ב"דו קיום" ולא "עולים" זה על זה. אך נשים לב שיש גם זמני ריצה לא מנוצלים – 8-10 , 18-20... 2 יחידות לא מנוצלות בכל מחזור.
ניתן גם לראות שהתהליכים יכולים לרוץ יחד על פי חישוב היחס C\T שיוצא קטן מ-1.

דוגמא ב' :
תהליך 1 חופף בחלק מזמן הריצה שלו לזמן הריצה של תהליך 2 וזה לא אפשרי. CPU בכל רגע נתון יכול לבצע תהליך אחד בלבד.
ניתן גם לראות שהתהליכים לא יכולים לרוץ יחד על פי חישוב היחס C\T שיוצא גדול מ-1.



← **שאלה** – 2023ב מועד 61 שאלה 9

התנאי לכך שיהיה ניתן לתזמן n משימות מחזוריות במערכת זמן אמת (real-time) הוא: $\sum_i \frac{C_i}{T_i} \le 1$ כאשר Ti הוא זמן המחזור (period) ו Ci הוא זמן ה CPU של המשימה ה-i-ית.
על איזו הנחה סמויה הנוסחה מתבססת?

א. זמן החלפת המשימות (context switch) הוא זניח.
ב. לכל i בין 1 ל n מתקיים Ti= Ci .
ג. ניתן לתזמן רק את המשימות המשלבות פעולות חישוביות עם פעולות קלט-פלט.
ד. לכל i בין 1 ל n מתקיים Ti < Ci.

תשובה – א.

כפי שלמדנו, יש התעלמות בנוסחה מהאפסילון המייצג את הזמן שהתבזבז על החלפת ההקשר בין התהליכים.

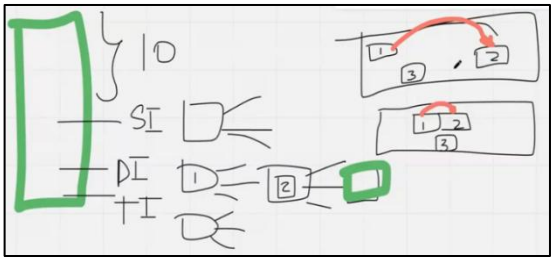
← **שאלה** – 2024א מועד 72 שאלה 6

בחרו את הפעולה היקרה ביותר במערכת קבצים מבוססת I-Node במונחים של מעברי בלוקים של הדיסק (disk block transfers) בהנחה שלא קיימים נתונים רלוונטיים בזיכרון המטמון (buffer cache) לפני הפעולות:

א. סדרת עשר קריאת של תו אחד באמצעות fgetc מקובץ קטן מאוד בגודל 10 Byte
ב. קריאת בלוק אחד באמצעות read מתחילתו של קובץ גדול מאוד בגודל 1 GigaByte
ג. קריאת בלוק מספר שלושים באמצעות read מתוך קובץ גדול מאוד בגודל 1 GigaByte ששמור ברצף בגישה אקראית(ישירה)
ד. קריאת בלוק מספר שלושים באמצעות read מתוך קובץ גדול מאוד בגודל 1 GigaByte ששמור **לא!** ברצף בגישה אקראית(ישירה)

תשובה –

- יש פה הנחה שאין מטמון ש"מזרז" את הוצאת המידע.
- הזמן כאן מבוזבז על מעבר מבלוק לבלוק, כלומר הגישה בדיסק אל הבלוק המתאים דורשת השקעת זמן.
- כדי שלא יהיה מצב של פרגמנטציה הבלוקים של i-node מפוזרים **בגישה אקראית (ישירה)**, כלומר הם לא מסודרים בהכרח קרוב זה לזה וגם זה מבזבז זמן. אם הם היו **בגישה רציפה** (הבלוקים קרובים זה לזה פיזית) עדיין מספר הגישות לזיכרון היה זהה, אבל השקעת הזמן בגישה תהיה קטנה יותר.



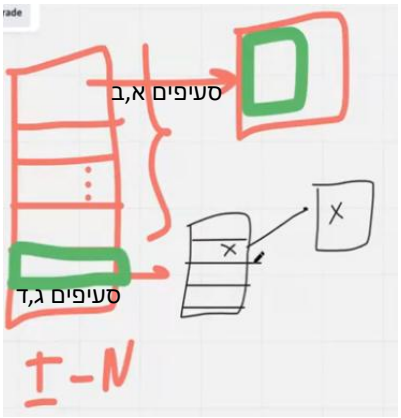
להלן דוגמא לגישה בזיכרון לבלוק ממסוים בדיסק.
הדיסק העליון הבלוקים מסודרים בגישה אקראית,
הדיסק התחתון הבלוקים מסודרים בגישה רציפה.
לכן מספר הגישות זהה (3) אך השקעת הזמן גדולה יותר בגישה האקראית כי הבלוקים לא בהכרח קרובים זה לזה.

fgetc = פונקציה בשפת C שקוראת תו אחד (= 1 בית = 8 ביטים) מקובץ.

read = פונקציה בה מציינים בסוף החתימה שלה כמה בתים אנחנו הולכים לקרוא מהקובץ.

ובחזרה לשאלה שלנו:

הפעולה היקרה ביותר = שדורשת את מספר הגישות הגדול ביותר לדיסק.



א – הקובץ קטן מאוד (10B), ולכן הוא ימוקם באחד מ-10 הבלוקים הראשונים ב-i-node שהם עם אחסון נמוך אך גישה ישירה. אז כל הקובץ יכנס לבלוק אחד וע"י הפונקציה fgetc עושים 10 קריאות ומביאים את הקובץ אל ה-RAM. סה"כ זה דורש מאיתנו גישה אחת לדיסק המכילה 10 קריאות קטנות כי הקובץ קטן.

ב – לא כתבו את גודל הבלוק, לכן הקריאה מתבצעת ע"י Read שבה כותבים את הגודל שהולכים לקרוא, אז גודל הבלוק לא רלוונטי לנו כי הפונקציה תקרא את כולו. הקובץ גדול מאוד (1GB) ונמצא בבלוק אחד שממוקם בתחילת הדיסק, ולכן הגישה אליו ישירה. אז יש כאן גישה אחת לדיסק שהיא כבדה מאוד כי הקובץ גדול.

ג – מספר הבלוק הוא 30 לכן בוודאות הוא לא ב-10 הבלוקים הישירים, ואנו נאלצים ללכת לבלוקים ה"פחות סימפטיים" מבחינת גישה. ב-i-node הוא ימוקם ב-SI מכיוון שיש בו הרבה כניסות ותכולה גדולה של סדר גודל יותר מ-100 , ולכן נצטרך לגשת ל-SI, לקרוא ממנו כתובת ומשם לגשת לבלוק. סה"כ 2 גישות לדיסק, אבל הקובץ שמור ברצף ולכן לאחר שניגשנו לכתובת הראשונה אז הכתובת השנייה היא במרחק יחסית קטן ולכן הגישות עצמן ידרשו מעט זמן.

ד – כאן לעומת זאת זה כמו ב-ג , יש 2 גישות אך הגישה הקובץ שמור לא ברצף ולכן יכול להיות שהמרחק אל הכתובת השנייה יכול להיות הרבה יותר גדול, ולכן הגישות עצמן ידרשו הרבה זמן.

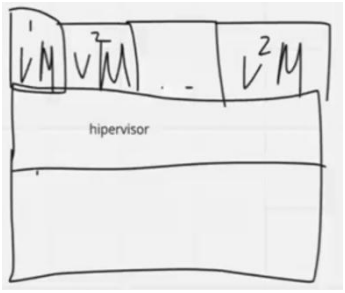
נסכם: סעיפים א,ב דרשו גישה אחת לדיסק. סעיפים ג,ד דרשו 2 גישות בדיסק, אבל בסעיף ד' בגלל שהגישה לא רציפה הגישות עצמן דרשו יותר זמן ולכן **התשובה היא ד'**.

← היפרוויזר (Hypervisor)

היפרוויזר הוא תוכנה שמאפשרת להריץ **בצורה מבודדת** כמה מערכות הפעלה (שרצות ב-VM) במקביל על אותו מחשב פיזי. היפרוויזר הוא **רכיב תוכנה שמאפשר וירטואליזציה** ע"י ניהול והרצת מספר מערכות הפעלה במקביל על אותה חומרה. ישנם 2 סוגים:

1. Bare Matal – Type 1 = רץ ישירות על החומרה, בלי מערכת הפעלה באמצע, יעיל וביצועי יותר.

במחשב פיזי חזק קיימת חומרה. בדרך כלל, מעל החומרה מותקנת מערכת הפעלה כללית ,שתפקידה לנהל את כל משאבי המערכת. עם זאת, ניהול זה כולל מעברי הקשר והחלפות רבות ,הגורמות לעלויות זמן גבוהות. במערכות גדולות וחזקות, ובעיקר במערכות הדורשות זמני תגובה קצרים ,ניתן לוותר על מערכת הפעלה כללית ובבדה, ולהריץ את המערכת ישירות על החומרה .במקרים אלו נדרש רק ניהול בסיסי של משאבים כגון תזמון וגישה לזיכרון, ללא כל המנגנונים המורכבים של מערכת הפעלה מלאה.
עם זאת, עדיין עולה הצורך לחלק את החומרה בין מספר משתמשים ,שכל אחד מהם ניגש למערכת באמצעות ממשק משלו, ומריץ סביבת עבודה נפרדת. לשם כך משתמשים ב־מכונות וירטואליות (VMs) כאשר לכל מכונה וירטואלית יש מערכת הפעלה וקבצים משלה.
כדי לנהל את החלוקה והבידוד בין המכונות הווירטואליות, משתמשים בהיפרוויזר מסוג 1.
ההיפרוויזר:



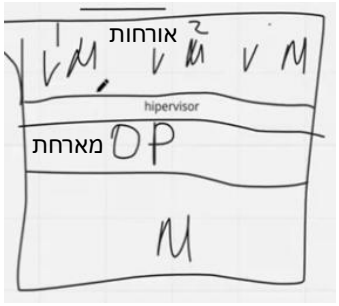
- רץ ישירות על החומרה (ללא מערכת הפעלה מארחת)
- מתקשר עם ה־ CPU בשפת המכונה.
- קוד של מערכת ההפעלה שרצה על ההיפרוויזר לא צריך לעבור שינוי או התאמה כדי שתוכל לרוץ
- מנהל ישירות את הזיכרון והמשאבים
- מבצע תזמון והפרדה בין המכונות הווירטואליות

כך מתקבלת מערכת שבה (כמו בציור) :

- חומרה "מברזלים" M נמצאת בתחתית
- מעליה רץ ההיפרוויזר
- ומעליו פועלות מספר מכונות וירטואליות VMs, כל אחת עם מערכת הפעלה משלה

כל זאת ללא שכבת מערכת הפעלה כללית באמצע, ובביצועים גבוהים יותר. אך את ההיפרוויזר מסוג 1 לוקח זמן להתקין כי הוא מורכב יותר.

2. Hosted – Type 2 = רץ מעל מערכת הפעלה קיימת, כמו תוכנה רגילה.



במחשב פיזי פחות חזק, קיימת חומרה פיזית שעליה מותקנת מערכת הפעלה מארחת (Host OS). מערכת הפעלה זו אחראית לניהול החומרה, והיא משמשת בסיס להפעלת מכונות וירטואליות. מעל מערכת ההפעלה המארחת פועלת שכבת ההיפרוויזר מסוג 2, שהיא שכבה תוכנתית ופשוטה יותר ביחס להיפרוויזר מסוג 1. ההיפרוויזר מתקשר עם מערכת ההפעלה המארחת, והיא זו שמבצעת בפועל את הגישה לחומרה . מעל ההיפרוויזר פועלות המכונות הווירטואליות, כאשר לכל מכונה וירטואלית יש מערכת הפעלה וקבצים משלה. ההיפרוויזר מסייע בתיאום ובניהול בין המכונות הווירטואליות לבין מערכת ההפעלה המארחת. בגישה זו קיימות יותר שכבות ביצוע, ולכן כל פעולה דורשת מעברי הקשר נוספים, מה שמוביל לביצועים נמוכים יותר ולזמני תגובה ארוכים יותר בהשוואה להיפרוויזר מסוג 1. עם זאת, יתרונה של שיטה זו הוא בכך שהיא קלה יותר להתקנה ומודולרית יותר.

מה נכון לגבי HYPERVISOR מסוג 2 בתצורת HOSTED ?

א. הקצאת כל משאבי החומרה hardware resources נשלטת **ע"י מערכת ההפעלה האורחת**.
ב. מערכת הפעלה אורחת GUEST רצה במצב משתמש.
ג. בתוך המכונה הוירטואלית ניתן להתקין רק את מערכת הפעלה האורחת GUEST שתואמת את המערכת המארחת HOST .
ד. לשתי מערכות- אורחת ומארחת חייב להיות אותו גרעין.

← **שאלה** – 2025א מועד א1 שאלה 2
ד – הגרעין שונה.
ג – מערכות ההפעלה יכולות להיות שונות. בזכות ההיפרוויזר מתאפשר התיווך ביניהם למרות השוני.
א – מערכות ההפעלה של ה-OP (המארחת) ושל כל ה-VMs (האורחות) מתווכות ע"י ההיפרוויזר, שעוטף את הקריאות שיורדות מהאורחת אל החומרה, ומתורגמות שם אל המארחת. לכן מערכת ההפעלה המארחת היא האחראית על ההקצאות.
ב – נכון.

מה נכון לגבי HYPERVISOR מסוג 1 בתצורת bare metal ?

א. כל משאבי החומרה hardware resources נשלטים **באופן ישיר** ע"י hypervisor ולא ע"י מערכת הפעלה.

ב. רק הקצאות זיכרון ראשי ופעולות קלט-פלט עוברות דרך hypervisor , אבל הקצאת זמן המעבד CPU הפיזי מבוצעת **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ג. רק הקצאות זיכרון ראשי עוברות דרך hypervisor , אבל פעולות קלט-פלט מבוצעות **ישירות** ע"י ה OS האורחת **בלי שיתוף** של hypervisor .

ד. קוד של כל מערכת הפעלה שרצה על hypervisor צריך לעבור שינוי והתאמה כדי שתוכל לרוץ.

← **שאלה** – 2025א מועד ב שאלה 2

ב,ג - מערכות ההפעלה האורחות לא יודעות לעשות כלום באופן ישיר. הן חייבות מתווך אל התוכנה שהוא ההיפרוויזר מסוג 1. לכן הסעיפים האלה לא נכונים.

ד – פעם זה היה ככה, היום זה כבר לא נכון.

א – נכון.

תזכורת: דיברנו על דרכים לתקשורת בין תהליכים:

← הנעזרים במערכת ההפעלה (העברת הודעות massage passing):

יתרונות – מערכת ההפעלה מבטיחה לנו שהפעולות יתבצעו בצורה נכונה.

חסרונות – החלפת ההקשר למערכת ההפעלה הוא בעלות של זמן.

1. סיגנלים – מנגנון תקשורת אסינכרוני, שבו תהליך אחד שולח התראה לתהליך אחר. מה שמועבר הוא מספר עבור הפעולה שיש לעשות. שליחת סיגנל מתבצעת באמצעות קריאת מערכת, ומערכת ההפעלה אחראית להעברת הסיגנל וטיפולו.

בהשתתפות מערכות ההפעלה:

2. צינורות (pipes) - מנגנון תקשורת שמעביר נתונים ממשיים בין תהליכים, בצורה של זרם בתים. הוא נראה כמו צינור כאשר יש לו 2 קצוות:
   - קצה כתיבה – נקרא לפעמים S (Send / Write)
   - קצה קריאה – נקרא לפעמים R (Receive / Read)

יש תהליך שכותב לקצה S, ותהליך אחר שמקבל מקצה R. **ההודעות שסיגנל שולח מבילות מידע יותר גדול (מחרוזות) ממה שסיגנל מסוגל להעביר (מספר של פקודה).** בנוסף, המימוש הפנימי של צינור הוא קובץ. הקובץ מתואר ע"י (file descriptor) fd שהוא מספר שלם שמתאים למקום מסוים במערכת ההפעלה, שבעזרתו התהליך אומר למערכת ההפעלה: "אני רוצה לעבוד עם הקובץ הזה". כשפותחים את הקובץ, אנו יכולים לציין את היעד שלו – לקריאה, כתיבה או תוך כדי יצירה. כשפותחים צינור, אז בצד S הקובץ פתוח בהרשאה של כתיבה. בצד R הקובץ פתוח לקריאה.

← לא נעזרים במערכת ההפעלה:

יתרונות – חוסך מעבר דרך מערכת ההפעלה, ובכך חוסך זמן.

חסרונות – עלולה להתרחש בעיית סנכרון (Race Condition) בין תהליכון קורא לכותב שמגיעים יחד לאזור הזיכרון. בנוסף, יצירת אזור הזיכרון היא בעלות של זמן ובעלות של מקום זיכרון.

Shared memory זיכרון משותף – הגדרת אזור זיכרון משותף. נמצא בתהליכונים, הנמצאים תחת תהליך אחד שמנהל את כולם. התהליכון הראשי הוא המייצג של התהליך. אז יהיה תהליכון אחד שקורא מאזור הזיכרון, ותהליכון אחר שכותב אליו.

שיטת העברת הודעות היא מהירה יותר משיטת האזור המשותף, בגלל עניין המעבר דרך מערכת ההפעלה.

שאלה 3

רוצים לאפשר העברת מידע בין תהליכונים שונים (לא שייכים לאותו תהליך אב). מתלבטים בין שימוש בזיכרון משותף (shared memory) לבין העברת הודעות (massage passing) באמצעות מערכת ההפעלה.

מה ניתן לומר על השיקולים לבחירה בין שתי האפשרויות הנ"ל?

א. אם מדובר בהעברת מידע יחידה(שמתבצעת פעם אחת), לא כדאי ליצור אזור זיכרון משותף להעברת הנתונים.

ב. כאשר ישנן מספר רב של העברות מידע, שימוש בהעברת הודעות ע"י מערכת ההפעלה מהיר יותר מהעברת אותה כמות נתונים ע"י זיכרון משותף.

ג. שתי אפשרויות העברת המידע פועלות פחות או יותר באותה המהירות – כיוון ששתיהן מערבות את מערכת ההפעלה ונדרשת החלפת הקשר עבור כל העברת מידע.

ד. העברת מידע באמצעות מערכת ההפעלה מתאימה רק להעברת מידע בין תהליכונים משותפים לאותו תהליך אב, לא בין תהליכים שונים.

← **שאלה** – 2025א מועד א2 שאלה 3

א – נכון. אכן עבור הודעה אחת בלבד יצירה של אזור זיכרון משותף יכול להיות מיותר מבחינת בזבוז הזמן ומקום הזיכרון שהוא דורש.

ב – לא נכון. זיכרון משותף יהיה מהיר ויעיל פה יותר.

ג – לא נכון. יש החלפת הקשר רק בהעברת הודעות.

ד – לא נכון. מערכת ההפעלה היא בסיגנלים וצינורות, ולכן עדיף להשתמש דווקא בה כאשר לתהליכים אין תהליך אב משותף, כלומר כשאין להם אזור זיכרון משותף. איך? פשוט לציין PID של כל תהליך.

שיעור 12 – תרגול כללי

← **שאלה** – 2025א מועד ב שאלה 7

דמיינו בית קפה עם 9 מקומות ישיבה.

- אם תגיעו כשיש מקום ריק, תוכלו לשבת מיד.
- אבל אם תגיעו כשכל 9 המושבים מלאים, זה אומר שכולם סועדים ביחד במסיבה, ותצטרכו לחכות עד **שכל הסועדים** יעזבו לפני שיתיישבו.

ניתנו המשתנים הבאים וקוד סועדים בבית קפה כפתרון אפשרי.

```
int  eating = 0 , waiting = 0  // מספר אנשים יושבים או מחכים בבר
semaphore  mutex = 1 // סמפור בינארי המאותחל ל 1
semaphore  block = 0  // סמפור מונה המאותחל ל 0
bool  must_wait = false
// eater process תהליך של אוכל
 1  down(mutex)
 2  if (must_wait) {
 3       waiting += 1
 4       up(mutex)
 5       block.wait ()
 6       down(mutex) // re-acquire mutex תפוס מחדש את
 7        waiting -= 1
 8       }
 9  eating += 1
10  if (eating == 9) must_wait = true
11  else must_wait = false
12  up(mutex)
13  // eat sushi אוכל סושי
14  down(mutex)
15  eating -= 1
16  if (eating == 0) {
17       n = min (9 , waiting )
18       up(block , n)
19       must_wait = false
20       }
21  mutex.signal()
```

9 מקומות ישיבה = 9 תהליכונים.

צריך לוודא שהקוד מתאר היטב את המתואר, תוך התחשבות במצבי:

- מניעה הדדית - עיקרון שלפיו רק תהליך אחד בכל רגע רשאי לגשת ל־משאב משותף או ל־קטע קריטי.
- Deadlock - מצב שבו שני תהליכים או יותר תקועים לנצח, כי כל אחד מהם מחזיק משאב ומחכה למשאב אחר שמוחזק ע"י תהליך אחר.
- הרעבה Starvation - מצב שבו תהליך לא מקבל משאבים או זמן מעבד לאורך זמן, למרות שהמערכת כן ממשיכה לפעול.

תזכורת: פעולות סנכרון אטומיות לשליטה בגישה לקטע קריטי:

down(mutex) = בקשת כניסה לקטע הקריטי

מה קורה:

- אם ה־ mutex פנוי ← התהליך נכנס
- אם ה־ mutex תפוס ← התהליך נחסם וממתין

up(mutex) = שחרור הקטע הקריטי

מה קורה:

- ה־ mutex משתחרר
- אם יש תהליך שממתין ← אחד מהם מתעורר ונכנס

כל עוד יש מקום במסעדה, מדלגים על if הראשון, מעלים את eating ב-1 , ואם הסועד שעבשיו נכנס הוא התשיעי נכנסים ל-If בשורה 10 ומעבירים את המשתנה הבוליאני לאמת כי צריך מעבשיו לחכות מחוץ למסעדה.

מה המשמעות של שורה 12? אולי כשעשינו את כל הבדיקות בשורות הקודמות בינתיים עוד אנשים נכנסו למסעדה, ואנחנו רוצים "לשחרר" אותם מלחכות שנסיים את כל הבדיקות. ורק אז נכנסים לאכול בשורה 13. כך אוגרים ואוגרים סועדים.

נניח והגענו ל-9 סועדים – must wait הפך לאמת והסועד העשירי נכנס ל-If בשורה 2 שם מעלים את מספר הממתינים. אבל בשורה 4 יש up(mutex) כדי:

א. לשחרר את כל הסועדים שחיכו שנסיים את הבדיקות.

ב. כי בשורה 14 עושים down(mutex) . אם לא היינו עושים את שורה 4 אז מי שהיה מסיים את שורה 14 כי סיים לאכול לא היה יכול להשתחרר ממנה, ואז היינו נכנסים לדדלוק.

נמשיך עם מי שסיים לאכול בשורה 13: מספר הסועדים קטן ב-1 , ואם הוא שווה 0 אז נכנסים ל- If בשורה 16 ונבדוק כמה מחכים להיכנס. מותר להכניס רק 9 אבל מספר ה- waiting יכול להיות גדול יותר – אז ב-17 מכניסים ל-n את המספר המינימלי מבינהם, ואת המספר הזה אנחנו משחררים מתוך הסמפור המונה בשורה 18. מי ששוחרר חיכה בשורה 5, ובושרה 6 מצפה להם קוד בו הם עושים down(mutex) ומקטינים את מספר המחכים ב-1 כי הם כבר לא מחכים.

אחרי ששחררנו את המחכים, בשורה 19 מעבירים לשקר את ה- must-wait כדי לצאת ממצב אגירת סועדים ובכך לאפשר לסועדים להיכנס. **ופה בשורה 19 יש בעיה !!!! כי אם min=9 , כלומר היו מעל 9 מחכים בחוץ, אז נתנו לכל ה-9 להיכנס למסעדה ושוב אין מקום במסעדה. אבל לא התחשבנו במצב הזה ועשינו must_wait=false ולכן יצליחו להיכנס:**

- **סועדים ישנים שכבר המתינו בתוך ה- If הראשון ויצליחו להיכנס משורה 6**
- **סועדים חדשים משורה 1**

כאן נכנס המתזמן. יכול להתרחש מצב בו המתזמן נותן דווקא לסועד חדש להיכנס קודם, והוא יספיק לעשות down(mutex) ולהגיע לשורה 2. המתזמן ברגע זה יעשה שוב החלפת הקשר וייתן גם לסועד ישן להיכנס, והוא גם יספיק לעשות down(mutex) ולצאת מהתור. המתזמן יעשה עוד החלפת הקשר בחזרה לחדש, שיספיק להיכנס למסעדה ולאכול בשורה 13, ועל הדרך בשורה 12 לבצע up(mutex) ולשחרר את כל הממתינים משורה 6, ואז נכנסים עוד ועוד סועדים (למרות שכבר יש שם 9 סועדים ממזמן...) ואז יקרה משהו גרוע יותר – היו במסעדה כבר 9 , ויספיקו להיכנס עוד ועוד סועדים אקסטרה – אחד מבחוץ ועוד 8 מרשימת ההמתנה. הסועד אקסטרה ה-9

מרשימת ההמתנה מגיע לשורה 10 ונכנס ל-if ומונע מחדשים להיכנס, אך בשורה 12 משחרר את התשיעי מרשימת ההמתנה שמשחרר סועדים חדשים, ואז יכנסו עוד ועוד סועדים... ולכן **אין מניעה הדדית**.
איפה יש כאן **הרעבה**? מצב בו יש יותר מ-9 שמחכים, והמתזמן מכניס רק חדשים ולא נותן לתהליכים מרשימת ההמתנה להיכנס.

ובקיצור...

<u>מניעה הדדית</u>
ברמת הסנכרון הבסיסית, הקוד שומר על מניעה הדדית :כל הגישה למשתנים המשותפים (eating, waiting, must_wait) מתבצעת תחת mutex, ולכן אין מצב שבו שני תהליכים מבצעים במקביל קטע קריטי.
עם זאת, קיימת בעיה לוגית :בשורה 19 מתבצע must_wait = false ללא בדיקה האם שוחררו בדיוק פחות מ־9 תהליכים. במקרה שבו n = 9 ויש יותר מ־9 ממתינים, הדגל מאפשר כניסה של תהליכים נוספים, הן ממתינים ישנים והן תהליכים חדשים, וכתוצאה מכך עלולים להימצא יותר מ־9 סועדים במסעדה בו־זמנית. כלומר, אין הפרת mutual exclusion ברמת ה־mutex, אך יש הפרת מגבלת הקיבולת הלוגית.

<u>Deadlock</u>
הקוד נמנע מ־ Deadlock בזכות השחרור של ה־ mutex לפני חסימה על block.wait() (שורה 4). אילו ה־ mutex לא היה משתחרר, תהליך שהיה מסיים לאכול לא היה מצליח להיכנס לקטע הקריטי בשורה 14, וכל התהליכים היו נתקעים בהמתנה הדדית. לכן, מניעת ה־ Deadlock בקוד תלויה במפורש בשחרור ה־ mutex לפני החסימה.

<u>הרעבה</u>
הקוד אינו מבטיח הוגנות (fairness) מאחר שאין הבטחת FIFO על ה־semaphore block, והמתזמן יכול להעדיף תהליכים חדשים, ייתכן מצב שבו תהליכים שממתינים זמן רב לא ישוחררו, בעוד שתהליכים חדשים מצליחים להיכנס למסעדה. לכן, כאשר מספר הממתינים גדול מ־9, עלולה להתרחש הרעבה של תהליכים ממתינים.

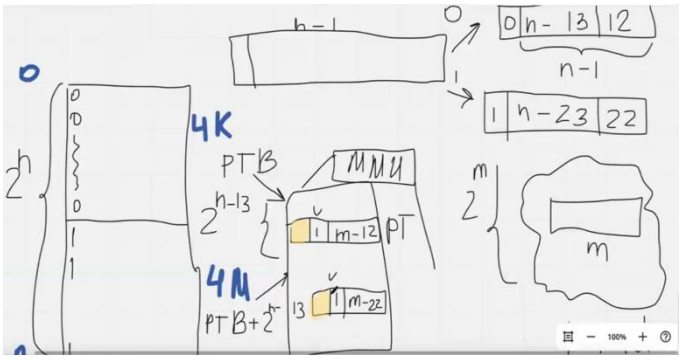
<u>ביצד נתקן את הקוד?</u>
נחליף את שורה 19 ב:
must_wait = (n == 9);
התיקון מבטיח ש־ must_wait משקף תמיד את מצב הקיבולת בפועל, ומונע כניסה של תהליכים חדשים כאשר המסעדה מלאה, תוך שמירה על מניעת Deadlock וצמצום הרעבה.

← **<u>שאלה</u>** – 2023ב מועד 82 שאלה 2


נתונה מערכת מחשב שתומכת בדפים בגדלים שונים: 4 Kbyte ו 4 Mbyte בלבד. כל דף בכל גודל יכול להימצא בכל מקום בזיכרון הפיזי RAM. תהליך שמבקש הקצאת זיכרון ראשי 8 Mbyte או יותר מקבל אותו בדפים בגודל 4 Mbyte ובהקצאות קטנות יותר בדפים בגודל 4 Kbyte. המרחב הווירטואלי מחולק לשני אזורים שווים, בחלק הנמוך גודל הדף 4 Kbyte ובחלק הגבוה 4 Mbyte **במרחב הזיכרון הפיזי אין חלוקה כזאת.**
הקומפיילר המיוחד שפותח בשביל המערכת ופונקציות הספרייה של שפת תכנות קובעים כתובות וירטואליות של משתנים בהתאם(בכתובות גבוהות או נמוכות, לא משנה מה הדרך לעשות את זה).

א. (9 נק')

תארו את האלגוריתם(את הדרך) של MMU לתרגום הכתובת הוירטואלית לכתובת פיזית. ציינו האם נדרשים שינויים בצורת הכתובת הוירטואלית ובטבלת הדפים(בהשוואה לצורה רגילה) כדי לאפשר את תרגום הכתובת ואם כן, מה הם. הדגישו במיוחד את שיטת חישוב מספר מסגרת זיכרון פיזית ואת הגודל שלה.



א. נניח כתובת של n ביטים, ולכן גודל זיכרון וירטואלי 2^n בתים
דרך א' ← אם גודל הדף הוא $4KB = 2^{12}$ אז צריך 12 ביטים ימניים ל-offset , 1 ביט שמאלי ל-MSB , n-13 ביטים למספר הדף
דרך ב' ← אם גודל הדף הוא $4KB = 2^{22}$ אז צריך 22 ביטים ימניים ל-offset, 1 ביט שמאלי ל-MSB , n-23 ביטים למספר הדף
מפה מבינים שמספר המסגרת בדרך א' מורכב מיותר ביטים ממספר הביטים בדרך ב' , ולכן בתוך טבלת הדפים מספר המסגרת יהיה מורכב ממספר ביטים שונה.

נגדיר את טבלת הדפים:
החלק הנמוך (כלומר עבור הביטים הנמוכים) הוא לדפי 4KB ולכן מספר הכניסות יהיה 2^{n-13}
החלק הגבוה (כלומר עבור הביטים הגבוהים) הוא לדפי 4MB .
מכאן שנגדיר 2 מצביעים לטבלה: מצביע PTE לחלק הנמוך, ומצביע לחלק הגבוה $PTE+2^{n-13}$

אלגוריתם הגישה לטבלת הדפים יעבוד כך:
נסתכל על הביט הגבוה ביותר MSB בכתובת פיזית עם m ביטים שקיבלנו:

אם MSB=0 ← אנחנו בחלק הנמוך של טבלת הדפים (דפי 4KB) ← ניגש למצביע PTB ולכניסה PTE ע"י לקיחה של n-13 ביטים ← בכניסה זו יהיה משמאל (בצהוב) ביטים כלשהם של תחזוקה, ביט אחד valid , ואז אם ביט valid שווה 1 אז מספר מסגרת m-12 ונשרשר לו את 12 הביטים של ה-offset.

אם MSB=1 ← אנחנו בחלק הנמוך של טבלת הדפים (דפי 4MB) ← ניגש למצביע ולכניסה PTE ע"י החישוב $PTE+2^{m-13}$ ← בכניסה זו יהיה משמאל (בצהוב) ביטים כלשהם של תחזוקה, ביט אחד valid , ואז אם ביט valid שווה 1 אז מספר מסגרת m-22 ונשרשר לו את 22 הביטים של ה-offset.

ונקבל M ביטים של הכתובת הפיזית לגישה ל-RAM.

אם valid=0 ← נקבל page fault ונטפל בו ע"י כניסה לדיסק.

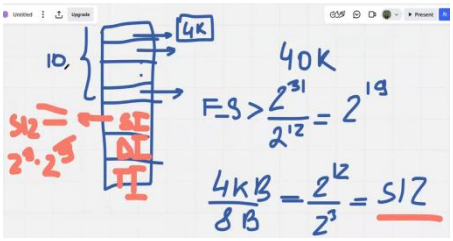
ב. מה החיסרון בשיטת ההקצאה שתוארה בשאלה? איך ניתן לתקן אותה?

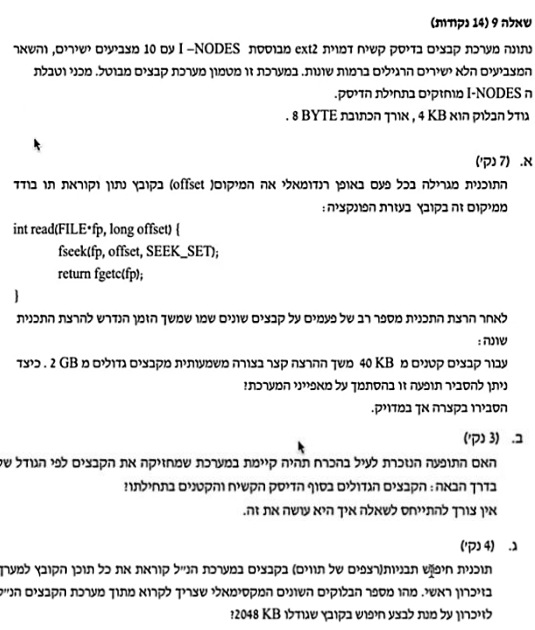
החיסרון הוא הפרגמנטציות שיתקבלו, שיגרמו לבזבוז מקום, כי אין מגוון של גדלי דפים שניתן להשתמש בהם אלא רק סוג דף בגודל מסוים עבור הקצאה גדולה/קטנה מ-8MB. אי אפשר לשלב בין גדלי דפים בהקצאות. זה יגרום מהר מאוד למצב שלא נוכל לקבל עוד הקצאות לזיכרון כי מקום הזיכרון על פניו יהיה תפוס כולו, אך בפועל לא יהיה מנוצל באופן מיטבי ויהיה עוד המון זיכרון פנוי שאין הרשאה להשתמש בו.

איך נוכל לתקן זאת? לאפשר גישה ל-2 סוגי הדפים בכל הקצאה ללא קשר לגודלה. זה ייתן לנו אופציה לשלב בין גדלי דפים, יצמצם פרגמנטציה וימקסם את ניצול הזיכרון. מה שכן זה יכול ליצור בעיה של alignment , מיקום בזיכרון לפי גבול שמתאים לגודל הנתון . למשל בדוגמא – עבור מערכת שמקונפגת לעבוד בקפיצות של 4M (מתחילים מכתובת 0, הכתובת הבאה 4M , הכתובת הבאה 8M וכו'...) אז יכול להיות שעבור דף שנשמר בצורה שלהלן (בכחול) נוצר מצב שעבור גישה אחת יש צורך לקרוא 2 דפים, וזה פחות טוב. במערכות אמיתיות מנסים שהגישות יהיו מסודרות לפי הקפיצות.



← **שאלה** – 2023א מועד 72 שאלה 9





שאלה 9 (14 נקודות)

נתונה מערכת קבצים בדיסק קשיח דמוית ext2 מבוססת I–NODES עם 10 מצביעים ישירים, והשאר המצביעים הלא ישירים הרגילים ברמות שונות. במערכת זו מטמון מערכת קבצים מבוטל. מכני וטבלת I-NODES מוחזקים בתחילת הדיסק.

גודל הבלוק הוא 4 KB , אורך הכתובת 8 BYTE .

א. (7 נק')

התוכנית מגרילה בכל פעם באופן רנדומאלי את המיקום(offset) בקובץ נתון וקוראת תו בודד ממיקום זה בקובץ בעזרת הפונקציה:

```
int read(FILE*fp, long offset) {
    fseek(fp, offset, SEEK_SET);
    return fgetc(fp);
}
```

לאחר הרצת התכנית מספר רב של פעמים על קבצים שונים שמו שמשך הזמן הנדרש להרצת התכנית שונה:

עבור קבצים קטנים מ 40 KB משך ההרצה קצר בצורה משמעותית מקבצים גדולים מ 2 GB . כיצד ניתן להסביר תופעה זו בהסתמך על מאפייני המערכת?

הסבירו בקצרה אך במדויק.

ב. (3 נק')

האם התופעה הנזכרת לעיל בהכרח תהיה קיימת במערכת שמחזיקה את הקבצים לפי הגודל שלהם בדרך הבאה: הקבצים הגדולים בסוף הדיסק הקשיח והקטנים בתחילתו?

אין צורך להתייחס לשאלה איך היא עושה את זה.

ג. (4 נק')

תוכנית חיפוש תבניות(רצפים של תווים) בקבצים במערכת הנייל קוראת את כל תוכן הקובץ למערך בזיכרון ראשי. מהו מספר הבלוקים השונים המקסימאלי שצריך לקרוא מתוך מערכת הקבצים הנייל לזיכרון על מנת לבצע חיפוש בקובץ שגודלו 2048 KB?

מדובר כאן על הזמן הממוצע בגישה לבלוק.

אז i-node נמצא בדיסק (ולא ב-RAM), כשבהתחלה שלו 10 בלוקים בגישה ישירה לכתובות לדיסק שם כל בלוק הוא 4KB.

בקבצים שגדולים מ-2GB , גודל הקובץ בבלוקים > $2^{19} = 2^{12} / 2^{31}$ כלומר יש בקבצים הללו מינימום 2^{19} בלוקים.

מבחינת מספר הכניסות בבלוק אחד יש סה"כ $4KB / 8B = 2^{12} / 2^3 = 2^9 = 512$. מכאן נבין שעל מנת לכסות את 2^{19} הבלוקים שיש מינימום בקובץ נצטרך גישה לכל הפחות ל-TI ב-inode מכיוון שיש שם סה"כ אופציה ל-2^9 2^9 2^9 = 2^{27} שזה מכסה קובץ (ברמה הקודמת DI כיסינו עד $2^9 2^9 = 2^{18}$ וזה לא הספיק...).

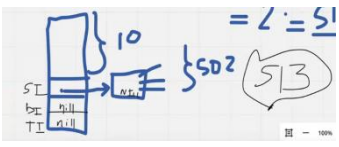
לכן, יש המון גישות שצריך לעשות בשביל קובץ בגודל 2GB וזה זמן יקר שמתבזבז , מה שמסביר את הזמנים הגבוהים שהתקבלו.

ב. בדיסק HDD (העגול), יש בכל פלטה סיגמנטים. הוא כל הזמן זז ומסתובב וכך מתאפשרת קריאה ממנו. בקבצים גדולים יש יותר סיגמנטים, ולכן הם ממוקמים במעגלים החיצוניים של הדיסק כי הרדיוס הפיזי גדול יותר. ואז הדרך המתוארת תהיה קיימת וזה היה משפר. אבל ב- SDD כבר אין את המבנה המכאני הזה אז זה לא משפר.

ג. עבור גודל קובץ $2^{21}B = 2048KB$, גודל הקובץ בבלוקים הוא $2^{21} / 2^{12} = 2^9 = 512$. נחשב את כמות הבלוקים המקסימלית:

10 בלוקים ישירים הראשונים, ונשארו 502 בלוקים לכסות.

ב-SI יש בלוק שמעביר ל-512 כניסות מהם מגיעים לבלוקים, וזה כבר מכסה. אז ניקח את הבלוק הראשון בחשבון ועוד 502 בלוקים וסה"כ יחד עם ה-10 ממקודם: 502+1+10=513 .



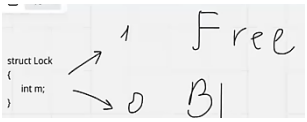
← **שאלה** – 2025א מועד א2 שאלה 7

א. פעולה אטומית – כשהיא מתחילה היא רצה ברצף ולא יורדת מה-CPU עד שהיא מסתיימת. אף אחד לא נכנס לה באמצע.

נגדיר m מספר שלם:

0 – המנעול סגור

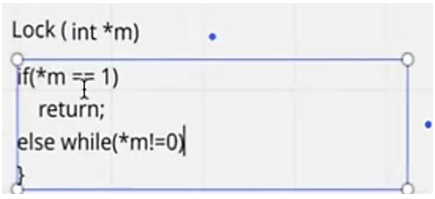
1 – המנעול פתוח



מימוש נעילת המנעול:

אם נעשה מימוש של תנאי ולולאה שבה תתרחש המתנה, זה פתרון לא טוב בגלל ה-Context Switch שאם הוא מתרחש באמצע התהליך הוא משבש את הכל.

דוג' לא טובה:



לבן, מה שנעשה זה מימוש ע"י ספין־לוק (Spin Lock) שהוא מנגנון סנכרון שמונע מכמה תהליכים/תהליכונים להיכנס בו-זמנית לקטע קריטי. איך? במקום להירדם כשאין גישה למשאב, התהליכון מסתובב בלולאה (spin) ובודק שוב ושוב האם הנעילה השתחררה כבר.

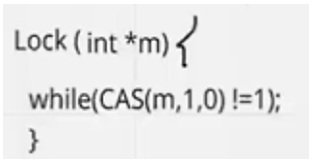
איך זה עובד?

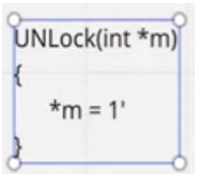
תהליכון מנסה לקחת את ה־lock :

אם ה־lock פנוי ← נכנס לקטע הקריטי

אם ה- lock תפוס ← נכנס ללולאת בדיקה רציפה עד שה־ lock משתחרר

לשם כך נעזר בפעולת ה-CAS האטומית הנתונה בשאלה, שה-Context Switch לא יכול להתרחש במהלכה.





- בנעילת המנעול, נעזר ב-CAS בתוך לולאת while שתעצור את התהליך בתוכה ב"ספין" עד שהמנעול ישתחרר.
- בשחרור המנעול, פשוט נהפוך את m ל-1, מצב פתוח.

ב. אם מספר תהליכונים משתמשים בקוד במקביל, אז ב-Context Switch הכל יכול להשתבש. כי ברגע שהיא מתרחשת, אז התהליך שנמצא ב"תרדמת" שומר את כל הערכים שהוא עצר איתם בזיכרון המקומי שלו. וכשהוא חוזר לעבוד הוא ממשיך עם הביצוע עם אותם ערכים. כך יכול לקרות למשל שהמשתנה במקום לגדול פעמיים ב-1 בגלל שני תהליכים שפעלו במקביל כדי להגדילו, יגדל לבסוף רק פעם אחת ב-1 כי התרחשה החלפת הקשר בין שני התהליכים הללו. זה מה שהיה קורה כשיש רק ליבת CPU אחת. אם יהיו מספר ליבות, אז לכל תהליך יש אוגר משלו ואז הסנכרון יעבוד.

ג. **Variable** זה המשתנה שנרצה לעדכן, **value** זה הערך שנוסיף לו (יכול גם להיות שלילי).

נגדיר משתנים: **expected** המשתנה הקודם variable, **new_val** המשתנה החדש שיתקבל לאחר ההוספה.

ואז התהליכון מגיע לולאה שעושה שימוש בפונקציית CAS האטומית, ואז מגיע הקטע הקריטי:

כל עוד איזשהו תהליך מבצע עדכון למשתנה אף תהליך אחר לא יכול להיכנס ולעדכן אותו גם, אלא צריך לחכות שהתהליך שבקטע הקריטי יסיים.

איך זה בא לידי ביטוי בתנאי? אם אף תהליכון אחר לא שינה את variable מאז שקראנו אותו, variable יתעדכן ל־new_val. אחרת נמתין.

```
change_atomic(variable, value)
{
    do{
      int expected = variable;
      int new_val = value + variable;
    }
    while(CAS(&variable, expected, new_val) != expected);
}
```

סעיף א. (10 נק')

נתונה מערכת מחשב שבה המעבד CPU תומך בפקודה אטומית CAS – COMPARE AND SWAP.

להלן פסודוקוד המתאר את פעולת הפקודה:

```
int compare _and_swap(int *value, int expected, int new_value) {
int temp = *value;
if (*value == expected) *value = new_value;
return temp; }
```

על בסיס הפקודה הזאות ממשו את נעילת המנעול (MUTEX) Lock(int * m) וממשו בדרך הפשוטה ביותר את שחרור המנעול Unlock(int * m).

המימוש חייב להבטיח את **מניעה הדדית והתקדמות**, אך **אינו צריך להבטיח המתנה מוגבלת** (העדר הרעבה).

סעיף ב. (7 נק')

קידום או הפחתת משתנה מתבצע ברמת החומרה בעזרת 3 פקודות(מימוש ++counter):

```
register1 = counter
register1 = register1 + 1
counter = register1
```

הסבירו איזה בעיה יכולה להתעורר בביצוע מקבילי של קידום/הפחתת משתנה משותף על ידי מספר תהליכונים? האם זה תלוי במספר ליבות CORES שיש במעבד?

סעיף ג. (8 נק')

בעזרת פקודת CAS המייצגת בפסודוקוד בסעיף א', ממשו קידום והפחתה אטומיים של משתה. ממשו את הפונקציה atomic_change(variable ,value) . ערך ה value יכול להיות חיובי או שלילי.

הערה: פרמטרים שני ושלישי של compare _and_swap יכולים להיות משתנה וביטוי אריתמטי, לדוגמא: CAS(&val, exp , new + 1) . בביטוי אריתמטי אפשר להשתמש גם במשתנה במקום המספר.